HANDBOOK

Combined Vocational Education Textbooks

Classes IX–XII

Data Entry • Web Development • Artificial Intelligence • Software Development

Combined volume of eight vocational textbooks

Contents

Domestic Data Entry Operator — Class IX **1**

Domestic Data Entry Operator — Class X **2**

Web Developer — Grade XI **3**

Web Developer — Grade XII **4**

Artificial Intelligence Assistant — Grade IX **5**

Artificial Intelligence Assistant — Grade X **6**

Junior Software Developer — Grade XI **7**

Junior Software Developer — Grade XII **8**

Domestic Data Entry Operator — Class IX

Domestic Data Entry Operator — Class X

Web Developer — Grade XI

Web Developer — Grade XII

Artificial Intelligence Assistant — Grade IX

Artificial Intelligence Assistant — Grade X

Junior Software Developer — Grade XI

Junior Software Developer — Grade XII

HANDBOOK

Domestic Data Entry Operator

Class IX Vocational Education

Information Technology and IT-Enabled Services

A textbook-style learning resource for Domestic Data Entry Operator

Data Entry • Digital Documents • Spreadsheets • Presentations • Digital Safety

Contents

1 Unit 1 – IT-ITeS and the Data Entry Workplace **1**
1.1 From facts to useful information . . . 1
1.2 What a data-entry operator may do . . . 1
1.3 Five dimensions of record quality . . . 1
1.4 A practical example: registration records . . . 2
1.5 The source of truth . . . 2
1.6 Lesson plan outline . . . 2

2 Unit 2 – Keyboarding, Accuracy and Workstation Practice **3**
2.1 A comfortable workstation . . . 3
2.2 Keyboard groups and familiarisation . . . 3
2.3 Build a practice sequence . . . 4
2.3.1 Accuracy before speed . . . 4
2.3.2 Simple recording sheet . . . 4
2.4 Common difficulties and coaching responses . . . 4
2.5 Feedback that motivates . . . 4

3 Unit 3 – Digital Documentation **5**
3.1 A document is more than typed words . . . 5
3.2 Demonstration: prepare a school notice . . . 5
3.3 Naming and saving files . . . 5
3.4 Formatting for readers . . . 5
3.5 Troubleshooting without panic . . . 6
3.6 What the trainer should observe . . . 6

4 Unit 4 – Spreadsheets and Data Quality **7**
4.1 Understanding a worksheet . . . 7
4.2 Enter data consistently . . . 7
4.3 Basic calculations and verification . . . 7
4.4 Find common data-quality issues . . . 7
4.5 Sorting, filtering and protecting the record . . . 8
4.6 A realistic classroom case . . . 8

5 Unit 5 – Digital Presentations **9**
5.1 Plan before opening the software . . . 9
5.2 Build a readable slide . . . 9
5.3 Use images responsibly . . . 9
5.4 Review from the audience’s seat . . . 9

6 Unit 6 – Lesson Planning and VT Microteaching **10**
6.1 Start with the learner’s outcome . . . 10
6.2 Plan the lesson sequence . . . 10
6.3 Microteaching in small steps . . . 10

6.4 Giving peer feedback . . . 10
6.5 Adapting when the plan changes . . . 11

7 Unit 7 – Assessment, Feedback and Learning Records 12
7.1 Match assessment to the skill . . . 12
7.2 Use a short, visible rubric . . . 12
7.3 Feedback while the task is fresh . . . 12
7.4 Keep records responsibly . . . 12

8 Unit 8 – Digital Safety, Privacy and Professional Conduct 13
8.1 Use fictional data for practice . . . 13
8.2 Account and device habits . . . 13
8.3 Email, links and unexpected requests . . . 13
8.4 Safe file sharing . . . 13
8.5 Professional conduct . . . 13

9 Planning a Master Trainer Workshop 14
9.1 A practical half-day structure . . . 14
9.2 Equipment and preparation checklist . . . 14
9.3 Adapting to local conditions . . . 14

10 Assessment Tools and Reusable Templates 15
10.1 Template: lesson preparation . . . 15
10.2 Template: practical observation . . . 15
10.3 Template: learner reflection . . . 16
10.4 Template: feedback record . . . 16

11 Practical Lab 1 – Enter and Verify a Small Record Set 18
11.1 Purpose and materials . . . 18
11.2 Procedure . . . 18
11.3 Evidence of learning . . . 18

12 Practical Lab 2 – Keyboarding with a Quality Check 19
12.1 Task . . . 19
12.2 Procedure . . . 19
12.3 Observation prompts . . . 19

13 Practical Lab 3 – Create, Save and Retrieve a Notice 20
13.1 Task brief . . . 20
13.2 Suggested work sequence . . . 20
13.3 Peer review checklist . . . 20

14 Practical Lab 4 – Spreadsheet Entry and Cell References 21
14.1 Starter data . . . 21
14.2 Tasks . . . 21
14.3 Trainer discussion . . . 21

15 Practical Lab 5 – Find and Report Data-Quality Issues 22
15.1 Case sheet . . . 22
15.2 Learner task . . . 22
15.3 Assessment focus . . . 22

16 Practical Lab 6 – Design a Short Presentation 23
16.1 Task brief . . . 23

16.2 Suggested structure 23
16.3 Review questions 23

17 Practical Lab 7 – Teach a Five-Minute Demonstration 24
17.1 Task brief 24
17.2 Observation rubric 24
17.3 Debrief 24

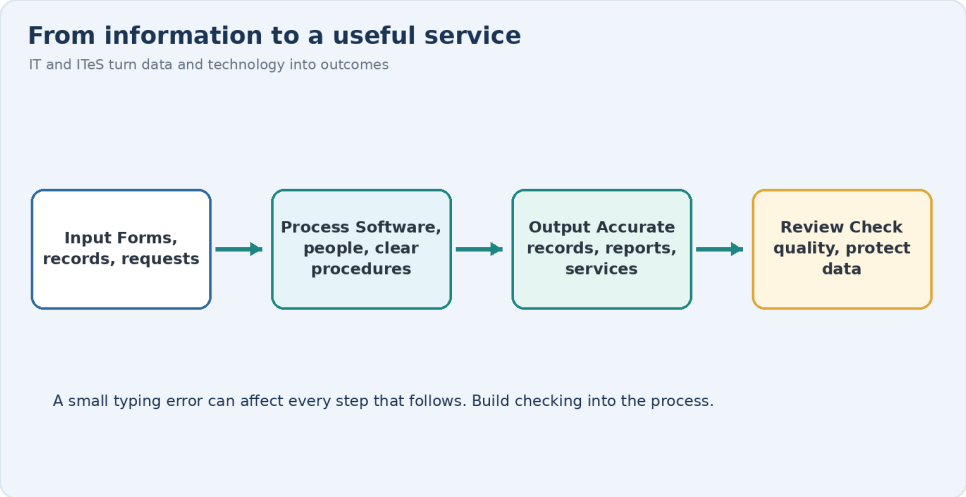
18 Practical Lab 8 – Make a Safe Data-Handling Decision 25
18.1 Scenarios 25
18.2 Expected decision process 25
18.3 Trainer note 25

Chapter 1

Unit 1 – IT–ITeS and the Data Entry Workplace

1.1 From facts to useful information

Data consists of recorded facts such as a name, date, quantity, identification number or attendance mark. Information is data arranged and interpreted so that it can answer a question. A list of daily attendance marks becomes more useful when it is organised by class and date and checked for missing entries. The computer stores and processes the entries, but a person still has to decide whether the information is complete and appropriate for its purpose.



1.2 What a data-entry operator may do

The work can include reading information from a form, entering it into a digital system, checking required fields, correcting errors against the source, updating an existing record, retrieving a record when authorised, and preparing a simple work report. The exact task depends on the organisation. In a school, a practice example might be an inventory list; in a clinic, records would be sensitive and require strict access controls. For classroom work, use fictional names and numbers.

1.3 Five dimensions of record quality

1. **Accuracy:** the entry matches the trusted source.
2. **Completeness:** required fields are present, or missing information is clearly flagged.
3. **Consistency:** the same kind of value is recorded in the same format.
4. **Timeliness:** the record is entered and updated when required.
5. **Traceability:** a reviewer can understand where the value came from and what was checked, without exposing information to unauthorised people.

These qualities can conflict with pressure to work quickly. A fast entry is not good work if a digit is wrong or a required field is guessed. Teach learners to maintain a steady pace and build checks into the workflow.

1.4 A practical example: registration records

Give pairs a fictional registration sheet containing eight records. Include a few deliberate issues: one missing phone number, a date written in a different format, one transposed digit in an ID, and one record with extra spaces. Ask learners to enter the records into a table and mark the issues they can identify. They must not invent a value to fill a blank. The correct response to missing source information is to flag it for clarification according to the agreed process.

1.5 The source of truth

When two values disagree, learners should not choose whichever looks more plausible. They should return to the designated source or ask the authorised supervisor. This habit matters because a plausible guess can become a false record that is copied into later reports. Demonstrate a clear correction note or flag where the local process allows it.

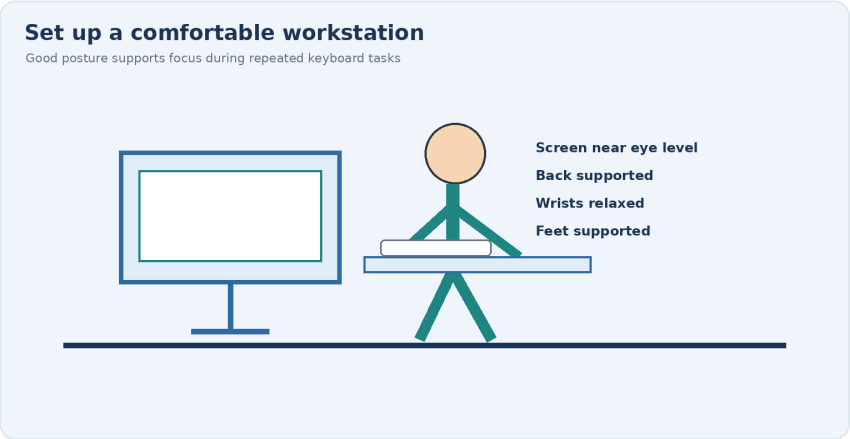
1.6 Lesson plan outline

Begin with examples of records learners recognise, such as a library book list or school equipment register. Ask what could go wrong if the data were inaccurate. Demonstrate one record from source to checked entry. Let pairs complete a short set and then compare against the source. End by asking each learner to name one check they will perform before submitting work.

Chapter 2

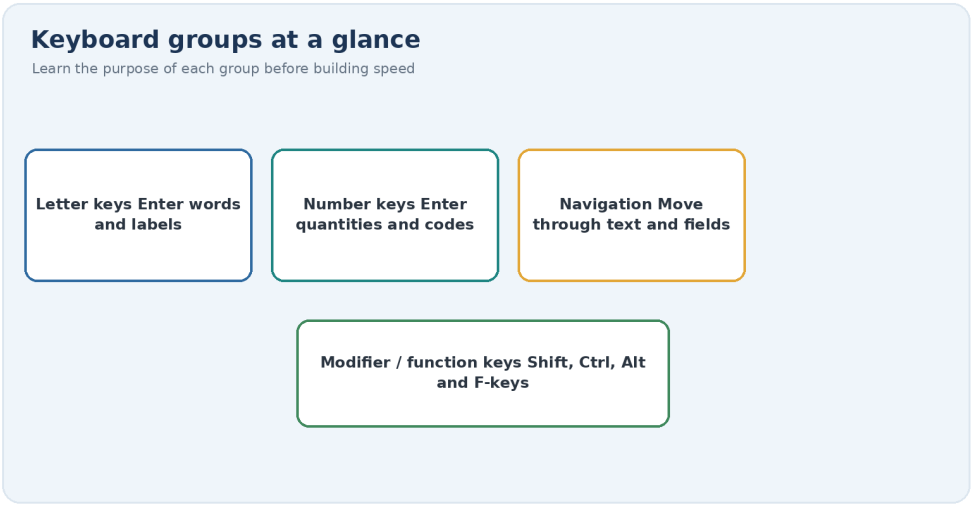
Unit 2 – Keyboarding, Accuracy and Workstation Practice

2.1 A comfortable workstation



Invite learners to adjust their chairs and screens so they can work without straining. Feet should be supported where possible, shoulders relaxed and wrists in a comfortable neutral position. The screen should be readable without leaning forward for long periods. There is no single posture that fits every body; the practical aim is to reduce unnecessary strain and allow regular changes of position.

2.2 Keyboard groups and familiarisation



Explain the letter keys, number row, numeric keypad where available, function keys, navigation keys and modifier keys such as Shift, Ctrl and Alt. The layout varies between keyboards, so learners should identify the controls on the equipment they actually use. Ask them to locate Backspace, Enter, Shift, Spacebar and the arrow keys before beginning a timed activity.

2.3 Build a practice sequence

A beginner benefits from short, focused practice. Start with familiar letter patterns, then common words, sentences, punctuation and numeric strings. Keep the source text visible and teach learners to compare the typed result with it. When a learner makes repeated errors, shorten the passage and return to the keys or combinations causing difficulty. Do not increase the time pressure just because the class is progressing at different speeds.

2.3.1 Accuracy before speed

Speed is meaningful only when measured alongside accuracy and under comparable conditions. If two attempts use different text lengths or difficulty, their results cannot be compared fairly. Use the same duration, similar text and the same method for each attempt. Give feedback on one or two observable behaviours: for example, “You checked each line before moving on,” or “Your number entries were accurate when you paused to compare the source.”

2.3.2 Simple recording sheet

Attempt	Duration	Errors found	Next practice goal
1	2 min		
2	2 min		
3	2 min		

2.4 Common difficulties and coaching responses

If a learner looks down at every key, reduce the task to a small set of keys and practise slowly. If the learner types quickly but misses characters, ask them to pause at the end of each line and compare. If they press the wrong modifier key, demonstrate the combination slowly and let them repeat it several times. If discomfort appears, stop and adjust the setup rather than asking the learner to push through it.

2.5 Feedback that motivates

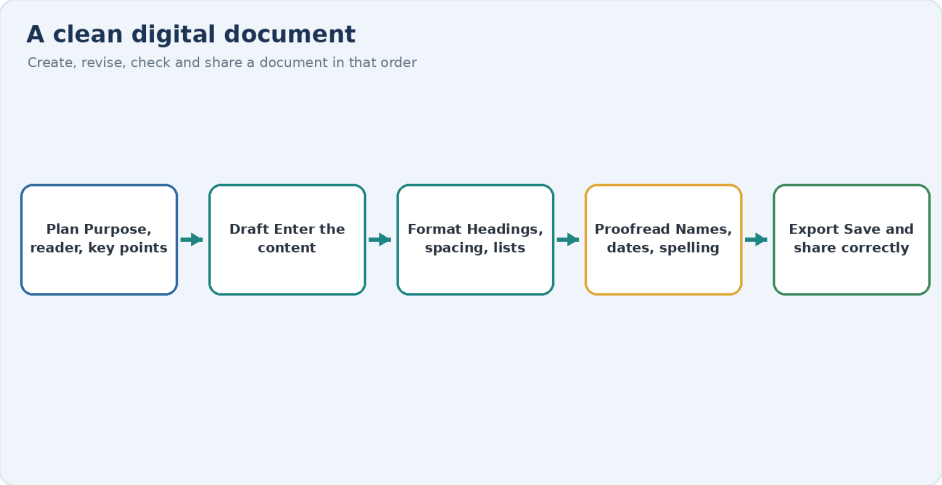
Use evidence instead of labels. “You made six errors” is information, but it is not a plan. A more useful comment is: “Four errors were in the number sequence. For the next attempt, read each group of three digits, enter it, and compare it before continuing.” Make the next step small enough to be achievable.

Chapter 3

Unit 3 – Digital Documentation

3.1 A document is more than typed words

A useful document has a clear purpose, readable structure and a reliable saved copy. Learners should be able to create a new file, enter text, correct mistakes, format headings, save in the correct folder, close the document and reopen it. The precise menu names differ across word processors, but the sequence remains useful across applications.



3.2 Demonstration: prepare a school notice

Use a fictional school event. Include a heading, date, venue, a short explanation, instructions and a contact role rather than a real phone number. First discuss what the reader needs to know. Type the content, check spelling and punctuation, make the heading distinct, keep body text readable and use spacing consistently. Avoid using multiple spaces to force text into position; use the application’s alignment and paragraph controls.

3.3 Naming and saving files

A filename should help another person identify the document without opening several files. A practical pattern is `Class9_EventNotice_Draft1`. Use a name that suits the school’s agreed convention and avoid vague names such as `newfile` or `finalfinal`. Save in the required folder. Then close and reopen the file; this confirms that the document was saved where expected.

3.4 Formatting for readers

Use headings to show structure, not just to decorate. Keep font choices limited, use a consistent hierarchy, and leave enough white space for the eye to follow the page. Check that the document remains readable when viewed at ordinary screen size or printed. Colour should support meaning and should not be the only way information is distinguished.

3.5 Troubleshooting without panic

If a file seems to have disappeared, first check the current folder, recent files list and filename. Avoid creating several copies immediately, because this can make the situation more confusing. If formatting changes unexpectedly, undo the last action if appropriate and inspect paragraph or style settings. If the program closes unexpectedly, follow the application’s recovery process and then save a clearly named copy.

3.6 What the trainer should observe

Look for a sensible sequence, not just a polished page. Can the learner explain why they chose the filename? Did they check the source text? Did they save in the correct location? Can they retrieve the file without assistance? These observations show whether the skill is likely to transfer to a new task.

Chapter 4

Unit 4 – Spreadsheets and Data Quality

4.1 Understanding a worksheet

Reading a spreadsheet

A cell is identified by its column letter and row number

	A	B	C	D
1	Item	Qty	Rate	Total
2	Notebook	3	20	=B3*C3
3				
4				

Example: D3 contains a formula that multiplies quantity in B3 by rate in C3.

A worksheet organises information in rows and columns. A cell is identified by its column letter and row number, such as B3. A range is a group of cells. Before entering data, learners should identify the purpose of each column and use clear headings. One row usually represents one record; each column should hold one type of information. This makes the table easier to check, sort and summarise.

4.2 Enter data consistently

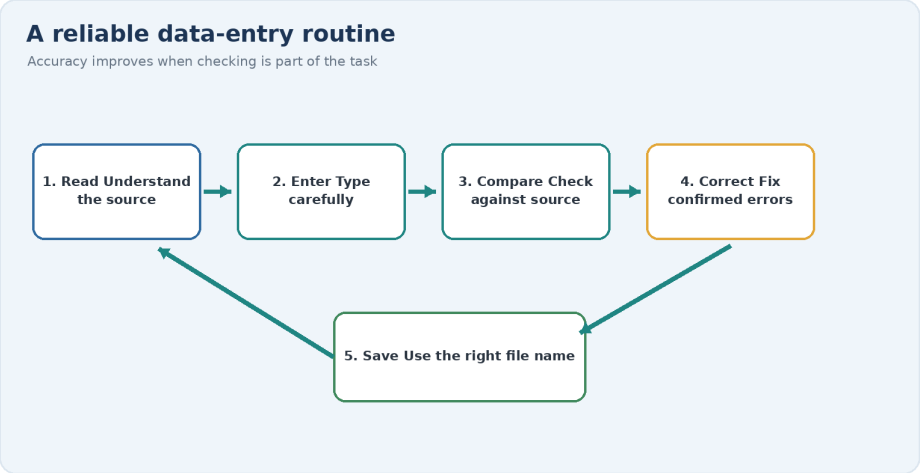
Use fictional inventory data for a school computer lab. Suggested columns are Asset ID, Item, Quantity, Condition and Last Checked. Show how to enter one row, then ask learners to complete the rest. Dates should use a consistent format. Quantity should be entered as a number, not as a mixture of words and digits. Required values should be checked against the source, not guessed.

4.3 Basic calculations and verification

Where the prescribed learning outcomes include formulas, demonstrate a simple total using a small number of values. Explain the range included in the formula and verify the result independently. A formula can produce a plausible answer even when it points to the wrong cells. Test the formula with a small example or recalculate the total manually before trusting it.

4.4 Find common data-quality issues

Typical issues include duplicate identifiers, blank required fields, inconsistent spelling, mixed date formats, numbers stored as text and values that do not fit the expected range. Give learners a worksheet with deliberate errors. Ask them to locate each issue, explain why it matters and identify the correct source for resolution. Do not reward learners for silently filling in missing values from guesswork.



4.5 Sorting, filtering and protecting the record

If sorting or filtering is part of the prescribed software practice, show the class how to select the full table before sorting. Sorting only one column can detach values from their records. Use a copy of the practice file before experimenting. Ask learners to check that each ID remains with its original item after the sort. Save a new version when the activity calls for a substantial change.

4.6 A realistic classroom case

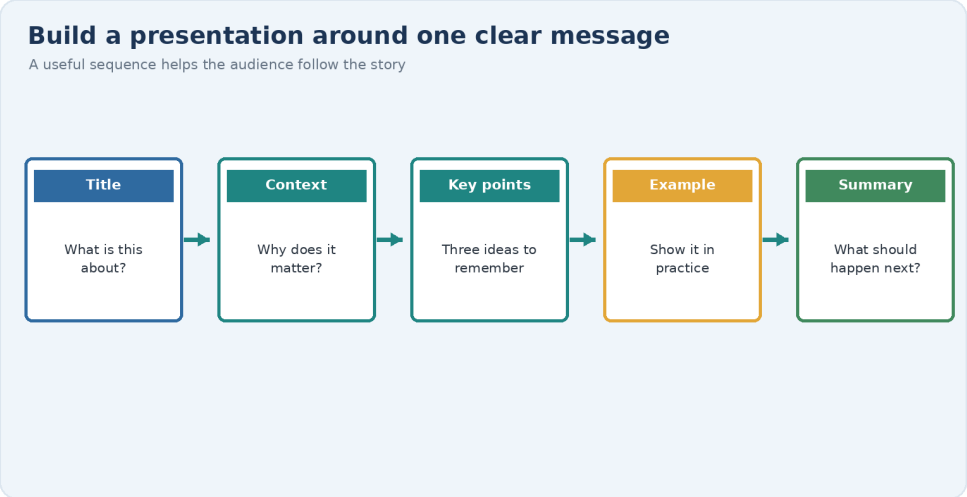
A lab assistant gives the class an inventory sheet with ten assets. Two assets have the same ID, one quantity is blank, and three dates use different formats. Learners should first compare the worksheet with the original sheet, mark the issues and write a question for the supervisor about the missing quantity. They then correct only the errors supported by the source. The class finishes by explaining why the duplicate ID is more serious than a minor spacing inconsistency.

Chapter 5

Unit 5 – Digital Presentations

5.1 Plan before opening the software

A presentation should help an audience understand a message. Before creating slides, ask: who will listen, what should they understand, and what should they do with the information? A short classroom presentation usually needs a beginning, a small number of supporting points and a clear ending. Adding slides is not the same as adding useful information.



5.2 Build a readable slide

Use a clear title, short phrases and a visual that supports the message. Keep alignment and typography consistent. If the text is too small to read from the back of the room, reduce the content or split it across slides. Avoid unnecessary transitions and decorative effects that compete with the information. A simple slide that the audience can follow is usually more effective than a crowded one.

5.3 Use images responsibly

Choose visuals that are relevant and suitable for the audience. Credit or follow the reuse conditions of external images when required. Do not include a photograph of a student, customer or colleague without the appropriate permission. For practice, use original shapes, approved school assets or clearly reusable images.

5.4 Review from the audience’s seat

Run the slideshow and check it in presentation mode. Ask a peer to sit at a distance and report which words are difficult to read. The presenter should practise explaining the slide rather than reading every word. Feedback should be concrete: “The diagram explains the sequence well, but the last line is too small,” is more useful than “Make it better.”

Chapter 6

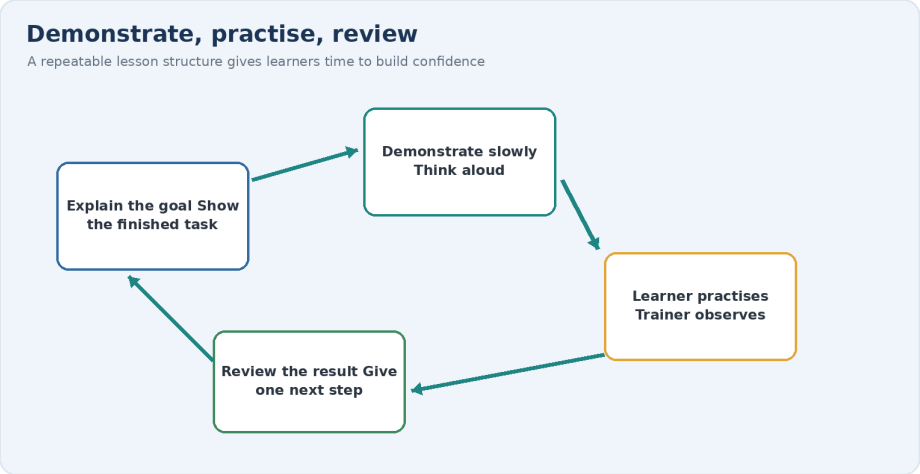
Unit 6 – Lesson Planning and VT Microteaching

6.1 Start with the learner’s outcome

A lesson outcome should describe what the learner can demonstrate by the end. “Understand spreadsheets” is too broad to observe. “Enter six fictional inventory records, identify a duplicate ID and explain how it should be resolved” gives the trainer and learner a shared target. Choose an outcome that fits the available time and equipment.

6.2 Plan the lesson sequence

A practical session can follow this structure: connect to a familiar situation, state the outcome, demonstrate the task, guide the first attempt, provide independent practice, check the work and close with reflection. Do not spend the entire period explaining. Learners need enough time to handle the keyboard, make decisions and recover from ordinary mistakes.



6.3 Microteaching in small steps

Ask each VT to teach a short section to peers acting as learners. The task should be narrow enough to demonstrate in five to eight minutes. The observer records whether the outcome was clear, the steps were visible, the explanation used accessible language, learners were invited to practise, and the final check matched the outcome. The purpose is rehearsal, not performance theatre.

6.4 Giving peer feedback

Use a simple structure: one thing that helped learning, one point that could be clearer, and one practical suggestion. Base comments on observed behaviour. “You showed how to save and then reopened the file to verify it” identifies a useful practice. “You are a natural teacher” may be kind, but it does not tell the VT what to repeat next time.

6.5 Adapting when the plan changes

If computers are unavailable, use printed screenshots or a step-card sequence to discuss decisions, then schedule hands-on practice as soon as possible. If the class needs more time, preserve the core outcome and shorten optional extension work. If the activity is too easy, add a realistic checking challenge rather than merely increasing the amount of typing.

Chapter 7

Unit 7 – Assessment, Feedback and Learning Records

7.1 Match assessment to the skill

If the outcome is to create and save a document, a written quiz alone is not enough. Ask the learner to perform the task and show the saved file. If the outcome is to identify data-quality problems, provide a small table and ask the learner to mark the issues and explain the next step. Evidence should reflect the actual skill being taught.

7.2 Use a short, visible rubric

A rubric should describe observable performance. Avoid vague criteria such as “good work” or “excellent understanding.” For a document task, criteria might include: required content present, readable formatting, meaningful filename, correct save location and successful retrieval. Explain the criteria before practice so learners know what quality looks like.

Criterion	Independent	Needs another practice
Task sequence	Completes steps in a sensible order	Misses or repeats important steps
Accuracy	Checks output against the source	Several unchecked mismatches remain
File handling	Saves and retrieves the correct file	Cannot locate the saved result
Explanation	Can explain at least one quality check	Cannot yet explain how correctness was checked

7.3 Feedback while the task is fresh

Feedback works best when it is prompt, specific and possible to act on. Name the observed behaviour, explain its effect and agree on the next step. For example: “You checked the first five rows, but the final two were not compared with the source. Before submitting the next sheet, scan every row and mark the ones you have verified.” Avoid comparing one learner’s result with another learner’s result when the useful comparison is with the task criteria.

7.4 Keep records responsibly

Record only the information needed for teaching and assessment. Store records in the authorised place and share them only with people who have a legitimate reason to see them. Follow the school’s retention and reporting requirements. Do not put learner scores or personal details in public chat groups or on an open display. If an error is found in a record, correct it through the agreed process and preserve any required audit trail.

Chapter 8

Unit 8 – Digital Safety, Privacy and Professional Conduct

8.1 Use fictional data for practice

A classroom exercise rarely needs real addresses, phone numbers, identification numbers or marks. Use clearly fictional data instead. If a real record must be used under an approved school process, follow the required permissions, access controls and storage rules. Never copy sensitive information into a personal device or unapproved cloud service merely because it is convenient.

8.2 Account and device habits

Use individual accounts where available, keep passwords private and lock the screen when leaving a shared computer. Do not ask learners to reveal their passwords to prove they can sign in. Follow the institution’s process for password resets. Keep software and security controls in place as directed by the responsible administrator, and do not install unapproved applications on school devices.

8.3 Email, links and unexpected requests

Pause before opening an unexpected attachment or entering a password after following a link. Check the sender and the context, and use a known route to contact the organisation if a request seems unusual. A message that creates urgency or asks for confidential data deserves extra care. Teach learners to report suspicious messages rather than forward them to classmates.

8.4 Safe file sharing

Before sharing a file, check the recipient, the contents and the permission level. A document intended for one teacher should not be placed in a folder that everyone can access. Use approved storage and remove access when it is no longer required, following the local procedure. Avoid sending sensitive records to a personal email account.

8.5 Professional conduct

Data-entry work involves trust. A professional operator does not guess missing information, share records out of curiosity, hide an error, or use information for personal reasons. If something seems wrong, stop and ask the authorised person. Explain to learners that reporting an error early is usually safer than quietly trying to conceal it.

Chapter 9

Planning a Master Trainer Workshop

9.1 A practical half-day structure

The schedule below is a sample, not a mandatory timetable. Adjust it to the local programme, the number of participants and the available equipment. Keep time for participants to practise; a workshop that contains only presentations will not show whether VTs can teach the tasks.

Time	Focus	Evidence or output
09:30–09:45	Welcome and outcomes	Participants state one goal and one question
09:45–10:15	Model a data-entry task	Shared list of quality checks
10:15–10:45	Hands-on practice	Completed fictional record sheet
10:45–11:00	Break	–
11:00–11:35	Document or spreadsheet task	Saved file and peer review
11:35–12:10	Lesson planning	Draft lesson outcome and sequence
12:10–12:40	Microteaching	Observation checklist and feedback
12:40–13:00	Reflection and next steps	Individual action commitment

9.2 Equipment and preparation checklist

- ☐ Confirm the room, power and seating.
- ☐ Test the display and every software activity.
- ☐ Prepare a starter file and a clean backup.
- ☐ Use fictional sample data only.
- ☐ Print task sheets, observation forms and reflection prompts.
- ☐ Plan an offline alternative for any activity that does not truly require the internet.
- ☐ Check that every participant will have hands-on time.

9.3 Adapting to local conditions

If the room has fewer computers than participants, use rotating roles: operator, observer and quality checker. Rotate roles so each person gets hands-on time. If the internet is unreliable, use locally saved files and demonstrate offline tasks. If participants have different levels of experience, provide a core task for everyone and an extension task for those who finish early. Keep the shared learning outcome consistent.

Chapter 10

Assessment Tools and Reusable Templates

10.1 Template: lesson preparation

Planning field	Notes
Lesson topic	
Observable learning outcome	
Prior knowledge to connect with	
Equipment and sample files	
Demonstration steps	
Learner practice task	
Evidence of learning	
Support or adaptation needed	
Reflection after the lesson	

10.2 Template: practical observation

Observable behaviour	Observed?	Evidence / next step
Reads the task instructions before starting		
Uses the correct source information		
Enters information carefully		
Checks required fields and likely errors		
Saves work in the correct location		
Can retrieve and explain the saved work		
Handles records and accounts responsibly		

10.3 Template: learner reflection

1. What task did I complete today?
2. Which step did I complete independently?
3. What error or difficulty did I notice?
4. How did I check whether my work was correct?
5. What will I practise next?

10.4 Template: feedback record

Field	Record

Task observed
What went well
Evidence noticed
One priority for improvement
Agreed practice step
Review date or next check

Chapter 11

Practical Lab 1 – Enter and Verify a Small Record Set

11.1 Purpose and materials

This lab gives VTs a complete example of the work cycle: read the source, enter the values, compare the result and report anything that cannot be resolved. Prepare a printed source sheet and a blank table. The example below uses fictional data only.

Record ID	Item	Quantity	Condition
LAB-101	Keyboard	12	Good
LAB-102	Mouse	14	Good
LAB-103	Monitor	8	Check required
LAB-104	Headset	6	Good
LAB-105	USB hub		Good

11.2 Procedure

Ask learners to enter the records exactly as supplied. They should not fill the blank quantity for LAB-105 from guesswork. They should preserve the note “Check required” for LAB-103 and identify it as a follow-up item. After entry, learners compare every field with the source. A partner then repeats the check independently and initials the practice sheet.

11.3 Evidence of learning

The learner has entered each supplied value accurately, has not invented the missing quantity, has flagged the follow-up item and can explain why the second check is useful. The trainer should ask the learner to point to the original source when explaining a correction.

Chapter 12

Practical Lab 2 – Keyboarding with a Quality Check

12.1 Task

Use this short passage for a timed practice. The aim is not to beat another learner. The aim is to produce an accurate copy and notice which kinds of characters need extra attention.

> The computer lab has 18 keyboards, 16 mice and 12 monitors. The equipment register was checked on 14 September. Two items need follow-up, and the final report must be saved before the end of the class.

12.2 Procedure

Give learners a short, unhurried first attempt. They should compare the result with the source and mark errors. Ask them to practise the sentence or number sequence where errors occurred, then repeat the task. Keep the conditions similar for both attempts. Do not use the score to rank learners publicly.

12.3 Observation prompts

- Did the learner keep the source text visible?
- Did they use punctuation and capital letters as shown?
- Did they check the finished text rather than assume it was correct?
- Did the second attempt show a specific improvement?

Chapter 13

Practical Lab 3 – Create, Save and Retrieve a Notice

13.1 Task brief

Create a one-page notice for a fictional “Digital Records Practice Day.” Include the date, venue, purpose, three instructions and a contact role. Do not use real personal phone numbers or private information.

13.2 Suggested work sequence

1. Plan the information the reader needs before opening the word processor.
2. Create a new document and type the content.
3. Make the title easy to identify and keep body text readable.
4. Check spelling, dates and punctuation against the task brief.
5. Save the file in the designated practice folder with a meaningful name.
6. Close the document, reopen it and confirm that the saved version is correct.

13.3 Peer review checklist

- ☐ The heading describes the purpose.
- ☐ The date and venue are present.
- ☐ Instructions are short and clear.
- ☐ Formatting is consistent and readable.
- ☐ The filename and folder make sense.
- ☐ The file can be reopened.

Chapter 14

Practical Lab 4 – Spreadsheet Entry and Cell References

14.1 Starter data

Enter the following fictional equipment records into a worksheet. Use one row per item and one column per field.

Asset ID	Item	Quantity
A-01	Keyboard	12
A-02	Mouse	14
A-03	Monitor	8
A-04	Headset	6
A-05	Webcam	5

14.2 Tasks

First enter the data and adjust column widths so the headings are readable. Identify the cell containing the quantity for the monitor. Identify the range containing all five quantities. If simple formulas are part of the prescribed syllabus, calculate the total quantity and check it independently by adding the five values manually. The expected total for this sample is 45.

14.3 Trainer discussion

Ask learners why headings matter, why each record should remain on one row, and how a sort could damage the table if only the Item column were selected. The purpose is to make the structure visible before introducing more complicated operations.

Chapter 15

Practical Lab 5 – Find and Report Data-Quality Issues

15.1 Case sheet

A fictional inventory export contains these records:

ID	Item	Qty	Checked on
A-01	Keyboard	12	14-09-2026
A-02	Mouse	14	14/09/2026
A-02	Monitor	8	14-09-2026
A-04	Headset		14-09-2026
A-05	Webcam	5	14-09-2026

15.2 Learner task

Identify the duplicate ID, the blank quantity and the inconsistent date formats. For each issue, state whether it can be corrected from the available source. Do not change A-02 on the third row until the original inventory source confirms the correct identifier. Do not fill the missing quantity until a reliable source provides it. The dates can be standardised only after the required format has been agreed.

15.3 Assessment focus

The trainer should listen for reasoning. A learner who spots an error but invents a replacement has not completed the full quality-control task. A strong response describes the issue, protects the record from an unsupported change and identifies the next person or source to consult.

Chapter 16

Practical Lab 6 – Design a Short Presentation

16.1 Task brief

Prepare three slides for new students on the topic “Three Checks Before Submitting a Digital Record.” The audience is a Class IX group who are new to data entry. Keep the wording simple and use one visual only if it helps explain the point.

16.2 Suggested structure

1. Slide 1: title and why checking matters.
2. Slide 2: compare with the source, check required fields, look for unusual values.
3. Slide 3: a small fictional error and the correct response.

16.3 Review questions

Can a learner at the back of the room read the text? Does each slide have one clear purpose? Does the visual support the message? Can the presenter explain the point without reading every line? Ask a peer to give one strength and one actionable suggestion.

Chapter 17

Practical Lab 7 – Teach a Five-Minute Demonstration

17.1 Task brief

Teach a partner how to save a document and retrieve it. Before starting, state what the learner should be able to do at the end. Use a short demonstration, then let the partner repeat the task while you observe.

17.2 Observation rubric

Evidence	Yes / not yet	Comment
Outcome stated clearly		
Steps shown at a learner-friendly pace		
Learner gets hands-on practice		
Trainer asks learner to verify the result		
Feedback names a useful next step		

17.3 Debrief

Ask the partner what was clear and where they hesitated. The trainer should not defend every choice. Select one adjustment to try in the next demonstration, such as showing the folder path more clearly or asking the learner to explain the retrieval step.

Chapter 18

Practical Lab 8 – Make a Safe Data-Handling Decision

18.1 Scenarios

1. A shared computer is left open with a learner record on screen.
2. A colleague asks you to send a list of phone numbers to a personal email address.
3. A spreadsheet has a blank required field and the deadline is near.
4. An unfamiliar attachment asks you to sign in urgently.

18.2 Expected decision process

For each scenario, learners should name the risk, take the immediate safe action and identify the authorised reporting route. Lock the unattended screen according to local practice. Verify whether the request for personal data is authorised before sharing anything. Flag the blank field instead of guessing. Do not enter credentials through an unexpected link; verify the request using a trusted route and report suspicious messages according to policy.

18.3 Trainer note

Avoid asking participants to disclose real passwords, real incidents or confidential examples in front of the group. The aim is to practise a safe decision, not to expose someone or investigate an incident in the training room.

HANDBOOK

Domestic Data Entry Operator

Class X Vocational Education

Digital Documentation • Spreadsheets • Databases

A textbook-style study resource with examples, diagrams and questions

Contents

1 Introduction to Styles **1**
1.1 Why styles matter . . . 1
1.2 Main style categories . . . 1
1.3 Applying a built-in style . . . 1
1.4 Creating and updating styles . . . 1
1.5 Templates and page layout . . . 2
1.6 Common errors . . . 2
1.7 Chapter questions . . . 2

2 Working with Images **3**
2.1 Choosing the right image . . . 3
2.2 Insert, size and position . . . 3
2.3 Cropping and visual correction . . . 3
2.4 Captions and accessibility . . . 3
2.5 Image layout checklist . . . 4
2.6 Chapter questions . . . 4

3 Advanced Features of Writer **5**
3.1 Table of contents . . . 5
3.2 Templates and document properties . . . 5
3.3 Track changes and review . . . 5
3.4 Compare, protect and export . . . 5
3.5 Practice scenario . . . 5
3.6 Chapter questions . . . 5

4 Analyse Data Using Scenarios and Goal Seek **6**
4.1 What-if analysis . . . 6
4.2 Build a small model . . . 6
4.3 Using Goal Seek . . . 6
4.4 Check the model . . . 6
4.5 Chapter questions . . . 7

5 Using Macros in a Spreadsheet **8**
5.1 Purpose of a macro . . . 8
5.2 Recording and reviewing . . . 8
5.3 Security and trust . . . 8
5.4 Maintainable automation . . . 8
5.5 Chapter questions . . . 8

6 Linking Spreadsheet Data **10**
6.1 Why link data? . . . 10
6.2 References between sheets . . . 10
6.3 External links and refreshing . . . 10

6.4 Practical case . . . 10
6.5 Chapter questions . . . 10

7 Share and Review a Spreadsheet 11
7.1 Collaboration and review . . . 11
7.2 Prepare a workbook for another user . . . 11
7.3 Comments, changes and versioning . . . 11
7.4 Final review checklist . . . 11
7.5 Chapter questions . . . 11

8 Introduction to Database Management System 12
8.1 From files to a database . . . 12
8.2 Tables, fields and records . . . 12
8.3 Keys and relationships . . . 12
8.4 Database models . . . 12
8.5 Chapter questions . . . 13

9 Starting with LibreOffice Base 14
9.1 Database objects . . . 14
9.2 Creating a new database . . . 14
9.3 Data types and validation . . . 14
9.4 Enter and inspect records . . . 14
9.5 Chapter questions . . . 14

10 Working with Multiple Tables 15
10.1 Avoiding repeated facts . . . 15
10.2 Relationships and integrity . . . 15
10.3 Plan tables before creating them . . . 15
10.4 Chapter questions . . . 15

11 Queries in Base 16
11.1 What a query does . . . 16
11.2 Criteria and sorting . . . 16
11.3 Queries involving several tables . . . 16
11.4 Queries and privacy . . . 16
11.5 Chapter questions . . . 16

12 Forms and Reports 18
12.1 Forms for data entry . . . 18
12.2 Reports for presentation . . . 18
12.3 Designing and checking . . . 18
12.4 Chapter questions . . . 18

13 Health, Safety and Security at Workplace 19
13.1 Recognising workplace risks . . . 19
13.2 Ergonomics and eye comfort . . . 19
13.3 Stress and work organisation . . . 19
13.4 Electrical and physical safety . . . 19
13.5 Information security at a workstation . . . 20
13.6 Chapter questions . . . 20

14 Workplace Quality Measures 21
14.1 Quality is built into the process . . . 21

14.2 Checks before submission 21
14.3 Documenting a problem 21
14.4 Continuous improvement 21
14.5 Chapter questions 21

15 Prevent Accidents and Emergencies **23**
15.1 Preparedness before an incident 23
15.2 During evacuation 23
15.3 Fire and electrical incidents 24
15.4 Injuries and first aid 24
15.5 After the event 24
15.6 Chapter questions 24

A Revision Questions **25**
A.1 Multiple-choice practice 25
A.2 Short-answer practice 25

B Answer Key for Revision Questions **26**
B.1 Multiple-choice answers 26
B.2 Short-answer marking guidance 26

Chapter 1

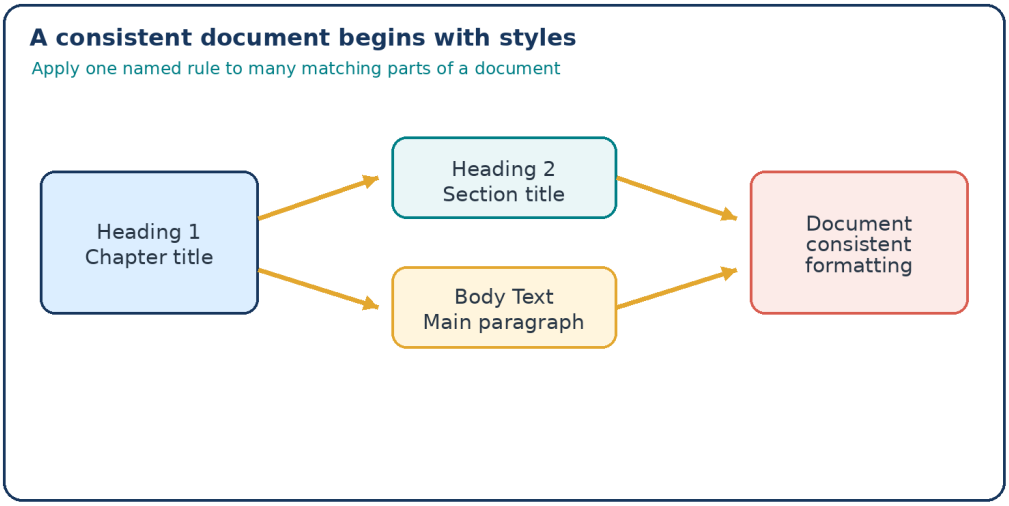
Introduction to Styles

1.1 Why styles matter

A long document is easier to maintain when similar parts share the same formatting. A heading style can keep chapter titles consistent; a body-text style can keep paragraphs readable; and a caption style can give every figure the same treatment. Manual formatting changes one selection at a time. Styles record a set of choices so the same visual rule can be applied repeatedly and updated centrally. A style is a named collection of formatting properties. Consistent styles improve readability and make automatic navigation features possible.

1.2 Main style categories

Paragraph styles control complete paragraphs, including spacing and alignment. Character styles format selected words or characters. Page styles govern the page layout, margins, orientation, headers and footers. Frame styles format containers that may hold text or pictures. List styles control bullets and numbering, while table styles organise the appearance of tables. Use the category that matches the item being formatted.



1.3 Applying a built-in style

Open a document and select the heading that needs formatting. Open the Styles panel from the menu or use the keyboard shortcut available in the installed version. Choose the paragraph-style category and apply an appropriate heading style. Repeat for other headings instead of formatting each one independently. For a longer report, check several pages to confirm that headings of the same level look alike.

1.4 Creating and updating styles

To create a custom style, first format a sample paragraph as required, then use the option to create a style from the selection. Give the style a meaningful name, such as `SectionHeading`. If a style already exists but needs a change, adjust a representative

paragraph and use the update-style command. Test the change on more than one section before relying on it throughout a document.

1.5 Templates and page layout

A template is a starting document that contains reusable layout and style choices. Loading styles from another file can help maintain a consistent look, but it is sensible to test the result on a copy first. Page styles control margins, paper orientation, header/footer placement, background and borders. Use portrait pages for ordinary reports; landscape may be useful for a wide table.

1.6 Common errors

A heading that only looks bold is not necessarily a real heading style. This can prevent it appearing in a table of contents. Repeated spaces used for alignment may shift when fonts or page sizes change. Inconsistent page styles can create unexpected blank pages or different margins. Correct the underlying style rather than patching each occurrence individually.

1.7 Chapter questions

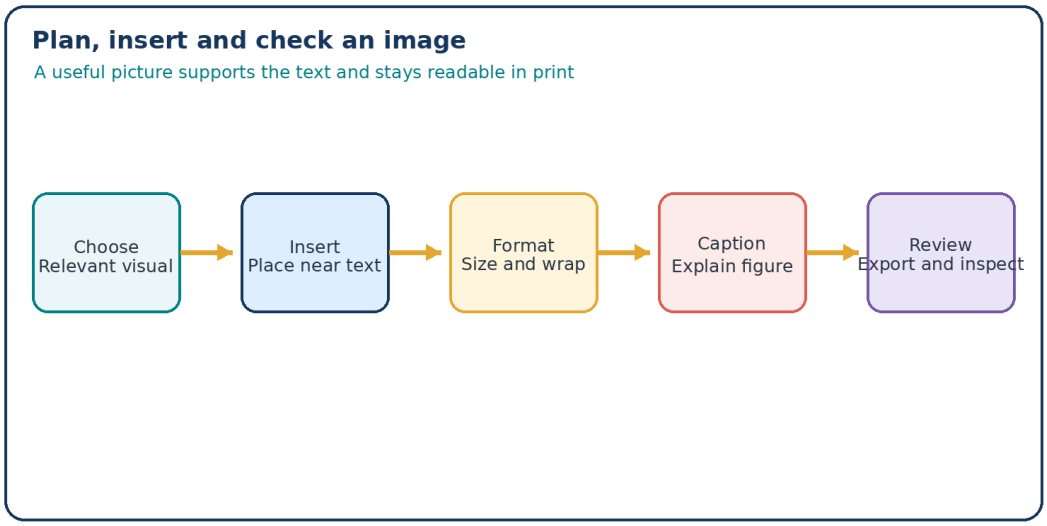
1. What is a style, and why is it useful in a long document?
2. Distinguish paragraph, character and page styles.
3. How can a custom style be created from a formatted selection?
4. Why should headings use heading styles when a table of contents is required?
5. Give two examples of formatting problems caused by manual formatting.

Chapter 2

Working with Images

2.1 Choosing the right image

Images can explain a process, identify an object or make a document easier to scan. A picture should support the nearby text rather than simply fill space. For an office report, a small labelled diagram may be clearer than a decorative photograph. Confirm that an image is appropriate to the subject and that its licence or permission allows the intended use.



2.2 Insert, size and position

Place the cursor near the paragraph that introduces the image, then use the insert-image option. Resize from a corner handle where available so the original proportions are preserved. Use text wrapping and anchoring deliberately: an image anchored as a character behaves more like a large text character, while other anchor modes allow it to float beside text. After moving a picture, review the full page to catch overlapping text or unwanted gaps.

2.3 Cropping and visual correction

Cropping removes unwanted outer portions without changing the message of the remaining image. Brightness, contrast, transparency and colour adjustments can improve legibility, but excessive correction can hide details or misrepresent a picture. Keep the source image so that edits can be reversed. For diagrams, check that labels remain readable after the image is resized.

2.4 Captions and accessibility

A caption states what a figure shows, for example “Figure 2.1: Sequence for checking a document before submission.” Refer to the figure in the surrounding paragraph. Where the software supports alternative text, provide a concise description for readers who cannot see the image. Do not rely on colour alone to communicate a distinction.

2.5 Image layout checklist

Before saving, check that the picture has a clear purpose, is not distorted, does not cover text, uses a suitable wrap and anchor setting, has a useful caption, and remains readable at print size. Export a PDF and inspect the result because on-screen layout can differ from the final page.

2.6 Chapter questions

1. Describe two reasons to include an image in a document.
2. Why should an image normally be resized proportionally?
3. Explain anchoring and text wrapping in your own words.
4. What is cropping, and how is it different from resizing?
5. Why are captions and alternative text helpful?

Chapter 3

Advanced Features of Writer

3.1 Table of contents

An automatically generated table of contents uses heading styles to identify chapter and section names. Apply consistent heading levels first, then insert the table of contents from the appropriate menu. When headings change, update the table rather than editing page numbers manually. A manual contents list may look correct at first, but becomes unreliable after pages are added or removed.

3.2 Templates and document properties

Templates support a uniform appearance across letters, notices and reports. Before using a template, check its paper size, margins, fonts and headings. Document properties can include title and author metadata; enter appropriate information and avoid leaving misleading details from a reused file. Save a new copy rather than overwriting an original template.

3.3 Track changes and review

Track Changes records additions and deletions so reviewers can inspect edits. Use it when a document is being reviewed by more than one person. Accept or reject changes deliberately and add comments that explain the reason for a proposed edit. Before sharing the final version, check whether comments or revision marks should remain visible and follow the organisation’s document-handling rules.

3.4 Compare, protect and export

When a report is revised, check both the content and formatting. Use available comparison tools if the software version supports them. Password protection and editing restrictions should be applied only when they suit the file’s purpose and local policy. Export to PDF for distribution when a stable layout is needed, then open the PDF and inspect headings, images, page numbers and page breaks.

3.5 Practice scenario

A team prepares a four-page meeting report. One colleague changes the headings, another corrects the wording, and a third adds two images. Use heading styles, a generated contents page and tracked changes to combine the work. Update the contents page at the end and export a review PDF. Identify which edits need acceptance before the report is final.

3.6 Chapter questions

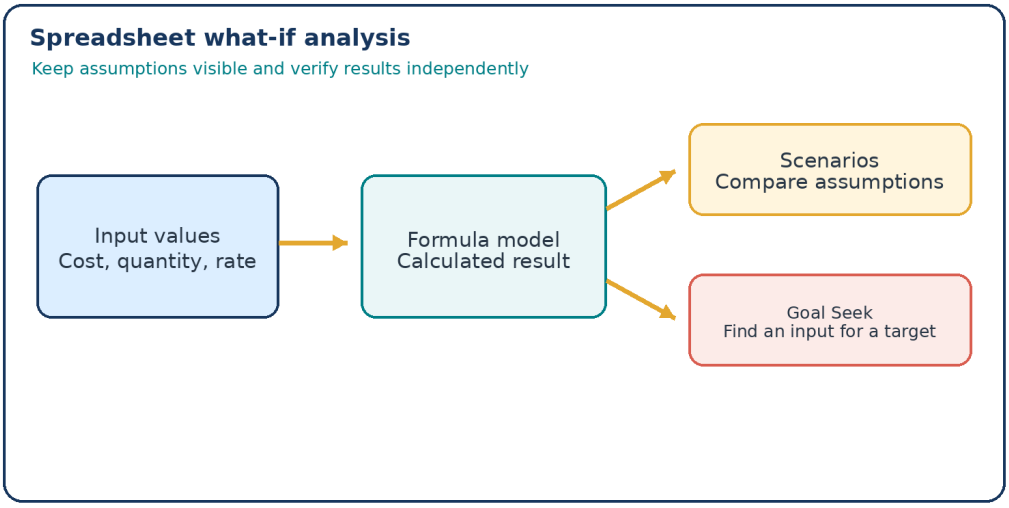
1. How does Writer build an automatic table of contents?
2. What is the purpose of Track Changes?
3. Why should comments and revisions be reviewed before distribution?
4. Name three items to verify after exporting a PDF.
5. How can templates save time while maintaining consistency?

Chapter 4

Analyse Data Using Scenarios and Goal Seek

4.1 What-if analysis

A spreadsheet model contains input values and formulas that calculate results. What-if analysis explores how the result changes when one or more inputs change. A scenario stores a group of assumptions, such as a low, expected or high estimate. Scenarios are useful for comparing possibilities; they do not predict which possibility will actually occur.



4.2 Build a small model

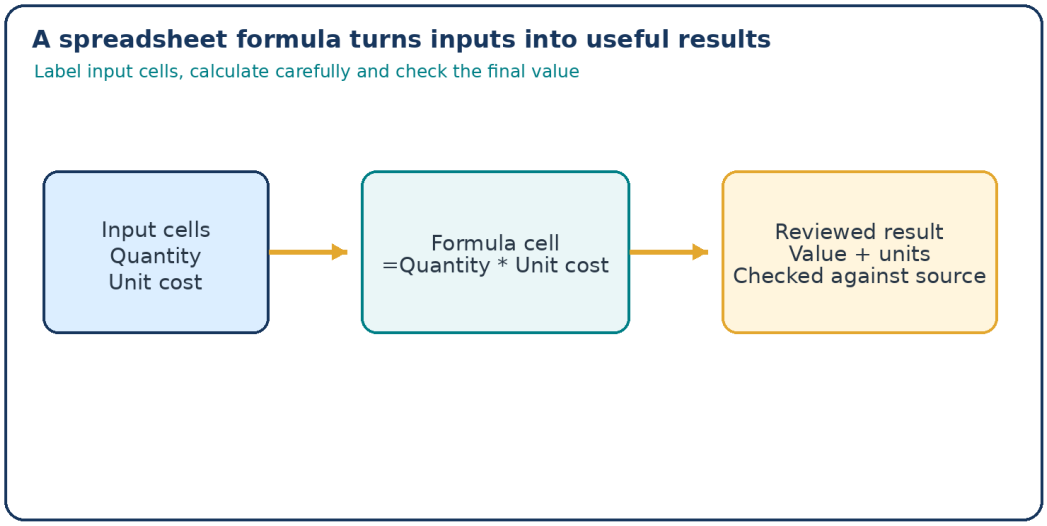
Imagine a school event budget with planned attendance, cost per participant and a fixed venue charge. Total cost can be calculated from the assumptions. Keep input cells clearly separated from formula cells and label units. Before comparing scenarios, confirm that each one changes the intended cells and that the formula uses the correct references.

4.3 Using Goal Seek

Goal Seek works backwards from a desired formula result to estimate the input value that would achieve it. Select the formula cell as the target, specify the desired value and identify the input cell the application may change. Review the returned value and confirm that the model is logical. It solves the selected relationship; it does not prove the assumptions are realistic.

4.4 Check the model

Test a simple case manually. Confirm that changing one input affects the expected outputs and that the target cell contains a formula. Save a separate copy before experimenting with a complex model. Write down the assumptions that make the result meaningful, especially when sharing the spreadsheet with someone who did not build it.



4.5 Chapter questions

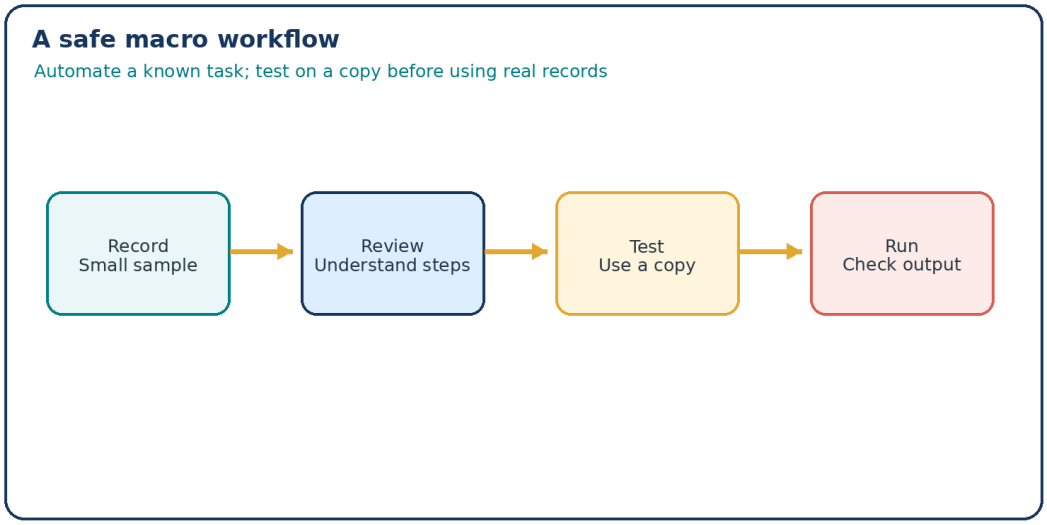
1. What is a scenario in a spreadsheet?
2. Distinguish scenario analysis from Goal Seek.
3. Why should assumptions and units be labelled?
4. What checks should be made after Goal Seek returns a value?
5. Give a practical example where comparing low, expected and high cases is useful.

Chapter 5

Using Macros in a Spreadsheet

5.1 Purpose of a macro

A macro automates a repeated sequence of operations. For example, a macro might apply the same number format to a report or prepare a standard print layout. Automation saves repetitive effort, but an incorrect macro can repeat a mistake across many cells. Understand what a macro does before using it on important files.



5.2 Recording and reviewing

Where macro recording is enabled, record a short harmless task on a sample sheet. Stop recording, give the macro a meaningful name and test it on a copy. Review whether it refers to fixed cells or to the intended active selection. Small changes in the workbook structure can affect recorded steps, so avoid assuming that a macro will work unchanged in every file.

5.3 Security and trust

Macros can carry out actions with the user’s permissions. Do not enable a macro from an unexpected file simply because a message says it is required. Follow the software’s security settings and the organisation’s policy. Use trusted sources and keep a backup before running a macro that changes many cells.

5.4 Maintainable automation

Name macros clearly, document their purpose and test them on representative data. If a workbook changes, run the macro on a test copy before using it in production. Keep the manual procedure documented as a fallback. If an automated result looks wrong, stop and investigate instead of repeatedly rerunning the macro.

5.5 Chapter questions

1. What kinds of repeated work can a macro automate?
2. Why should a macro be tested on a copy of the workbook?

3. Explain one security risk related to macros.
4. Why is a clear macro name useful?
5. What should you do if the output appears incorrect?

Chapter 6

Linking Spreadsheet Data

6.1 Why link data?

A workbook may contain related information on different sheets, or a report may need data from another file. Linking avoids typing the same values repeatedly, but it introduces a dependency: if the source moves or is unavailable, the link may not update as expected. Decide which file is authoritative and keep an understandable folder structure.

6.2 References between sheets

A cell reference points to a location in a worksheet. A formula can refer to another sheet so a summary uses the original record. Use descriptive sheet names and clear headings. When copying formulas, understand whether the references should change relative to the new position or remain fixed; test a copied formula with a small example.

6.3 External links and refreshing

External links may ask whether to update values when a workbook opens. Update only when the source is trusted and expected. If an imported result is unexpected, inspect the source path and the range being referenced. A displayed number may be old, linked to the wrong file or calculated from a range that no longer matches the data.

6.4 Practical case

An office keeps monthly stock records in separate sheets and a summary sheet with totals. Link the summary to the monthly sheets, change one practice quantity and check whether the summary changes. Then rename a copied practice file and observe how the external link behaves. Record the correct way to repair or re-establish the link in your software version.

6.5 Chapter questions

1. Give two advantages and one risk of linking data.
2. What is the difference between a reference within a workbook and an external link?
3. Why should the source workbook be clearly identified?
4. What might cause a linked value not to update?
5. How can you test whether a link points to the correct cell?

Chapter 7

Share and Review a Spreadsheet

7.1 Collaboration and review

Spreadsheet review can involve checking formulas, confirming data sources and making sure that changes are understandable. In shared work, agree who is responsible for the master copy and how edits will be combined. The exact sharing and review tools depend on software version and storage platform; use only the features approved for the workplace.

7.2 Prepare a workbook for another user

Give each sheet a meaningful name, label columns and include units. Remove temporary test data from the copy intended for distribution. If some cells contain formulas, use cell protection where suitable and explain which cells may be edited. Protection can prevent accidental changes, but it is not a substitute for secure storage or access control.

7.3 Comments, changes and versioning

Use comments or notes to explain unusual values and assumptions. When several versions exist, use filenames or a document-management system that make the current version clear. Before merging edits, compare key totals and formulas. Do not assume two workbooks are equivalent simply because their first page looks the same.

7.4 Final review checklist

- Check titles, date ranges, units and source references.
- Recalculate or refresh formulas where required.
- Look for blanks, duplicates and out-of-range values.
- Verify that sorting has not separated related columns.
- Check print area, page breaks and displayed sheet names.
- Confirm that recipients are authorised to view the workbook.

7.5 Chapter questions

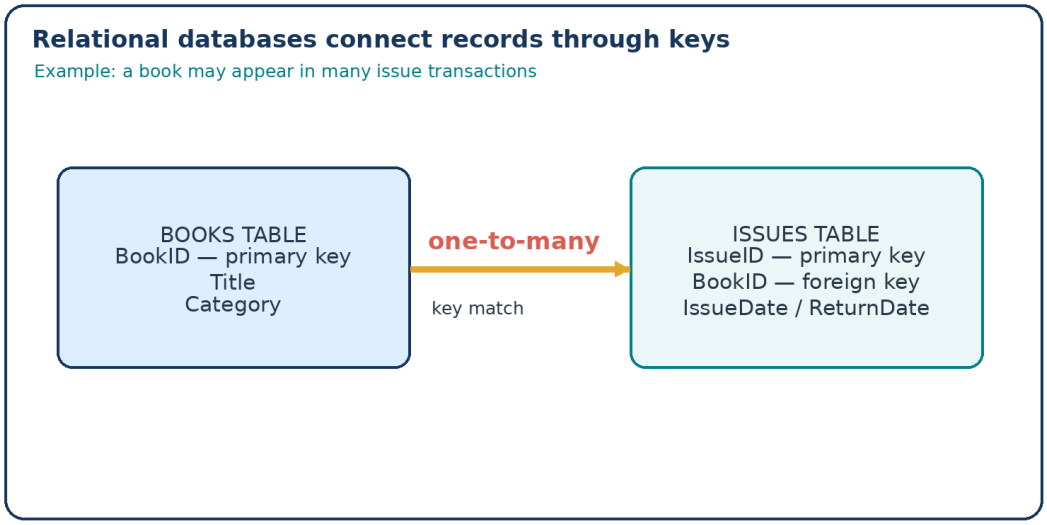
1. What should be agreed before a spreadsheet is edited by several people?
2. Why are clear sheet names and units important?
3. Explain the difference between protecting cells and securing a file.
4. Name four checks to perform before sharing a workbook.
5. Why is version control important?

Chapter 8

Introduction to Database Management System

8.1 From files to a database

A database is an organised collection of related data. A Database Management System (DBMS) helps create, store, retrieve and maintain that data. A paper register or separate files can work for a small task, but duplicate records and inconsistent updates become harder to manage as the information grows. A database can enforce rules and connect related information in a structured way.



8.2 Tables, fields and records

A table stores one kind of entity, such as students, books or issued items. A field is a column describing an attribute; a record is one row describing an instance. For a library table, fields might include BookID, Title and Category. Choose data types that match the values, and avoid putting unrelated details into one field if they need to be searched or validated separately.

8.3 Keys and relationships

A primary key uniquely identifies each record. A foreign key stores a value that refers to a record in another table. Relationships reduce the need to repeat the same facts. In a library example, a Books table can be related to an Issues table through BookID. Referential integrity helps prevent a related record from pointing to a key that does not exist, subject to the design and rules configured in the database.

8.4 Database models

A relational database represents data in related tables. Other models organise relationships differently. For introductory office work, the relational model is particularly useful because tables, keys and queries can be explained with familiar records. Good design begins with understanding what each record represents and which facts belong together.

8.5 Chapter questions

1. Define a database and a DBMS.
2. Distinguish a field from a record.
3. What is the role of a primary key?
4. Explain a foreign key using a library example.
5. Why can a relational database reduce unnecessary duplication?

Chapter 9

Starting with LibreOffice Base

9.1 Database objects

LibreOffice Base is a database front end that can work with different database arrangements. Depending on the chosen database type, a file may store data directly or connect to an external database engine. The main objects commonly introduced are tables, queries, forms and reports. Each serves a different purpose: tables store records, queries select or calculate information, forms support data entry, and reports present organised output.

9.2 Creating a new database

Start a new database from the application and follow the wizard for the required database type. Save it in the intended practice folder with a meaningful filename. Create a small table with fields such as AssetID, ItemName, Quantity and Condition. Select suitable data types and designate a unique field as the primary key where appropriate. Reopen the file to ensure the saved database is accessible.

9.3 Data types and validation

Text fields store names and codes; numeric fields store quantities or measurements; date fields store dates; and Boolean fields represent a true/false condition. A code that contains leading zeros may need to be stored as text so the zeros are preserved. Decide which fields may be blank and which should be required. Validation rules help reduce errors at the point of entry.

9.4 Enter and inspect records

Add a few fictional asset records, then navigate between rows and edit one value. Check that the changed value appears in the intended field. Try leaving a required field blank if the rule permits testing, then observe the validation response. Keep a clean copy before performing experiments that may affect the table structure.

9.5 Chapter questions

1. Name four common database objects in Base.
2. What is a data type? Give three examples.
3. Why might an identification code be stored as text?
4. What is the purpose of a primary key?
5. Why should a new database be reopened after saving?

Chapter 10

Working with Multiple Tables

10.1 Avoiding repeated facts

If an office stores a person's details in every transaction row, the same name or address may be repeated many times. Updating one row but missing another creates inconsistency. Separate tables can store the entity once and link other records to it by a key. The right structure depends on the information and the questions users need to answer.

10.2 Relationships and integrity

A one-to-many relationship occurs when one record in a table relates to several records in another. For example, one customer may have many orders. The foreign key in the order table points to the customer key. Relationship rules should prevent invalid links and should consider what happens if a referenced record is updated or deleted. Do not delete a record without understanding its linked records and the organisation's retention needs.

10.3 Plan tables before creating them

List the entities, their attributes and the unique identifier for each. Draw the relationships and identify which table contains the foreign key. Then create the tables and test with a small set of records. If a value appears repeatedly in several records, ask whether it belongs in a separate related table.

Table	Example fields
Books	BookID (key), Title, Category
Issues	IssueID (key), BookID (reference), IssueDate, ReturnDate

10.4 Chapter questions

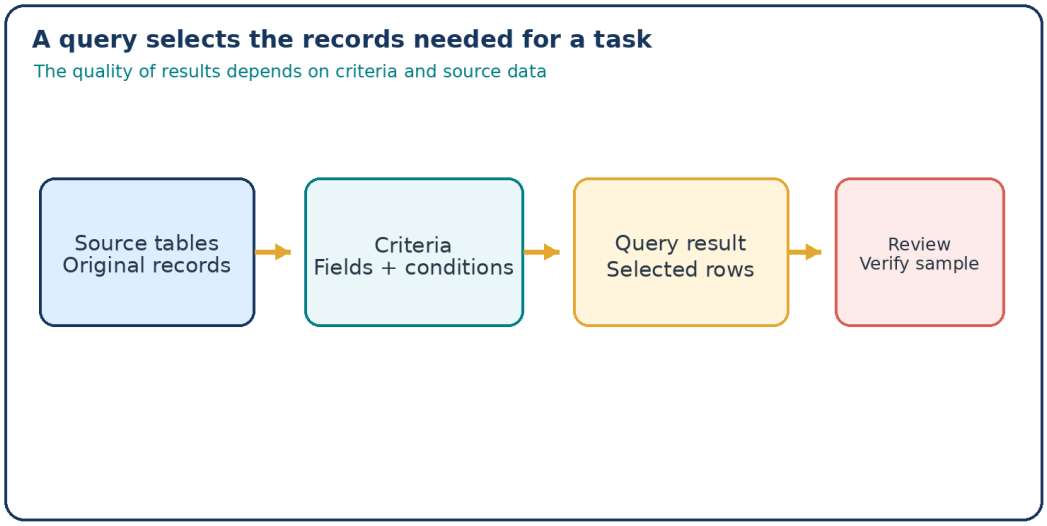
1. Why are related tables often preferable to one very large table?
2. Explain a one-to-many relationship.
3. What is referential integrity?
4. In the example, which table contains the foreign key?
5. What should be checked before deleting a record that other records reference?

Chapter 11

Queries in Base

11.1 What a query does

A query asks the database for a subset or calculated view of its data. It can select records matching conditions, choose particular fields, sort results and combine related tables. A query normally does not replace the underlying source table; it describes which information to retrieve. The exact design interface depends on the selected database and version.



11.2 Criteria and sorting

Suppose the asset register contains equipment in several conditions. A query can show only records whose Condition is “Needs Repair,” or sort all assets by ItemName. Criteria should match the field’s data type. A date condition should use a valid date value, while text criteria need the correct spelling and should account for how the database handles case and spaces.

11.3 Queries involving several tables

A join connects related records through matching keys. When joins are incorrect, the output may omit records or multiply rows unexpectedly. Check that the joined fields represent the same type of identifier and that the relationship is correct. Use a small test with known records to verify the results.

11.4 Queries and privacy

A query can expose information even if the user does not open the original table directly. Return only fields needed for the task and share results with authorised recipients. Do not create broad exports containing personal information when a smaller summary would meet the requirement.

11.5 Chapter questions

1. What is a query?

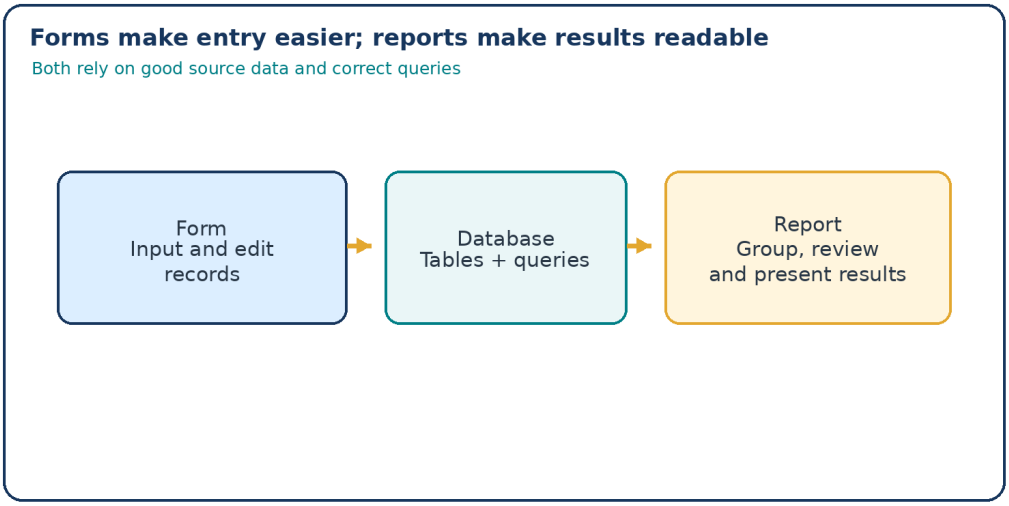
2. How can criteria be used to select records?
3. Why can an incorrect join produce misleading results?
4. What should be checked when filtering date fields?
5. How can a query support data privacy?

Chapter 12

Forms and Reports

12.1 Forms for data entry

Forms provide a user-friendly way to view or enter records without working directly in a table grid. Labels make fields understandable, and suitable controls can limit input errors. A form should follow the table design and make required fields clear. Test both adding and editing records so you know where the data is stored.



12.2 Reports for presentation

Reports organise selected data for reading, printing or review. A report may group records by category, include totals or show a date range. Before producing it, decide who will read it and which fields are necessary. Check that headings are meaningful and long values are not cut off. A neat report can still be wrong if the query or source data is wrong.

12.3 Designing and checking

Give forms clear labels and logical tab order. Use validation rules where supported and explain any required fields. For reports, include a title, date or reporting period where relevant, and suitable page numbering. Preview the report before printing or exporting. Compare a sample of displayed records with the source tables to confirm correctness.

12.4 Chapter questions

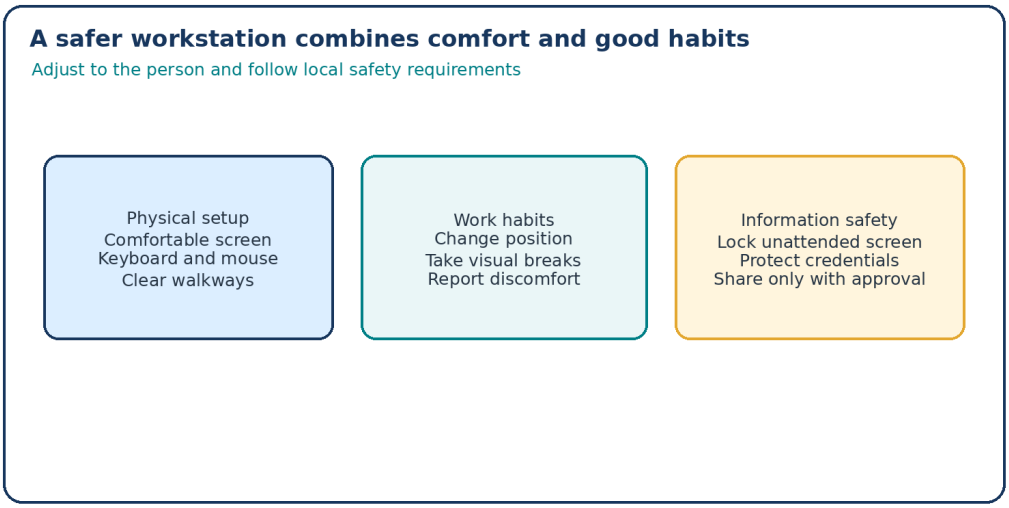
1. What is the purpose of a form?
2. How does a report differ from a table?
3. Why should forms make required fields clear?
4. Name four checks to make before distributing a report.
5. Why must a report be checked against the query and source data?

Chapter 13

Health, Safety and Security at Workplace

13.1 Recognising workplace risks

Computer-based work can involve physical, environmental and information-security risks. Poor workstation setup may contribute to discomfort; clutter can create trip hazards; exposed electrical equipment can cause serious injury; and weak information handling can expose private records. Risk management starts by recognising hazards, considering who may be harmed and following the organisation's procedures.



13.2 Ergonomics and eye comfort

Arrange the chair, monitor, keyboard and mouse to suit the user and the task. Keep the screen readable and reduce distracting reflections. Change position periodically and take short visual breaks during extended work. If a person experiences persistent pain or eye discomfort, they should report it and seek appropriate professional advice rather than treating a textbook as medical guidance.

13.3 Stress and work organisation

Long periods of intense work, unclear instructions and constant interruptions can increase strain. Divide work into manageable tasks, clarify priorities and ask for assistance when a deadline conflicts with the expected quality. A realistic pace supports both wellbeing and accurate data handling. Supervisors should encourage reporting concerns early rather than rewarding unsafe shortcuts.

13.4 Electrical and physical safety

Keep liquids away from electrical equipment and do not handle plugs or devices with wet hands. Do not attempt repairs unless authorised and trained to do so. Keep walkways clear, route cables safely and report damaged equipment. Follow local procedures for ventilation, lighting, noise and equipment placement.

13.5 Information security at a workstation

Lock the screen when leaving a shared computer, use approved accounts and keep passwords confidential. Do not install unauthorised software or connect unknown devices. Confirm the recipient before sending a file and follow access rules for personal information. Physical safety and information security are both part of a responsible workplace.

13.6 Chapter questions

1. What is a workplace hazard? Give three examples.
2. Describe two adjustments that can make computer work more comfortable.
3. Why should damaged electrical equipment be reported rather than repaired without authorisation?
4. Name two actions that protect information on a shared workstation.
5. How can work organisation help reduce avoidable stress?

Chapter 14

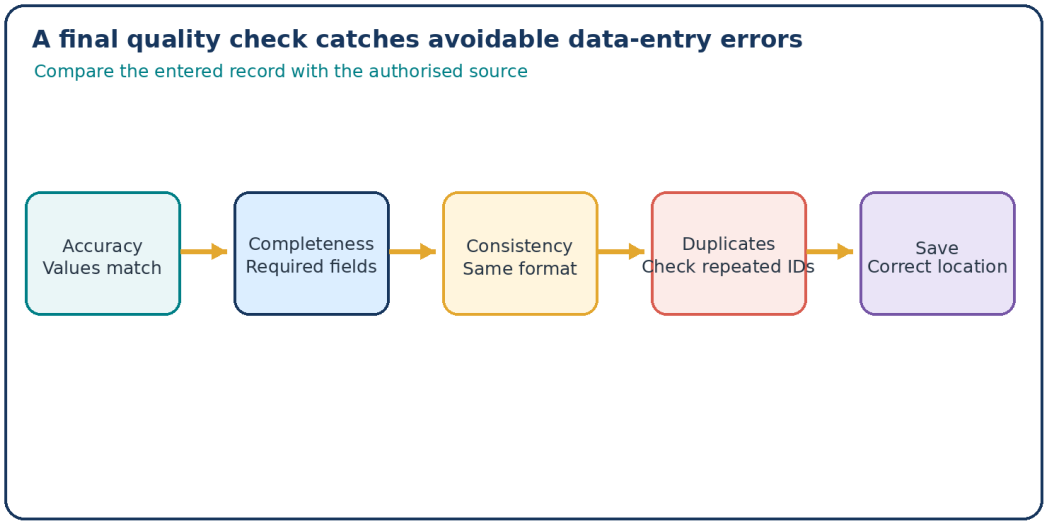
Workplace Quality Measures

14.1 Quality is built into the process

Quality work meets the stated requirement reliably, not just quickly. For data entry, this means using the correct source, entering values in the intended fields, checking completeness and protecting records. A quality process makes the expected standard visible and gives people a way to identify and correct errors.

14.2 Checks before submission

A final review can include a field-by-field comparison with the source, checking required fields, identifying duplicates, confirming date and number formats, and verifying that the file is saved in the correct place. The amount of checking should reflect the task's risk. High-impact records deserve careful review and the required approval.



14.3 Documenting a problem

When an error cannot be resolved from the available source, mark it for follow-up and identify the appropriate person or process. Do not silently substitute a plausible value. A correction should follow the organisation's rules so the record remains understandable and, where required, traceable. Keep the note concise and avoid including unnecessary personal information.

14.4 Continuous improvement

If the same error occurs repeatedly, look for a process cause. The instruction may be unclear, a form may invite mistakes, or a field may lack a useful validation rule. A small change to the process can prevent many corrections later. Test the change and confirm that it improves quality without creating a new problem.

14.5 Chapter questions

1. What is meant by quality in data-entry work?

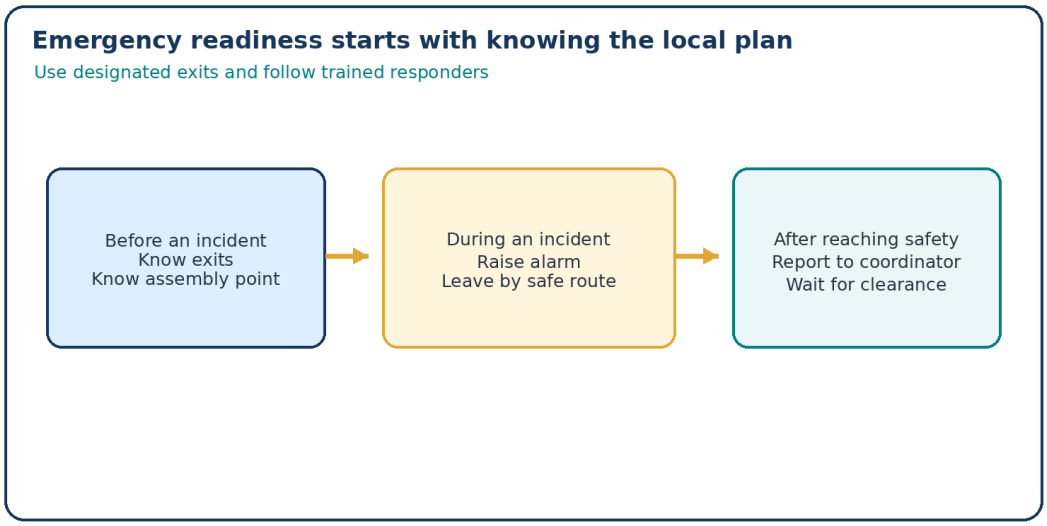
2. Name five checks that can be made before submitting a record set.
3. Why is guessing a missing value unacceptable?
4. How can repeated errors reveal a process problem?
5. What does traceability mean in a correction process?

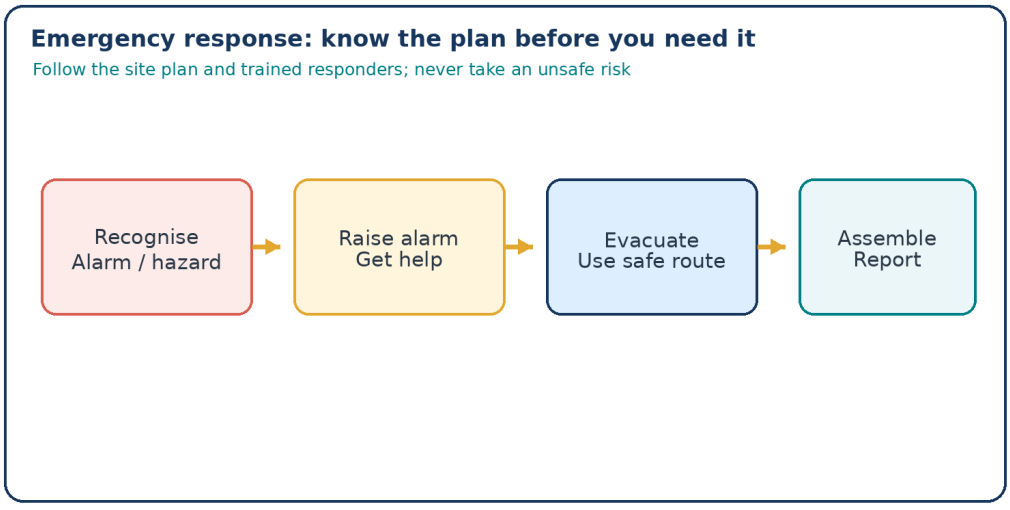
Chapter 15

Prevent Accidents and Emergencies

15.1 Preparedness before an incident

An emergency may arise from fire, electrical failure, injury, smoke, severe weather or another unexpected event. Each organisation should have an emergency action plan that identifies alarms, exit routes, assembly points, communication roles and procedures for obtaining assistance. Learn the local plan before an incident occurs; do not wait until the alarm sounds to look for an exit.





15.2 During evacuation

When an alarm or authorised evacuation instruction is given, follow the site's procedure, use the designated safe route and move calmly. Do not return to collect personal belongings. Do not use lifts where the plan says not to. Go to the assigned assembly point and follow instructions from the responsible coordinator. Never re-enter until the responsible authority says it is safe.

15.3 Fire and electrical incidents

Raise the alarm and contact the designated emergency service or internal response team according to local procedure. Only people with the proper training and a safe escape route should attempt to use firefighting equipment. Do not touch a person who may still be in contact with live electricity; keep clear and ensure power is isolated by a competent person when it is safe to do so. Follow trained responders' directions.

15.4 Injuries and first aid

Get help promptly. Provide first aid only within the limits of your training and the organisation's procedure. Do not move a seriously injured person unless there is an immediate danger that makes moving necessary. Give responders clear information about the location and known circumstances. Do not attempt a rescue that places you or others at further risk.

15.5 After the event

Once safe, report what happened through the appropriate channel. Record objective facts rather than guesses about the cause. Participate in the review process if asked and help identify practical steps that may reduce the chance of recurrence. Incident reports should be handled carefully and shared only with authorised people.

15.6 Chapter questions

1. What information should an emergency action plan contain?
2. What actions should be taken when an evacuation alarm is raised?
3. Why should only trained people use firefighting equipment?
4. What should you avoid doing during a suspected electrical accident?
5. Why is objective incident reporting important?

Chapter A

Revision Questions

A.1 Multiple-choice practice

Choose the best answer.

1. Which feature helps keep headings consistently formatted? (a) Manual spaces (b) Styles (c) Repeated blank lines (d) Random font changes
2. Which operation removes unwanted outer parts of a picture? (a) Cropping (b) Sorting (c) Filtering (d) Joining
3. Which Writer feature can generate a contents list from headings? (a) Word count (b) Track Changes (c) Table of Contents (d) AutoCorrect
4. What-if scenarios are mainly used to: (a) compare sets of assumptions (b) repair a keyboard (c) lock a screen (d) create a database key
5. Goal Seek works from a desired result to find: (a) an input value (b) a picture size (c) a page margin (d) a file name
6. A macro is most useful for: (a) repeating a known sequence of operations (b) replacing backups (c) proving data is correct (d) granting access to everyone
7. A primary key should: (a) identify a record uniquely (b) contain a paragraph (c) duplicate every other row (d) be optional for every design
8. A query is used to: (a) retrieve selected data (b) replace all training (c) switch off the computer (d) crop an image
9. A form mainly supports: (a) data entry and viewing (b) emergency evacuation (c) file compression only (d) keyboard repair
10. When a building evacuation is ordered, you should: (a) collect all belongings first (b) follow the designated safe route (c) use any lift (d) return immediately

A.2 Short-answer practice

1. Explain how paragraph styles help a long report remain consistent.
2. Describe a safe sequence for inserting and reviewing an image.
3. Give an example of a spreadsheet model suitable for Scenario Manager and Goal Seek.
4. State two precautions to take before running a macro.
5. Explain primary key, foreign key and referential integrity.
6. Describe how a query can return only the fields needed for a report.
7. List five quality checks for a spreadsheet before it is shared.
8. Give four actions that contribute to a safe computer workstation.
9. What should an employee do when a record contains missing information that cannot be confirmed?
10. Why are practice records preferably fictional?

Chapter B

Answer Key for Revision Questions

B.1 Multiple-choice answers

1. (b) Styles; 2. (a) Cropping; 3. (c) Table of Contents; 4. (a) compare sets of assumptions; 5. (a) an input value; 6. (a) repeating a known sequence of operations; 7. (a) identify a record uniquely; 8. (a) retrieve selected data; 9. (a) data entry and viewing; 10. (b) follow the designated safe route.

B.2 Short-answer marking guidance

Answers should use the concepts explained in the relevant chapters. Award credit for accurate definitions, suitable examples and a clear explanation of why the process matters. For practical questions, a strong answer describes a safe sequence, identifies the source of truth and includes a verification step. Accept other correct examples that meet the same principle.

HANDBOOK

Web Developer

Grade XI – IT-ITeS Vocational Education

Web Essentials • HTML • CSS • JavaScript • SDLC

A structured textbook guide designed for practical web development mastery.

Based on Grade XI Web Developer study material topics

Contents

1 Web Design and Development **1**
1.1 Understanding Web Design . . . 1
1.1.1 User Interface and User Experience . . . 1
1.2 Understanding Web Development . . . 2
1.2.1 Static and Dynamic Websites . . . 2
1.3 The Web Design and Development Life Cycle . . . 3
1.3.1 Stage 1: Discover the Need . . . 3
1.3.2 Stage 2: Plan the Work . . . 3
1.3.3 Stage 3: Design the Structure and Appearance . . . 3
1.3.4 Stage 4: Develop the Website . . . 3
1.3.5 Stage 5: Test and Review . . . 4
1.3.6 Stage 6: Publish and Deploy . . . 4
1.3.7 Stage 7: Maintain and Improve . . . 4
1.4 Core Principles of Effective Web Design . . . 4
1.4.1 Visual Hierarchy . . . 4
1.4.2 Consistency and Simplicity . . . 4
1.4.3 Colour and Typography . . . 4
1.4.4 Navigation and Content . . . 4
1.5 Responsive Design and Device Support . . . 4
1.6 Accessibility and Inclusive Design . . . 5
1.7 Designing for Quality, Security and Performance . . . 5
1.8 Example: Planning a Learning Centre Website . . . 5
1.9 Chapter Review . . . 6

2 HTML: Document Architecture **7**

3 CSS Layouts and Box Model **8**

4 JavaScript Fundamentals **9**

5 Requirements, Planning and Project Documents **10**
5.1 Understanding the Need . . . 10
5.1.1 Stakeholders and Users . . . 10
5.1.2 Functional and Non-functional Requirements . . . 10
5.2 Brief Requirements Document . . . 10
5.3 Planning and Scope . . . 10
5.4 Business and Software Requirements . . . 11
5.5 High-Level and Low-Level Design . . . 11
5.6 Design Models and Interfaces . . . 11

6 Web Technologies, Browsers and HTTP **12**
6.1 How a Web Page Reaches a Browser . . . 12
6.2 HTTP and HTTPS . . . 12

6.2.1 Common Status Codes 12
6.3 Browser Developer Tools 12
6.4 Development Tools and Standards 12
6.5 Common Application Development Models 13

7 Semantic HTML and Accessible Structure 14
7.1 Meaningful HTML Elements 14
7.2 Headings and Content Hierarchy 14
7.3 Lists and Grouped Information 14
7.4 Links, Images and Alternative Text 15
7.5 Forms and Labels 15

8 CSS Selectors, Cascade and Responsive Layout 16
8.1 Selectors 16
8.2 Cascade, Specificity and Inheritance 16
8.3 Responsive Design 16
8.4 Flexbox and Grid 17

9 JavaScript Variables, Types and Operators 18
9.1 Storing Values 18
9.2 Common Data Types 18
9.3 Operators and Comparisons 18
9.4 Readable and Safe Code 18

10 Conditions, Loops, Functions and Arrays 19
10.1 Conditional Decisions 19
10.2 Loops 19
10.3 Functions 19
10.4 Arrays and Objects 19
10.5 Working with Strings 19

11 DOM, Events and Form Interaction 21
11.1 The Document Object Model 21
11.2 Events 21
11.3 Validation and Feedback 21
11.4 Safe DOM Updates 21
11.5 Drawing with the HTML Canvas 21
11.6 Using JavaScript with Images 22

12 Testing, Debugging and Quality Assurance 23
12.1 Why Testing Matters 23
12.1.1 Useful Test Categories 23
12.2 A Practical Debugging Method 23
12.3 Test Case Example 24

13 Accessibility, Performance and Secure Publishing 25
13.1 Accessibility Essentials 25
13.2 Performance Basics 25
13.3 Security Awareness 25
13.4 Publishing Checklist 25

14 Integrated Mini Project: Learning Centre Website 26
14.1 Project Brief 26

14.2 Suggested Page Structure 26
14.3 Implementation Plan 26
14.4 Evaluation Rubric 26

Chapter 1

Web Design and Development

A website often begins with a simple question: what should people be able to find or do here? The answer shapes every decision that follows, from the first sketch of a page to the code that runs in a browser. Good web design helps visitors understand where they are and what they can do next. Good development turns that plan into a working site, checks that it behaves properly, and makes it dependable on different devices.

The two activities work best together. A page may look attractive but still frustrate people if its navigation is confusing or its text is difficult to read. Likewise, a technically correct page is not very useful if visitors cannot find the information they came for. A thoughtful developer keeps the people using the website in mind while planning, designing, coding, testing, and improving it.

Figure 1.1. A web developer's workspace. Photograph: Shixart1985, Wikimedia Commons, CC BY 2.0.

1.1 Understanding Web Design

Web design concerns the organisation and presentation of information on a website. It includes page layout, colour, typography, images, navigation, spacing, and the way a visitor moves from one task to another. Good design makes the purpose of each page clear and helps users find what they need without unnecessary effort.

A designer begins by asking who will use the website, what those people need to do, and what information they expect to find. A school website, for example, may need clear links to admissions, courses, notices, and contact details. An online shop needs product information, prices, search and filtering, a cart, and a clear checkout path. The design should reflect the real purpose of the site rather than decorative trends alone.

1.1.1 User Interface and User Experience

The user interface (UI) is the part of the system with which a person interacts: menus, buttons, links, forms, headings, and visual controls. User experience (UX) is the broader experience of using the site, including how easy it is to learn, how quickly a task can be completed, and whether the process feels predictable and trustworthy.

A visually attractive interface can still provide a poor experience if its navigation is confusing or its forms give unclear errors. Similarly, a simple interface can provide a strong experience when the content is well organised, controls are labelled clearly, and users receive helpful feedback after an action.

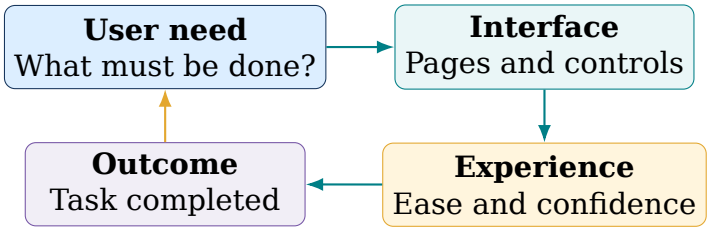


Figure 1.1. UI elements support a user experience that should lead to a clear outcome.

1.2 Understanding Web Development

Web development is the process of building and maintaining the website's structure, appearance, behaviour, and supporting services. Front-end development creates the part that runs in the browser. HTML describes the content and its meaning, CSS controls presentation and layout, and JavaScript adds behaviour such as menu interaction, input validation, and dynamic updates.

Back-end development, when required, handles server-side tasks such as business rules, user authentication, data storage, and communication with other services. A database stores information in an organised form. An application programming interface (API) can provide a defined way for the browser or another application to request data or perform an operation. A small static information site may not need a back end; a learning portal with student accounts and saved results usually does.

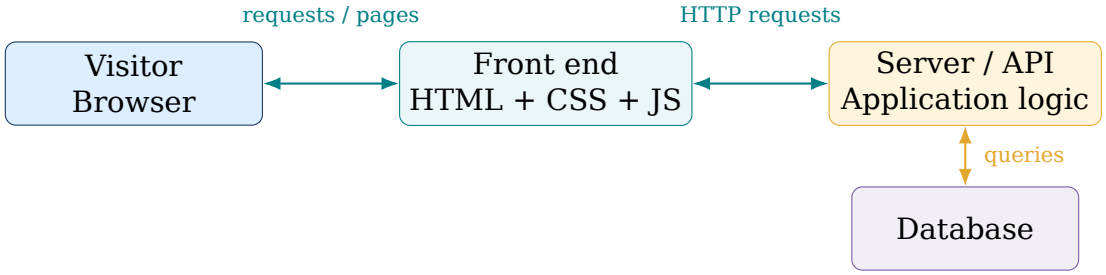


Figure 1.2. A common website architecture. A simple static site may use only the browser and front-end files.

1.2.1 Static and Dynamic Websites

A static website serves pages whose content is mainly prepared in files such as HTML, CSS, JavaScript, and images. It is suitable for portfolios, simple information pages, event announcements, and small documentation sites. A dynamic website can change its output based on user input, stored data, permissions, or other conditions. Examples include online banking, booking systems, shopping sites, and student portals.

The distinction is about how the content is produced, not whether the page has styling or animation. A static page can be visually rich and interactive in the browser. A dynamic service often combines browser code with server-side processing and a database.

1.3 The Web Design and Development Life Cycle

The development life cycle provides a sequence of activities that helps a team move from an idea to a reliable website. The stages may overlap, and teams often return to earlier decisions after testing or feedback. The purpose of a life cycle is not to create paperwork for its own sake; it is to reduce avoidable mistakes, make decisions visible, and ensure that the final site meets its requirements.

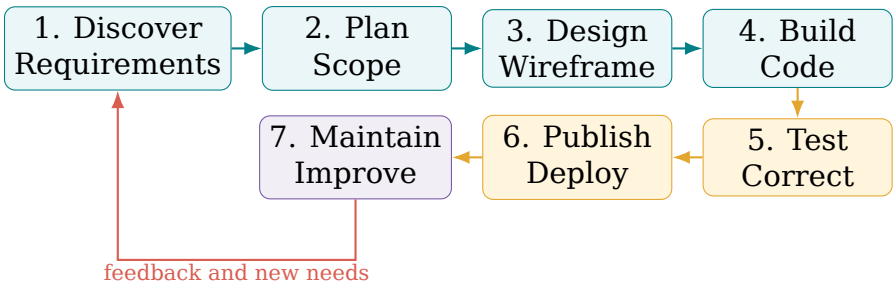


Figure 1.3. A practical website life cycle with feedback after release.

1.3.1 Stage 1: Discover the Need

Identify the problem the website should solve and the people who will use it. Gather information from stakeholders, review existing material, and list the most important user tasks. For a school information site, these tasks may include finding a course, checking an announcement, or contacting the office. Clear goals prevent the project from becoming a collection of unrelated pages.

1.3.2 Stage 2: Plan the Work

Define the scope, deliverables, schedule, resources, and responsibilities. Decide which features are essential for the first version and which can wait. A page list and a simple task board can be sufficient for a small project. Planning should also consider risks such as missing content, unavailable images, device compatibility, privacy, and limited time.

1.3.3 Stage 3: Design the Structure and Appearance

Create a sitemap to show how pages relate to one another. Sketch wireframes to decide where headings, navigation, content, forms, and images should appear. Choose a consistent colour palette, type scale, spacing system, and button style. Review the design against the user tasks before investing time in detailed implementation.

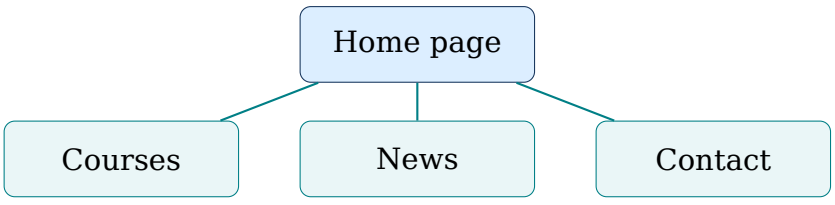


Figure 1.4. Example sitemap for a small educational website.

1.3.4 Stage 4: Develop the Website

Build the page structure with semantic HTML, apply CSS for layout and responsive behaviour, and add JavaScript where interaction is needed. Keep files organised and use meaningful names. Work in small steps: create a page shell, check it in a browser, add one feature, and test again. This makes it easier to identify which change introduced a problem.

1.3.5 Stage 5: Test and Review

Testing checks whether the site behaves as intended. Confirm that links work, forms handle missing or invalid values, images load, text is readable, and the layout works on narrow and wide screens. Use keyboard navigation and inspect error messages in the browser's developer tools. Fix serious problems before publishing, then repeat the relevant tests after changes.

1.3.6 Stage 6: Publish and Deploy

Publishing makes the site available to its intended audience. Deployment may involve uploading static files to a hosting service, connecting a domain name, enabling HTTPS, configuring server settings, and checking that all paths work on the live site. A site that works on a developer's computer can still fail after deployment if file names, capitalisation, paths, permissions, or environment settings differ.

1.3.7 Stage 7: Maintain and Improve

After release, check for broken links, outdated information, security updates, performance problems, and user feedback. Keep a copy of the source files and a record of important changes. Maintenance is part of development, not an optional activity that begins only when something breaks.

1.4 Core Principles of Effective Web Design

1.4.1 Visual Hierarchy

Visual hierarchy helps readers recognise what is most important. Main headings should be more prominent than body text; related items should be grouped; and primary actions should be easy to identify. Use size, weight, spacing, alignment, and colour consistently. Avoid giving every element the same visual emphasis, because this makes it harder to scan a page.

1.4.2 Consistency and Simplicity

Use the same navigation pattern, heading styles, button shapes, and spacing rules across related pages. Consistency reduces the effort required to learn the site. Simplicity does not mean removing necessary information; it means presenting the information in a clear order and avoiding decoration that competes with the task.

1.4.3 Colour and Typography

Choose colours that support the site's purpose and provide sufficient contrast between text and its background. Do not communicate important status using colour alone; pair colour with a label or icon where appropriate. Use a small, consistent set of readable fonts and line lengths. Body text should remain comfortable to read on both desktop and mobile displays.

1.4.4 Navigation and Content

Navigation labels should describe their destination, for example, "Courses" or "Contact", rather than vague labels such as "Click here". Put frequently used destinations where people expect them. Write concise headings, divide long material into meaningful sections, and ensure that images have a clear purpose. Content should answer the questions visitors came to resolve.

1.5 Responsive Design and Device Support

A responsive website adapts its layout to the available screen space. A multi-column desktop layout may become a single column on a phone; navigation may wrap or

collapse; images should scale without distortion; and buttons should remain large enough to tap comfortably. Responsive design uses flexible layouts, relative sizing, and CSS media queries rather than building an entirely separate page for every device.

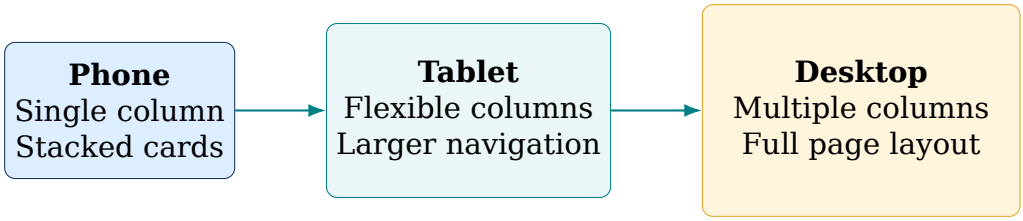


Figure 1.5. The same content can adapt to different viewport sizes.

1.6 Accessibility and Inclusive Design

Accessible design allows people with different abilities and ways of interacting to use a website. Provide meaningful headings, labels for form controls, alternative text for informative images, sufficient colour contrast, and visible keyboard focus. Ensure that interactive elements can be reached and operated without a mouse. Plain, direct language also helps users who are new to the subject or reading in a second language.

Accessibility is best considered during planning and design, then checked throughout development. Retrofitting it at the end may require changing page structure and interaction patterns. A practical review includes keyboard-only navigation, zooming text, checking labels, and verifying that the purpose of each link is understandable.

1.7 Designing for Quality, Security and Performance

Quality includes correctness, usability, reliability, maintainability, accessibility, and performance. Optimise images so that they are not unnecessarily large, avoid loading resources that are not needed, and test pages on a realistic connection. Keep dependencies and source files organised so another developer can understand the project.

Security must also influence design decisions. Collect only the information the site actually needs, explain why it is collected, and avoid placing passwords, private keys, or secret tokens in front-end code. Use HTTPS for deployed sites and apply appropriate validation and access controls when server-side features are involved. A polished interface cannot compensate for unsafe handling of user data.

1.8 Example: Planning a Learning Centre Website

Consider a learning centre that needs to publish courses, class schedules, announcements, and contact information. Before coding, the team identifies the main visitors: learners, parents, and staff. The essential pages are Home, Courses, Notices, and Contact. The home page introduces the centre and links to the most common tasks; the courses page provides names, descriptions, and eligibility information; the notices page lists dated updates; and the contact page gives verified contact details.

The first version can be a static website if the information is updated manually. If staff later need to sign in and manage notices, the project may require authentication, a server, and a database. This illustrates an important planning rule: begin with the simplest solution that satisfies the confirmed need, then add complexity only when a real requirement justifies it.

1.9 Chapter Review

1. Distinguish web design from web development and give two examples of work done in each area.
2. Explain the difference between user interface (UI) and user experience (UX).
3. Describe the role of HTML, CSS, JavaScript, a server, an API, and a database in a website.
4. Compare static and dynamic websites and provide one suitable example of each.
5. List the major stages of the web design and development life cycle.
6. Explain why requirements and scope should be discussed before coding begins.
7. What is a wireframe, and how does it help a development team?
8. Describe four principles of effective web design.
9. Explain responsive design and name two ways a page can adapt to a phone screen.
10. Identify four accessibility checks that should be completed before launch.
11. Why should testing and maintenance continue after a website is published?
12. For a learning centre website, identify the main users, essential pages, and one feature that could be postponed to a later release.

Chapter 2

HTML: Document Architecture

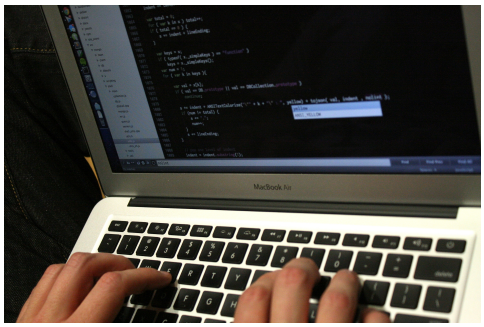


Figure 2.1. A learner working with HTML and CSS code. Photo: hackNY / Matylda, CC BY-SA 2.0. HyperText Markup Language (HTML) provides the fundamental skeletal structure for web content. Elements encapsulate textual data and media objects using semantic tags.

```
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width, initial-scale=1.0">
  <title>Developer Handbook</title>
</head>
<body>
  <header>
    <h1>Web Development Basics</h1>
  </header>
  <main>
    <p>Semantic document structures improve accessibility and indexing.</p>
  </main>
</body>
</html>
```

Review Questions

1. What purpose does the `<!DOCTYPE html>` declaration serve?
2. Write out a basic semantic HTML skeleton including a header, main container, and footer.

Chapter 3

CSS Layouts and Box Model

Figure 3.1. Responsive design adapts a website to desktop, tablet and phone screens. Photo: Muhammad Rafizeldi, CC BY-SA 3.0. Cascading Style Sheets (CSS) control visual presentation, spacing, typography, colors, and layout positioning across device viewports.

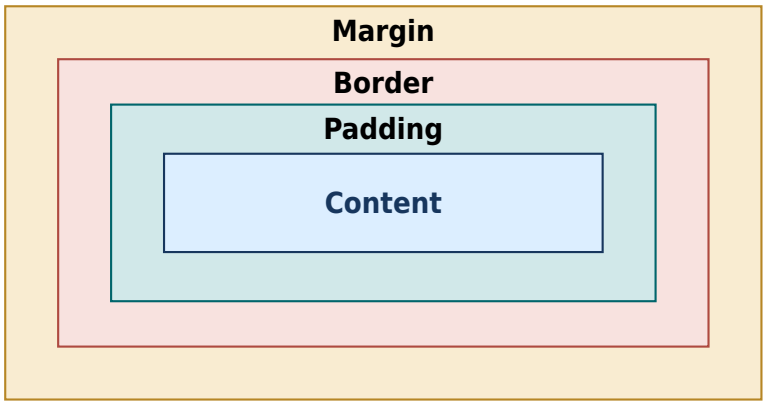


Figure 4.1. Visual layers of the CSS Box Model.

```
* {
  box-sizing: border-box;
  margin: 0;
  padding: 0;
}
.card {
  width: 100%;
  max-width: 400px;
  padding: 1.5rem;
  border: 1px solid #007F86;
  background-color: #EAF6F7;
}
```

Review Questions

1. Describe the four components comprising the CSS Box Model.
2. Explain how `box-sizing: border-box` simplifies spatial calculations.

Chapter 4

JavaScript Fundamentals

Figure 4.1. Source code is written and reviewed in a programming editor. Photo: Martin Vorel, CC BY-SA 4.0. JavaScript enables dynamic interactive behaviors, state manipulation, event processing, and asynchronous network communication directly within client browser environments.

Figure 5.1. The triad of front-end web development technologies.

```
const calculateTotal = (price, taxRate) => {
  if (price <= 0) {
    throw new Error("Invalid price value provided.");
  }
  return price + (price * taxRate);
};

console.log(calculateTotal(100, 0.18));
```

Review Questions

1. Contrast `let` and `const` variable scoping rules.
2. Explain how event listeners bind user actions to script logic.

Chapter 5

Requirements, Planning and Project Documents

5.1 Understanding the Need

A web project begins by understanding the problem it is expected to solve. A website for a school, a small shop, a public service office, and an online learning platform may all use a browser, but each has different users, content, rules, and success measures. Clear requirements prevent avoidable rework and help a team agree on what must be delivered.

5.1.1 Stakeholders and Users

A stakeholder is a person or group affected by the project. Stakeholders may include the customer, website visitors, content editors, administrators, support staff, and the organisation responsible for maintaining the site. Users should be described by their goals rather than by assumptions about their age, device, or technical knowledge.

5.1.2 Functional and Non-functional Requirements

Functional requirements describe what the system must do: display a product catalogue, validate a form, search for an article, or allow an authorised editor to update content. Non-functional requirements describe qualities and constraints, such as response time, accessibility, security, browser support, reliability, and ease of maintenance.

Requirement type	Example for a school website
Functional	A visitor can view announcements and download the academic calendar.
Usability	Navigation labels are understandable to first-time visitors.
Accessibility	Content can be navigated using a keyboard and read by assistive technology.
Performance	Pages avoid unnecessarily large images and scripts.
Security	Administrative editing features require authentication.
Maintainability	Repeated page elements use consistent templates and styles.

5.2 Brief Requirements Document

A simple project brief should record the purpose, target audience, main pages, required functions, content owner, constraints, and acceptance criteria. Acceptance criteria are observable conditions used to decide whether a feature is complete. For example, “the contact form is finished” is vague; “the form identifies missing required fields and displays a clear confirmation after valid submission” is testable.

5.3 Planning and Scope

Scope defines what is included and what is excluded. A small first release may include a home page, about page, services page, and contact form. A payment system or user account feature may be planned for a later release. A scope list helps the team manage time and avoid adding unplanned work without reviewing its impact.

5.4 Business and Software Requirements

A Business Requirements Document (BRD) describes the business need, intended users, goals, scope, constraints, and expected outcomes. A Software Requirements Specification (SRS) translates the agreed need into detailed system behaviour, interfaces, data requirements, and quality expectations. These documents should be clear enough that a developer can implement the work and a tester can verify it.

A practical BRD may contain the problem statement, project objectives, stakeholders, scope, assumptions, constraints, risks, and acceptance criteria. An SRS may describe functional requirements, non-functional requirements, input and output formats, error handling, external interfaces, and security expectations. For a small classroom project, these can be concise documents rather than lengthy reports.

5.5 High-Level and Low-Level Design

High-level design (HLD) describes the major components of a solution and how they communicate. For a website, it may show the browser interface, application server, database, and external services. Low-level design (LLD) describes the details needed to implement a component, such as field names, validation rules, function responsibilities, or the structure of a page.

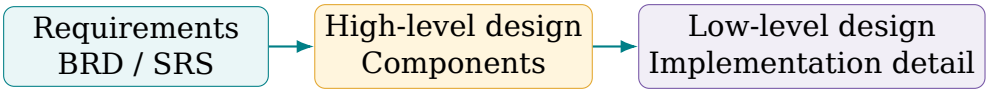


Figure 3.1. Requirements are refined into architecture and implementation detail.

5.6 Design Models and Interfaces

Object-oriented design organises a system around objects that combine data and behaviour. Database design identifies the information that must be stored and the relationships between records. API design defines how separate software components exchange requests and responses. The chosen design should match the project size and should not introduce complexity without a clear need.

Review Questions

1. Distinguish functional requirements from non-functional requirements.
2. Name four stakeholders who may be involved in a website project.
3. Write three acceptance criteria for a school announcements page.
4. Why is it useful to define project scope before coding begins?

Chapter 6

Web Technologies, Browsers and HTTP

6.1 How a Web Page Reaches a Browser

When a visitor enters a website address, the browser uses network services to locate the destination server and request resources. The server returns a response, often containing HTML, CSS, JavaScript, images, fonts, or data. The browser interprets these resources and constructs the visible page. This process may involve many requests rather than a single file.

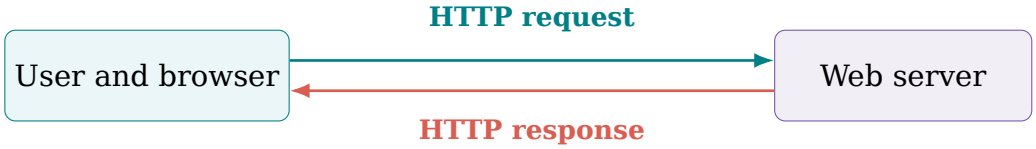


Figure 7.1. A simplified browser-to-server request and response.

6.2 HTTP and HTTPS

HTTP is a protocol used for communication between clients and web servers. A request commonly identifies a method, a resource path, headers, and sometimes a body. A response contains a status code, headers, and usually content. HTTPS protects HTTP communication using TLS, helping protect data from interception or alteration while it travels across the network.

6.2.1 Common Status Codes

- **200 OK:** The request was processed successfully.
- **301 or 302:** The resource has been redirected.
- **400 Bad Request:** The server could not process the request as sent.
- **401 Unauthorized:** Authentication is required or has not succeeded.
- **403 Forbidden:** The server refuses access to the requested resource.
- **404 Not Found:** The requested resource could not be found.
- **500 Internal Server Error:** The server encountered an unexpected condition.

6.3 Browser Developer Tools

Developer tools help inspect HTML elements, view computed CSS, monitor network requests, read console messages, and test responsive layouts. They are useful for learning because a developer can change a style temporarily, observe the result, and then apply a tested correction to the source code.

6.4 Development Tools and Standards

A web developer commonly uses a code editor, browser, developer tools, image editor, version-control system, and a local testing environment. A code editor helps organise files and detect syntax mistakes; a browser displays the result; developer tools help investigate layout, script, and network problems; and version control records changes so that earlier working versions can be recovered.

HTML, CSS, and JavaScript are interpreted by browsers according to web standards. Consistent indentation, meaningful file names, and a clear folder structure make a

project easier to review. A small site might use `index.html`, `styles.css`, `script.js`, and an `images` folder. Keep source files separate by responsibility unless a particular platform requires another structure.

6.5 Common Application Development Models

The Waterfall model organises work into sequential phases such as requirements, design, implementation, testing, and maintenance. It can be straightforward when requirements are stable, but changes discovered late may be expensive. The Iterative model develops a solution through repeated cycles, refining it after feedback. Agile approaches organise work into small increments and encourage frequent collaboration, review, and adaptation.

No single model is best for every project. A small student website may be planned with a simple checklist and improved after each review. A regulated or large system may need formal documentation and approval steps. The useful question is whether the chosen process helps the team deliver the required quality while responding appropriately to change.

Review Questions

1. Describe the basic request-and-response cycle of a web page.
2. Why is HTTPS preferred when information is sent between a browser and server?
3. Explain the difference between a 404 response and a 500 response.
4. Name three tasks that browser developer tools can support.

Chapter 7

Semantic HTML and Accessible Structure

7.1 Meaningful HTML Elements

Semantic HTML uses elements that communicate the role of content. Elements such as `header`, `nav`, `main`, `article`, `section`, and `footer` provide more meaning than using generic containers for every part of a page. Clear structure can improve maintainability and help assistive technologies interpret the document.

```
<header>
  <h1>Community Learning Centre</h1>
  <nav aria-label="Main navigation">
    <a href="#courses">Courses</a>
    <a href="#contact">Contact</a>
  </nav>
</header>
<main>
  <section id="courses">
    <h2>Available courses</h2>
    <p>Explore the current learning options.</p>
  </section>
</main>
<footer>
  <p>Contact the centre for further information.</p>
</footer>
```

7.2 Headings and Content Hierarchy

Headings should describe the topic beneath them and follow a logical hierarchy. A page usually has one main heading, followed by section headings and, where necessary, subheadings. Heading levels should not be selected merely to obtain a desired font size; visual appearance belongs in CSS, while the heading element communicates structure.

7.3 Lists and Grouped Information

HTML supports ordered lists, unordered lists, and description lists. Use an unordered list when the order does not matter, such as a list of required materials. Use an ordered list when the sequence matters, such as installation steps. A description list can pair terms with their explanations.

```
<h2>Preparation checklist</h2>
<ul>
  <li>Open the code editor.</li>
  <li>Create a project folder.</li>
  <li>Save the first HTML page.</li>
</ul>

<h2>Development sequence</h2>
<ol>
  <li>Plan the page.</li>
  <li>Build the structure.</li>
  <li>Test the result.</li>
```

```
</ol>
```

Nested lists are useful when a main item has related sub-items, but deep nesting can make information difficult to scan. Choose the list type according to the meaning of the content, not its default appearance.

7.4 Links, Images and Alternative Text

Link text should tell users what will happen when it is selected. Phrases such as “Read the admission requirements” are more informative than repeated “click here” links. Images that communicate information should have concise alternative text. Decorative images can use empty alternative text so that assistive technologies do not announce unnecessary details.

7.5 Forms and Labels

A form control should have a clear label. Placeholder text is not a replacement for a label because it disappears when the user enters a value and may be difficult to read. Required fields should be indicated clearly, and error messages should explain how to correct a problem.

```
<form>
  <label for="email">Email address</label>
  <input id="email" name="email" type="email" required>
  <button type="submit">Send request</button>
</form>
```

Review Questions

1. What is the purpose of semantic HTML?
2. Explain why heading levels should follow a meaningful order.
3. What makes alternative text useful?
4. Why should a form input have an associated label?

Chapter 8

CSS Selectors, Cascade and Responsive Layout

8.1 Selectors

CSS selectors identify the elements to which a rule applies. Element selectors target a tag name, class selectors target a reusable class, and ID selectors target an element with a specific identifier. Selectors should be chosen to make styles understandable and easy to maintain.

```
/* Element selector */
p { line-height: 1.6; }

/* Reusable class selector */
.notice {
  border-left: 4px solid #007F86;
  padding: 1rem;
}

/* Layout for a group of cards */
.card-grid {
  display: grid;
  grid-template-columns: repeat(3, minmax(0, 1fr));
  gap: 1rem;
}
```

8.2 Cascade, Specificity and Inheritance

When more than one CSS rule can apply to an element, the cascade determines which declaration wins. Origin, importance, specificity, and source order affect the result. Inherited properties such as text colour may pass from a parent to its children, while properties such as margin do not normally inherit. Overusing highly specific selectors can make future changes difficult.

8.3 Responsive Design

Responsive design helps a layout adapt to different screen sizes. Flexible widths, relative units, CSS Grid, Flexbox, and media queries can be combined to create interfaces that work on phones, tablets, laptops, and larger screens. A responsive page should not merely shrink; content must remain readable and controls must remain usable.

```
.card-grid {
  display: grid;
  grid-template-columns: repeat(3, minmax(0, 1fr));
  gap: 1rem;
}
@media (max-width: 700px) {
  .card-grid {
    grid-template-columns: 1fr;
  }
}
```

8.4 Flexbox and Grid

Flexbox is useful for arranging items along one primary axis, such as a navigation row. Grid is useful when a layout needs rows and columns together, such as a card gallery. Choosing the simpler layout model usually makes the code easier to understand.

Review Questions

1. Compare class selectors and ID selectors.
2. What does CSS specificity help determine?
3. Explain one practical difference between Flexbox and Grid.
4. How can a media query improve mobile usability?

Chapter 9

JavaScript Variables, Types and Operators

9.1 Storing Values

Variables give names to values used by a program. JavaScript provides `const` for bindings that should not be reassigned and `let` for bindings that may change. `var` is part of the language but has different scoping behaviour and is usually avoided in new beginner examples.

```
const courseName = "Web Development";
let completedLessons = 3;
completedLessons = completedLessons + 1;

const isReady = completedLessons >= 4;
console.log(courseName, completedLessons, isReady);
```

9.2 Common Data Types

Strings represent text, numbers represent numeric values, booleans represent true or false, and arrays hold ordered collections. Objects group related properties. The value `null` can represent an intentional absence of a value, while `undefined` commonly indicates that a value has not been assigned or is missing.

9.3 Operators and Comparisons

Arithmetic operators perform calculations. Comparison operators produce Boolean results, and logical operators combine conditions. Strict equality (===) compares values without automatically converting different types, which makes it a good default for most comparisons.

9.4 Readable and Safe Code

Descriptive variable names, consistent indentation, and small expressions make code easier to review. Validate user-provided data before relying on it. Never treat browser-side validation as the only protection for a system that stores sensitive data; server-side checks are also required when a server is involved.

Review Questions

1. Compare `let` and `const`.
2. Name four JavaScript data types and give an example of each.
3. Why is strict equality often preferred?
4. Why should important validation not rely only on browser-side code?

Chapter 10

Conditions, Loops, Functions and Arrays

10.1 Conditional Decisions

Conditional statements allow a program to choose an action based on a condition. An `if/else` statement can validate input, choose a message, or control which content is shown. Conditions should be written clearly and tested for both expected and unexpected values.

```
const score = 76;
if (score >= 80) {
  console.log("Excellent");
} else if (score >= 50) {
  console.log("Pass");
} else {
  console.log("Needs more practice");
}
```

10.2 Loops

Loops repeat an operation while a condition is true or for each item in a collection. A loop needs a clear stopping condition; otherwise it may continue indefinitely. For arrays, methods such as `forEach`, `map`, and `filter` can express common operations when used appropriately.

10.3 Functions

A function groups instructions into a reusable unit. Parameters receive input, and a return value provides a result. Small functions are easier to test than one long block of mixed responsibilities.

```
function calculateAverage(values) {
  if (values.length === 0) return 0;
  const total = values.reduce((sum, value) => sum + value, 0);
  return total / values.length;
}
console.log(calculateAverage([70, 80, 90]));
```

10.4 Arrays and Objects

Arrays are suitable for ordered lists such as course names. Objects are suitable for related attributes such as a course title, duration, and level. Choosing an appropriate data structure reduces repetition and helps the program represent information clearly.

10.5 Working with Strings

A string is a sequence of text characters. JavaScript provides properties and methods for checking length, changing letter case, finding text, extracting a portion, and joining values. String methods usually return a result rather than changing the original string, so store the returned value when a change is needed.

```
const learner = "  Siva Sai  ";
```

```
const cleaned = learner.trim();
const greeting = "Welcome, " + cleaned + "!";
console.log(greeting);
console.log(cleaned.toUpperCase());
```

When processing names or messages, consider empty input and extra spaces. Do not assume that every text value is valid simply because it is a string; validation should match the purpose of the field.

Review Questions

1. Explain how an `if`/`else` statement makes a decision.
2. What is the purpose of a loop's stopping condition?
3. Define function parameters and return values.
4. Give one example of information that is better represented by an object than by separate variables.

Chapter 11

DOM, Events and Form Interaction

11.1 The Document Object Model

The Document Object Model (DOM) represents a page as a tree of objects. JavaScript can select elements, read or update text, change classes, and respond to events. Changes should be made deliberately so that the page remains understandable and accessible.

```
const button = document.querySelector("#show-message");
const message = document.querySelector("#message");

button.addEventListener("click", () => {
  message.textContent = "Your action was received.";
});
```

11.2 Events

Events describe interactions such as clicks, keyboard input, form submission, and changes to a field. An event listener connects an event to a function. For forms, listening to the submit event makes it possible to validate data before displaying a result or sending a request.

11.3 Validation and Feedback

Good validation explains what needs correction. It should preserve user input where possible, identify the field with an issue, and avoid relying on colour alone to communicate errors. A successful action should also provide clear confirmation.

11.4 Safe DOM Updates

When displaying untrusted text, prefer `textContent` rather than inserting arbitrary HTML with `innerHTML`. This reduces the risk of interpreting user input as markup or script. More complex applications need additional security controls appropriate to their data and architecture.

11.5 Drawing with the HTML Canvas

The HTML `canvas` element provides a drawing surface that JavaScript can control. It is useful for simple charts, sketches, or interactive visual demonstrations. Canvas content is drawn with script and is not automatically exposed as ordinary text to assistive technology; provide a text explanation or an accessible alternative where the drawing conveys important information.

```
<canvas id="demo-canvas" width="320" height="140">
  A simple coloured rectangle illustration.
</canvas>
```

```
const canvas = document.querySelector("#demo-canvas");
const context = canvas.getContext("2d");

context.fillStyle = "#007F86";
context.fillRect(20, 20, 180, 70);
```

```
context.fillStyle = "#17365D";
context.font = "16px sans-serif";
context.fillText("Web Developer", 30, 115);
```

The canvas dimensions define its drawing surface. CSS can control how large it appears on the page, but changing only its displayed size may scale the drawing. For beginner projects, keep the example small and test it in the browser.

11.6 Using JavaScript with Images

JavaScript can respond to a button click by changing an image's source, updating alternative text, or adjusting presentation classes. Alternative text should continue to describe the image's purpose after a change. Images should have appropriate dimensions and should not be stretched in ways that distort their content.

```
<img id="course-image"
     src="images/html-basics.jpg"
     alt="HTML code displayed in an editor">
<button id="change-image" type="button">
  Show CSS image
</button>
```

```
const image = document.querySelector("#course-image");
const changeButton = document.querySelector("#change-image");

changeButton.addEventListener("click", () => {
  image.src = "images/css-basics.jpg";
  image.alt = "CSS rules displayed in an editor";
});
```

The image paths must point to files that actually exist in the project. If an image cannot load, provide a useful fallback or ensure that the page still communicates its essential information.

Review Questions

1. What does the DOM represent?
2. Describe how an event listener works.
3. Give two characteristics of helpful form validation.
4. Why is `textContent` a safer default for displaying plain user input?

Chapter 12

Testing, Debugging and Quality Assurance

Figure 12.1. Careful development and review are part of building reliable websites. Photo: Free-Photos / Pixabay, CC0 1.0.

12.1 Why Testing Matters

Testing checks whether a website behaves as expected and helps detect defects before release. Testing should cover more than whether a page appears on one developer's computer. Different screen sizes, keyboard use, input errors, navigation paths, and browser behaviours can reveal problems that are otherwise missed.

12.1.1 Useful Test Categories

- **Functional testing:** Confirms links, forms, buttons, and core features behave correctly.
- **Visual testing:** Checks alignment, spacing, contrast, and layout consistency.
- **Responsive testing:** Checks content at narrow and wide viewport sizes.
- **Accessibility testing:** Checks keyboard navigation, labels, headings, and alternative text.
- **Compatibility testing:** Checks behaviour in supported browsers and devices.
- **Regression testing:** Rechecks existing features after a change.

12.2 A Practical Debugging Method

First reproduce the issue and record the steps. Next inspect the relevant element, console message, network response, or source code. Form a small hypothesis, change one thing at a time, and repeat the test. Once fixed, check nearby features to make sure the correction did not introduce another defect.

12.3 Test Case Example

Field	Example
Test objective	Verify required email input.
Starting condition	Contact form is open.
Action	Submit without entering an email address.
Expected result	The form identifies the missing required field and does not report a successful submission.
Observed result	Record what actually happened.
Status	Pass or fail after comparison with the expected result.

Review Questions

1. Why is responsive testing important?
2. Distinguish functional testing from regression testing.
3. List the steps in a systematic debugging process.
4. What information should a useful test case record?

Chapter 13

Accessibility, Performance and Secure Publishing

13.1 Accessibility Essentials

Accessible design helps people with different abilities use a website. Provide clear headings, descriptive links, form labels, sufficient colour contrast, keyboard access, and useful alternative text. Interactive controls should show a visible focus indicator. Automated checks can help, but manual keyboard and content checks remain valuable.

13.2 Performance Basics

Large images, unnecessary scripts, repeated network requests, and complex layouts can slow a page. Resize images appropriately, use modern image formats where supported, avoid loading resources that are not needed, and keep code organised. Performance should be evaluated with measurements rather than guesses.

13.3 Security Awareness

Use HTTPS for published sites. Do not place passwords, private API keys, or confidential tokens in public HTML, CSS, JavaScript, or source repositories. Client-side code is visible to visitors. If a feature requires a secret credential, it generally belongs in a protected server-side environment. Keep software dependencies maintained and collect only the information a site actually needs.

13.4 Publishing Checklist

Before publishing, verify that links work, page titles are meaningful, images have suitable alternative text, forms give clear feedback, layouts work on mobile screens, and no test credentials or private data remain in the repository. Confirm the correct files are deployed and keep a copy of the source so that later updates can be made safely.

Review Questions

1. List five practices that improve accessibility.
2. Name three common causes of slow web pages.
3. Why should secret API keys not be stored in browser-side JavaScript?
4. Create a short checklist for publishing a small informational website.

Chapter 14

Integrated Mini Project: Learning Centre Website

14.1 Project Brief

Create a small informational website for a learning centre. The website should display the centre name, a list of courses, basic contact information, and a contact form interface. The project demonstrates the combined roles of HTML for structure, CSS for presentation, and JavaScript for interaction.

14.2 Suggested Page Structure

- Header containing the site name and primary navigation.
- Main section describing the centre and its courses.
- Course cards with titles, brief descriptions, and duration.
- Contact section with labelled form fields.
- Footer containing non-sensitive contact and copyright information.

14.3 Implementation Plan

Begin by creating semantic HTML. Add CSS rules for typography, colour, spacing, and a responsive card layout. Add JavaScript only where it improves the user experience, such as providing a clear message after a valid client-side form submission. If the project is later connected to a real service, server-side validation and secure handling of submitted information will be required.

14.4 Evaluation Rubric

Criterion	Marks
Semantic structure and meaningful headings	20
Readable and consistent visual design	20
Responsive layout	20
Working interaction and useful feedback	15
Accessibility checks	15
Code organisation and testing notes	10
Total	100

Review Questions

1. Identify the HTML, CSS, and JavaScript responsibilities in the project.
2. Describe how you would test the site on a narrow mobile screen.
3. What security changes would be required before collecting real contact details?
4. Suggest two improvements that could be included in a future version.

HANDBOOK

Web Developer

Grade XII – IT-ITeS Vocational Education

MySQL • SQL • PHP • Workplace Safety

A structured textbook for database development, server-side programming and responsible workplaces.

Developed with reference to the topics in the supplied Grade XII draft study material.

Contents

1 Database Management Concepts **1**
1.1 Data, databases and DBMS . . . 1
1.2 Tables, records, fields and keys . . . 1
1.3 Relational and other database models . . . 2
1.4 File systems and database systems . . . 2
1.5 Normalisation and data quality . . . 2
1.6 Transactions and ACID . . . 2

2 Structured Query Language and MySQL **4**
2.1 Main categories of SQL commands . . . 4
2.2 Creating a database and table . . . 4
2.3 Retrieving and filtering data . . . 5
2.4 Adding, updating and deleting records . . . 5
2.5 Functions, grouping and aggregation . . . 5
2.6 Joins and relationships . . . 5
2.7 Installation and safe database practice . . . 6

3 SQL Functions and Data Handling **7**
3.1 String functions . . . 7
3.2 Numeric functions . . . 7
3.3 Date and time functions . . . 7
3.4 Aggregate functions and NULL values . . . 7
3.5 Permissions and transactions . . . 7

4 Introduction to PHP and Server-Side Scripting **9**
4.1 How a PHP request works . . . 9
4.2 Running a first PHP script . . . 9
4.3 Variables, types and operators . . . 10
4.4 Conditions and loops . . . 10
4.5 Forms and request data . . . 10

5 PHP Arrays, Strings and Functions **11**
5.1 Arrays . . . 11
5.2 Working with strings . . . 11
5.3 Functions and parameters . . . 11
5.4 Server variables and HTTP headers . . . 12
5.5 Error handling and maintainability . . . 12

6 PHP and MySQL Database Applications **13**
6.1 Connection choices . . . 13
6.2 Prepared statements . . . 13
6.3 Create, read, update and delete . . . 14
6.4 SQL injection . . . 14

6.5 Output encoding and sessions 14
6.6 A responsible development checklist 14

7 Productivity, Quality and Professional Work Habits 15
7.1 Planning and prioritising 15
7.2 Quality and continuous improvement 15
7.3 Communication and teamwork 15
7.4 Managing digital work responsibly 15

8 Health and Safety Procedures in the Workplace 17
8.1 Hazard and risk 17
8.2 Hierarchy of controls 17
8.3 Computer-work ergonomics 18
8.4 Electrical safety and emergency response 18
8.5 Incident reporting 18

9 Waste Management, Recycling and Resource Conservation 19
9.1 Identify and separate waste 19
9.2 The waste hierarchy 19
9.3 Electronic waste 20
9.4 Conserving energy and water 20
9.5 Planning a small waste programme 20

10 Diversity, Equality and Inclusive Workplaces 21
10.1 Diversity and equality 21
10.2 Everyday inclusive behaviour 21
10.3 Discrimination and harassment 21
10.4 Gender equality and work-life responsibilities 21
10.5 Building an inclusive team 21

11 Accessibility and Policies for Persons with Disabilities 22
11.1 Recognising different access needs 22
11.2 Digital accessibility 22
11.3 Physical and workplace adjustments 22
11.4 Respectful support and privacy 22

12 Integrated Case Study and Revision 24
12.1 Case study: a training-centre portal 24
12.2 Plan the database 24
12.3 Plan the application 24
12.4 Plan quality and workplace responsibility 24
12.5 Revision questions 24
12.6 Answer-planning guide 25

Chapter 1

Database Management Concepts

A database becomes useful when information can be stored consistently, found quickly and updated without creating confusion. A school may need student records, a shop may maintain stock and sales, and a web application may keep account details and orders. A database management system (DBMS) provides the tools needed to organise this information and control how people and applications use it.

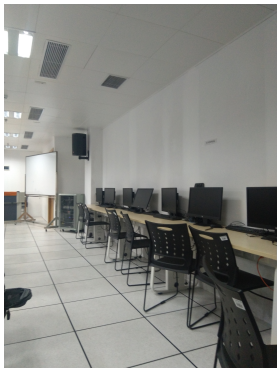


Figure 1.1. A computer room with server equipment illustrates infrastructure that supports database systems. Photograph: StellaAnnie, Wikimedia Commons, CC BY-SA 4.0.

1.1 Data, databases and DBMS

Data is a recorded value such as a name, date, price or attendance status. A database is an organised collection of related data. A DBMS is software that creates, stores, retrieves and protects that data. MySQL is a widely used relational DBMS; other examples include PostgreSQL, Oracle Database and Microsoft SQL Server.

A database system supports more than storage. It can enforce rules, control access, manage simultaneous users, support backup and recovery, and help applications retrieve only the information they need. These controls reduce the risk of inconsistent or lost records.

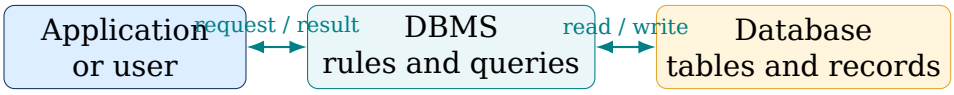


Figure 1.2. The DBMS mediates between applications and stored data.

1.2 Tables, records, fields and keys

In a relational database, a table stores one kind of entity. Rows represent records; columns represent attributes. A student table might contain student ID, name, class and date of admission. Each column has a data type, and each row should be identifiable in a reliable way.

A **primary key** uniquely identifies each row and cannot be missing. A **foreign key** links a row to a matching key in another table. A **unique constraint** prevents duplicate values in a column or column group. A **NOT NULL constraint** requires a value,

while a **CHECK constraint** can enforce a condition supported by the database engine.

1.3 Relational and other database models

Relational databases organise data in related tables and use SQL to work with it. NoSQL databases use other models, including document, key-value, column-family and graph structures. The appropriate model depends on the shape of the data, consistency needs, access patterns and operational requirements; no single model is best for every application.

1.4 File systems and database systems

A file system stores files and folders. It is appropriate for documents, images and many ordinary files. A DBMS adds structured queries, constraints, relationships and transaction handling that are valuable when many records must be managed together. File storage and databases often work side by side in one application.

Aspect	File-based approach	Database approach
Structure	Files and folders; format depends on the application	Tables, columns, types and relationships
Searching	Often depends on filenames or application-specific logic	SQL queries can filter, sort and join records
Integrity	Rules are often implemented separately in code	Constraints can be enforced centrally
Concurrent changes	Applications must coordinate carefully	DBMS provides transaction and concurrency controls
Security	File permissions and application controls	Database accounts, privileges and application controls
Recovery	File backups and application procedures	Backup, logging and database recovery features

1.5 Normalisation and data quality

Normalisation is a way of organising tables to reduce unnecessary repetition and update anomalies. For example, storing a department's address in every employee row means an address change must be repeated in many places. A separate department table allows employees to refer to one department record. Good normalisation begins with clear entities, keys and relationships; it should also reflect actual reporting and performance needs.

1.6 Transactions and ACID

A transaction groups operations that should succeed or fail as a unit. ACID summarises four important properties: Atomicity (all or nothing), Consistency (valid rules remain satisfied), Isolation (concurrent work is controlled), and Durability (committed changes persist according to the system's guarantees). Transactions matter in banking, ordering, stock control and any workflow where partial updates would create a misleading result.

Review Questions

1. Define DBMS and give two examples.

2. Distinguish a row from a column.
3. Explain primary and foreign keys.
4. State two advantages and two limitations of a file-based approach.
5. What does each ACID property protect?

Chapter 2

Structured Query Language and MySQL

SQL is used to define database structures, retrieve records, change data and manage access. Although database products differ in some details, the basic ideas of tables, filtering, sorting, joins and constraints are widely shared.

Figure 2.1. A code screen illustrates the syntax used to write and review database queries. Photograph: Lorenzo Cafaro, Wikimedia Commons, CC0 1.0.

2.1 Main categories of SQL commands

- **DDL (Data Definition Language):** CREATE, ALTER and DROP define or change database objects.
- **DML (Data Manipulation Language):** INSERT, UPDATE and DELETE change records.
- **DQL (Data Query Language):** SELECT retrieves information; this label is often used for query statements.
- **DCL (Data Control Language):** GRANT and REVOKE manage privileges.
- **TCL (Transaction Control Language):** COMMIT and ROLLBACK manage transaction outcomes.

2.2 Creating a database and table

The following examples use MySQL-style syntax. Run them in a practice database, not against real records.

```
CREATE DATABASE training_centre;
USE training_centre;

CREATE TABLE students (
  student_id INT PRIMARY KEY,
  full_name VARCHAR(100) NOT NULL,
  course_name VARCHAR(80),
  marks DECIMAL(5,2) CHECK (marks >= 0 AND marks <= 100)
);
```

Choose data types that fit the values. INT is suitable for whole numbers; VARCHAR stores variable-length text; DATE stores a calendar date; DECIMAL is appropriate

for exact decimal quantities such as money. Avoid storing every value as text when calculations, sorting or validation matter.

2.3 Retrieving and filtering data

SELECT names the columns to return. WHERE filters rows, ORDER BY sorts the result, and LIMIT can restrict the number of returned rows in MySQL.

```
SELECT student_id, full_name, marks
FROM students
WHERE marks >= 60
ORDER BY marks DESC
LIMIT 10;
```

Use explicit column names rather than relying on SELECT * in long-lived application queries. Explicit selections make the expected output easier to review and reduce accidental exposure when a table gains new columns.

2.4 Adding, updating and deleting records

```
INSERT INTO students (student_id, full_name, course_name, marks)
VALUES (101, 'Ananya Rao', 'Web Development', 86.50);

UPDATE students
SET marks = 88.00
WHERE student_id = 101;

DELETE FROM students
WHERE student_id = 101;
```

UPDATE and DELETE should be used with particular care. Before running a change, check the WHERE condition with a SELECT query and confirm that it identifies exactly the intended records. In real systems, use appropriate permissions, backups and transactions.

2.5 Functions, grouping and aggregation

SQL functions can transform values or summarise a set of records. String functions include UPPER, LOWER, CONCAT and LENGTH; numeric functions include ROUND, CEIL, FLOOR and ABS; date functions depend on the DBMS. Aggregate functions such as COUNT, SUM, AVG, MIN and MAX calculate a summary across rows.

```
SELECT course_name,
       COUNT(*) AS learner_count,
       ROUND(AVG(marks), 2) AS average_marks
FROM students
GROUP BY course_name
ORDER BY average_marks DESC;
```

GROUP BY creates groups for aggregation. HAVING filters groups after aggregation, while WHERE filters rows before grouping. NULL values require care: most aggregate functions ignore NULL inputs, and comparisons with NULL use IS NULL or IS NOT NULL rather than ordinary equality.

2.6 Joins and relationships

A JOIN combines rows from related tables. An INNER JOIN returns matching rows. A LEFT JOIN retains every row from the left table and includes matching data from

the right where available. Joins are easier to reason about when key columns are named clearly and their relationships are documented.

```
SELECT s.full_name, c.course_title
FROM students AS s
INNER JOIN courses AS c
  ON s.course_id = c.course_id;
```

2.7 Installation and safe database practice

For learning, use the official MySQL Community Edition download and follow current installation instructions for your operating system. A typical setup may include MySQL Server and a client such as MySQL Workbench. Create a dedicated practice account, use a strong password, keep the server local while learning, and do not expose the database port to the public internet without a carefully designed security plan.

Review Questions

1. Compare DDL and DML.
2. What is the purpose of a primary key?
3. Write a query to sort marks from highest to lowest.
4. Explain the difference between WHERE and HAVING.
5. Compare INNER JOIN and LEFT JOIN.
6. Why should UPDATE and DELETE statements be checked before execution?

Chapter 3

SQL Functions and Data Handling

Functions help queries express common calculations without requiring every operation to be performed manually outside the database. A function accepts a value or expression and returns a result. Exact names and behaviour can differ across database products, so check the documentation for the system in use.

3.1 String functions

UPPER and LOWER change letter case, CONCAT joins text, LENGTH reports the length according to the DBMS's rules, and TRIM removes surrounding whitespace. SUBSTRING returns part of a string. These operations are useful for formatting output, preparing search values and producing reports. They should not be used as a substitute for storing clean, well-validated data.

```
SELECT UPPER(full_name) AS display_name,
       LENGTH(full_name) AS name_length
FROM students;
```

3.2 Numeric functions

ABS returns a non-negative magnitude, ROUND rounds a number to a specified precision, and CEIL/FLOOR round upward/downward to integer boundaries. Check the effect of rounding before using it in financial or assessment calculations; the displayed value and stored value are not always the same.

3.3 Date and time functions

Date functions can extract parts of a date, compare dates, format a value or calculate intervals. MySQL provides functions such as CURDATE(), NOW(), YEAR() and `DATE_FORMAT()`. Other SQL systems may use different names or arguments. Store dates using date/time types when possible, rather than ambiguous strings such as 03/04/25.

3.4 Aggregate functions and NULL values

COUNT counts rows or non-NULL values depending on its argument; SUM totals numeric values; AVG computes a mean; MIN and MAX return the smallest and largest values. NULL means that a value is unknown or absent, not zero and not an empty string. Queries should handle missing values intentionally.

```
SELECT COUNT(*) AS total_records,
       COUNT(marks) AS records_with_marks,
       AVG(marks) AS mean_marks,
       MIN(marks) AS lowest_marks,
       MAX(marks) AS highest_marks
FROM students;
```

3.5 Permissions and transactions

GRANT assigns permitted actions to a database account; REVOKE removes permissions. Follow least privilege: give each account only the access it needs. COMMIT

confirms a transaction and ROLLBACK abandons uncommitted changes. Transaction support and behaviour depend on the storage engine and the operations being performed.

Review Questions

1. Give examples of string and numeric functions.
2. Compare COUNT(*) with COUNT(column).
3. Why should dates be stored using date types?
4. What is NULL?
5. Explain least privilege in database access.

Chapter 4

Introduction to PHP and Server-Side Scripting

PHP is a server-side scripting language often used to create dynamic web pages and web services. The server executes PHP code and sends a response to the browser. Visitors normally receive the resulting HTML, not the server's PHP source. PHP can process form input, apply application rules, communicate with databases and generate a response.

Figure 4.1. Server-side scripts are written and tested in a programming environment. Photograph: Martin Vorel, Wikimedia Commons, CC BY-SA 4.0.

4.1 How a PHP request works

Figure 4.2. A typical request-response path for a server-side application.

4.2 Running a first PHP script

Use a supported PHP installation or a local development bundle, and follow the current official installation instructions. Save the following as `hello.php` in the configured project directory and open it through the local development server.

```
<?php
$course = "Web Development";
echo "<h1>Welcome to " . htmlspecialchars($course, ENT_QUOTES, 'UTF-8') . "
    </h1>";
?>
```

PHP variables begin with a dollar sign. Statements commonly end with a semicolon. The `echo` construct outputs content. When displaying user-controlled text inside HTML, escape it for the correct output context; escaping is a security control, not just a formatting choice.

4.3 Variables, types and operators

Variables hold values such as strings, integers, floating-point numbers, booleans, arrays and objects. PHP is dynamically typed, but developers should still validate input and make assumptions clear. Arithmetic operators calculate values; comparison operators compare values; logical operators combine conditions. Use strict comparisons where appropriate and avoid depending on surprising type conversions.

4.4 Conditions and loops

```
$marks = 74;
if ($marks >= 80) {
    echo "Excellent";
} elseif ($marks >= 50) {
    echo "Pass";
} else {
    echo "Needs more practice";
}
```

Conditions allow code to choose between actions. Loops repeat work while a condition holds or for each item in a collection. Every loop needs a sensible stopping condition so that it does not run forever.

4.5 Forms and request data

HTML forms commonly submit data with GET or POST. GET data is typically included in the URL and is suitable for retrieval requests; POST sends data in the request body and is often used for changes. POST does not automatically make data private or safe: use HTTPS, validation, access control and appropriate request protections.

```
<?php
if ($_SERVER['REQUEST_METHOD'] === 'POST') {
    $name = trim($_POST['name'] ?? '');
    if ($name === '') {
        echo "Please enter a name.";
    } else {
        echo "Received a valid name.";
    }
}
?>
```

The example demonstrates basic checking only. A real form should include a suitable HTML form, a clear label, server-side validation, CSRF protection where relevant, safe output encoding and meaningful error handling.

Review Questions

1. What is server-side scripting?
2. Describe the journey of an HTTP request to a PHP server.
3. What is the difference between GET and POST?
4. Why should user-provided text be escaped before it is inserted into HTML?

Chapter 5

PHP Arrays, Strings and Functions

5.1 Arrays

An array stores several values under one variable. An indexed array uses numeric keys, an associative array uses named keys, and a multidimensional array contains arrays inside another array. Select the structure that best represents the data.

```php
$courses = ["HTML", "CSS", "JavaScript"];
$learner = [
    "name" => "Ananya",
    "course" => "Web Development",
    "completed" => true
];

foreach ($courses as $course) {
    echo htmlspecialchars($course, ENT_QUOTES, 'UTF-8') . "<br>";
}
```

Array functions can sort values, count elements, test for keys and transform data. Be mindful that some sorting functions reindex numeric keys, while others preserve key associations. Check the function's documented behaviour before depending on the original order.

5.2 Working with strings

String operations can trim extra spaces, change letter case, find a substring, split text or join pieces. When processing input, consider empty strings, unexpected encodings and very long values. A value should be checked according to its intended meaning, not merely accepted because it is a string.

5.3 Functions and parameters

A function gives a name to a reusable task. Parameters receive input, and a return value communicates a result. Small functions are easier to test, reuse and maintain than long blocks that mix unrelated responsibilities.

```php
function calculateAverage(array $scores): ?float {
    if (count($scores) === 0) {
        return null;
    }
    return array_sum($scores) / count($scores);
}

$average = calculateAverage([70, 80, 90]);
```

PHP passes values by value by default; a parameter can be explicitly passed by reference using an ampersand in its declaration. References should be used only when changing the caller's variable is part of the function's intended behaviour. Recursive functions call themselves and must have a base case to stop the recursion.

5.4 Server variables and HTTP headers

The `$_SERVER` superglobal provides request and server information, such as the request method and script path. Values should not be blindly trusted as proof of identity or permission. HTTP headers communicate metadata and response instructions. Header operations must be sent before body output, and redirects should be followed by an exit where appropriate.

5.5 Error handling and maintainability

Use clear names, consistent formatting, comments that explain intent, and functions with a focused purpose. During development, errors can be logged for diagnosis. Production systems should avoid showing stack traces, secrets or internal paths to visitors. Logs should not unnecessarily contain passwords, session tokens or sensitive personal information.

Review Questions

1. Compare indexed and associative arrays.
2. What is the purpose of a foreach loop?
3. Explain function parameters and return values.
4. What is a base case in recursion?
5. Why should internal error details not be displayed to every visitor?

Chapter 6

PHP and MySQL Database Applications

A web application often needs to save and retrieve information. PHP can connect to MySQL, send queries and use the returned rows to generate a response. The connection, query and output stages all need careful error handling and security controls.

Figure 6.1. Web applications depend on server systems to process requests and access stored data. Photograph: Ricardo Moctezuma López / ronK, Wikimedia Commons, CC BY-SA 2.5.

6.1 Connection choices

Modern PHP applications commonly use PDO or MySQLi. Both support prepared statements. PDO offers a consistent interface across supported database drivers; MySQLi is designed for MySQL-compatible databases. Use the interface supported by the project and keep credentials outside public source files, for example in environment configuration that is not committed to a repository.

6.2 Prepared statements

Never build a query by joining untrusted input directly into SQL text. Prepared statements separate the query structure from the supplied values and help prevent SQL injection.

```
<?php
$pdo = new PDO(
    'mysql:host=127.0.0.1;dbname=training_centre;charset=utf8mb4',
    getenv('DB_USER'),
    getenv('DB_PASSWORD'),
    [PDO::ATTR_ERRMODE => PDO::ERRMODE_EXCEPTION]
);

$stmt = $pdo->prepare(
    'SELECT student_id, full_name FROM students WHERE student_id = :id'
);
$stmt->execute(['id' => $studentId]);
$student = $stmt->fetch(PDO::FETCH_ASSOC);
?>
```

This is a learning pattern, not a complete production configuration. The variable `$studentId` must be validated, the credentials must be configured securely, and exceptions should be handled by the application without exposing sensitive details to the browser.

6.3 Create, read, update and delete

CRUD describes four common data operations: create a record, read records, update existing information and delete records. Each operation should verify that the user is authorised to perform it. Deleting records may require confirmation, audit logging or a retention process depending on the system's requirements.

6.4 SQL injection

SQL injection occurs when untrusted input is interpreted as part of a query's executable SQL syntax. It may expose data, bypass checks or modify records. Prepared statements are the primary defence for values; least-privilege database accounts, validation, safe error handling and security testing add further protection. Escaping alone is not a substitute for prepared statements.

6.5 Output encoding and sessions

When displaying text in HTML, use context-appropriate output encoding, such as `htmlspecialchars` for ordinary HTML text. Authentication systems should use secure session settings, regenerate session identifiers after login, enforce permissions on the server and protect sensitive actions. Hiding a button in the browser is not authorisation.

6.6 A responsible development checklist

- Use a separate database account with only the privileges needed.
- Store credentials in environment variables or a protected configuration system.
- Use prepared statements for values in SQL queries.
- Validate input on the server and encode output for its context.
- Use HTTPS for real deployments and protect sessions.
- Log useful security events without recording secrets.
- Back up data and test recovery procedures.

Review Questions

1. Name two PHP interfaces used with MySQL.
2. How do prepared statements reduce SQL injection risk?
3. What do CRUD operations mean?
4. Why is output encoding necessary?
5. Give four checks to complete before a database-backed form is released.

Chapter 7

Productivity, Quality and Professional Work Habits

Productivity is not simply completing a large number of tasks. Good work is accurate, useful and delivered with appropriate use of time and resources. In a software workplace, productivity improves when requirements are understood, tasks are prioritised, tools are maintained, communication is clear and defects are found early.

Figure 7.1. A workstation should be kept organised so equipment and materials can be used safely. Photograph: Frederick Noronha, Wikimedia Commons, CC BY-SA 4.0.

7.1 Planning and prioritising

Break large assignments into smaller tasks with clear outcomes. Estimate the effort, identify dependencies and reserve time for review. A simple task board can show work that is planned, in progress, waiting for review and complete. When priorities change, communicate the impact rather than silently missing a deadline.

7.2 Quality and continuous improvement

Quality means meeting agreed requirements reliably. Use checklists, peer review, testing and documented procedures. When an error occurs, look for the cause and improve the process instead of focusing only on blame. Useful measures might include defects found after release, rework, task completion and customer feedback; a single metric rarely tells the whole story.

7.3 Communication and teamwork

Share information in a clear and respectful way. Confirm important instructions, record decisions and raise risks early. When giving feedback, describe the observed issue, its impact and a practical next step. Protect confidential information and use approved channels for work discussions.

7.4 Managing digital work responsibly

Organise files with meaningful names, keep versions under control, back up important work and avoid saving confidential material in unapproved locations. Lock the screen when stepping away, use strong authentication and report suspicious messages or unexpected access requests.

Review Questions

1. Distinguish productivity from simply working faster.
2. Give three methods for improving quality.
3. Why should a project risk be reported early?
4. List four habits that support responsible digital work.

Chapter 8

Health and Safety Procedures in the Workplace

A safe workplace is created through planning, training, suitable equipment and shared responsibility. Hazards may be physical, electrical, ergonomic, chemical, environmental or psychosocial. Identifying a hazard early gives people a chance to remove it or reduce the risk before someone is harmed.

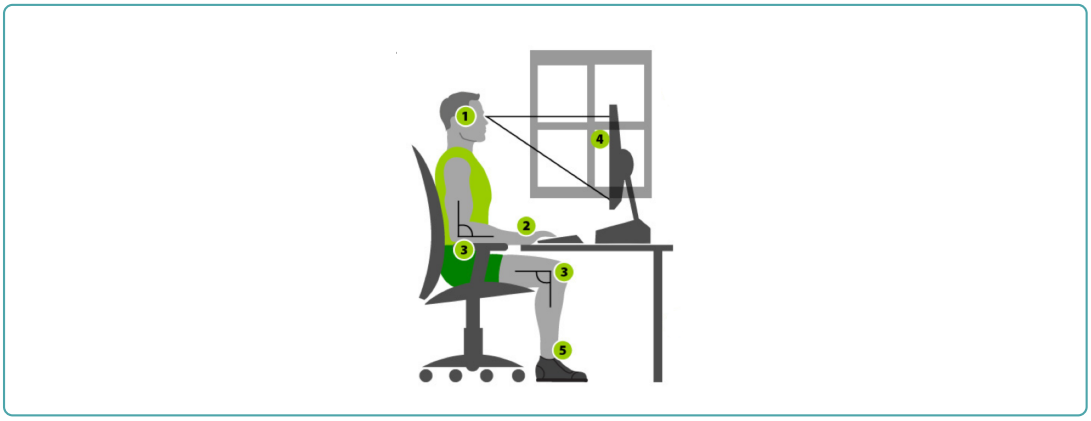


Figure 8.1. An ergonomic workstation supports comfortable computer use and reduces avoidable strain. Illustration: Marcel Kollmar and Partynia, Wikimedia Commons, CC BY-SA.

8.1 Hazard and risk

A hazard is something that can cause harm. Risk considers how likely that harm is and how serious it could be. A loose cable across a walkway is a hazard; the risk increases when people regularly walk through the area and could trip. Report hazards promptly and follow the organisation’s safety procedure.

8.2 Hierarchy of controls

Where possible, remove the hazard first. If elimination is not practical, consider substituting a safer option, using engineering controls, improving administrative procedures and using personal protective equipment where required. PPE is important but should not be the only control when more effective options are available.

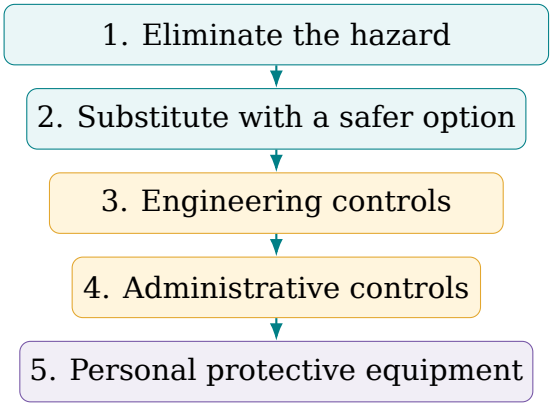


Figure 8.2. The hierarchy of controls prioritises stronger ways of reducing risk.

8.3 Computer-work ergonomics

Adjust the chair, desk and screen so the body can remain in a comfortable, neutral position. Keep frequently used items within easy reach, avoid glare and take regular short movement breaks. Repeated discomfort, headaches or eye strain should be reported and addressed rather than treated as an unavoidable part of computer work.

8.4 Electrical safety and emergency response

Do not use damaged plugs, exposed wires or overloaded extension boards. Keep liquids away from electrical equipment and never attempt repairs beyond your training or authorisation. Know emergency exits, alarms, first-aid arrangements and reporting contacts. In an emergency, follow local procedures and contact trained responders; do not put yourself at risk by attempting an unfamiliar rescue.

8.5 Incident reporting

Report injuries, near misses, unsafe conditions and equipment faults according to workplace policy. Record factual details such as location, time, what was observed and immediate controls taken. Timely reporting helps prevent recurrence and supports a safer environment for everyone.

Review Questions

1. Define hazard and risk.
2. Name the main levels in the hierarchy of controls.
3. Give five ways to improve computer-work ergonomics.
4. What should an employee do after noticing a damaged power cable?
5. Why are near misses worth reporting?

Chapter 9

Waste Management, Recycling and Resource Conservation

Workplaces use electricity, water, paper, equipment and other materials. Waste prevention and careful resource use reduce environmental impact and can also lower operating costs. Sustainability works best when it is part of routine purchasing, maintenance and daily decisions.

9.1 Identify and separate waste

Waste should be separated according to local rules and the organisation’s procedures. Common categories include recyclable paper and cardboard, metals, glass, suitable plastics, organic waste and residual waste. Batteries, electronic equipment, lamps and other special waste may need dedicated collection because they can contain hazardous materials.

Figure 9.1. Separate collection bins help people sort recyclable materials. Photograph: Si Griffiths, Wikimedia Commons, CC BY-SA 3.0.

9.2 The waste hierarchy

The preferred order is to prevent waste first, reduce unnecessary use, reuse suitable items, recycle correctly and dispose of residual waste safely. Recycling is valuable, but avoiding unnecessary consumption usually prevents more waste from being created in the first place.

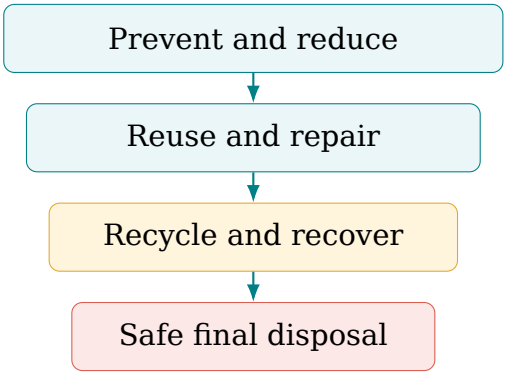


Figure 9.2. A simple waste hierarchy: prevention comes before recycling and disposal.

9.3 Electronic waste

Computers, cables, printers, batteries and other electronic items should not be placed in ordinary waste when local rules require special handling. Use authorised collection or recycling channels, follow data-wiping and asset-disposal policies, and ensure that storage devices containing sensitive information are handled securely.

9.4 Conserving energy and water

Switch off equipment when it is not needed and when policy permits; use power-saving settings; maintain equipment; avoid unnecessary printing; repair leaks; and use water responsibly. Purchasing decisions can consider durability, repairability, energy use and end-of-life disposal, not only the initial price.

9.5 Planning a small waste programme

Start by observing what the workplace throws away and where the waste is produced. Set a realistic target, provide clearly labelled collection points, explain the process, identify an approved collection route and review results periodically. Do not mix separated waste after collection, and never handle unknown or hazardous material without the required training.

Review Questions

1. Put the main waste-management options in preferred order.
2. Why does electronic waste need special attention?
3. List five ways to conserve resources in an office.
4. Outline a simple workplace recycling programme.

Chapter 10

Diversity, Equality and Inclusive Workplaces

An inclusive workplace makes it possible for people with different backgrounds, experiences and abilities to participate and contribute. Respect is shown through fair procedures, accessible communication, equitable opportunities and prompt action when discrimination or harassment occurs.

10.1 Diversity and equality

Diversity describes differences among people, including experience, culture, language, age, gender, disability and perspective. Equality means people are treated fairly and have equal rights and opportunities. Equity recognises that removing barriers may require different forms of support so people can participate meaningfully.

10.2 Everyday inclusive behaviour

Use respectful language, listen without interruption, avoid assumptions about a person's abilities or background, and give credit fairly. Meetings should make room for different voices. Work assignments, training, performance assessment and promotion should follow transparent and relevant criteria.

10.3 Discrimination and harassment

Discrimination involves unfair treatment linked to a protected or personal characteristic, according to applicable law and policy. Harassment can include unwelcome conduct that intimidates, humiliates or creates a hostile environment. Follow organisational reporting channels and relevant law. Bystanders should not escalate a situation; they can support the affected person and report concerns safely.

10.4 Gender equality and work-life responsibilities

Fair workplaces apply consistent standards, provide equal access to learning and advancement, and address barriers that prevent people from participating. Family responsibilities, caregiving and safety concerns should be handled through applicable policies and law rather than stereotypes about who should perform particular roles.

10.5 Building an inclusive team

Use clear selection criteria, accessible training material, fair performance feedback and a reliable process for raising concerns. Review workplace practices regularly. Inclusion is not achieved by a slogan; it requires people to notice barriers and improve the way work is organised.

Review Questions

1. Explain the difference between equality and equity.
2. Give four examples of inclusive behaviour.
3. Why should hiring criteria be transparent?
4. What should a worker do if they experience or witness harassment?

Chapter 11

Accessibility and Policies for Persons with Disabilities

Accessibility means that people can use facilities, information, services and digital tools as independently as possible. A barrier may be physical, communicative, procedural or digital. Inclusive practice identifies barriers and makes reasonable adjustments in line with applicable law and workplace policy.

Figure 11.1. An accessible workplace considers clear space and practical access around desks. Photograph: David290, Wikimedia Commons, CC BY-SA 4.0.

11.1 Recognising different access needs

People may have mobility, vision, hearing, learning, communication or other access requirements. Do not assume that everyone with the same disability needs the same support. Ask respectfully what adjustment is useful, protect privacy and involve the person in decisions that affect their work.

11.2 Digital accessibility

For websites and learning resources, use meaningful headings, descriptive link text, form labels, sufficient colour contrast, keyboard-accessible controls and alternative text for informative images. Do not communicate important information through colour alone. Captions and transcripts can help people who cannot hear audio; text alternatives can help people who cannot access a visual diagram.

11.3 Physical and workplace adjustments

Possible adjustments include accessible routes, suitable desk arrangements, assistive technology, flexible communication formats, adjusted work processes and accessible emergency information. The correct adjustment depends on the person, role, environment and applicable requirements. Keep routes clear and ensure emergency plans include people who may need assistance.

11.4 Respectful support and privacy

Speak directly to the person rather than only to a companion. Ask before offering physical assistance or moving mobility equipment. Do not disclose a person’s disabil-

ity or medical information without an appropriate basis. Apply workplace policies and relevant Indian law, including the Rights of Persons with Disabilities Act, 2016, where applicable; consult official guidance for current obligations.

Review Questions

1. What is an accessibility barrier?
2. Give five examples of digital accessibility practices.
3. Why should adjustments be discussed with the person concerned?
4. How can a workplace improve emergency access for people with disabilities?

Chapter 12

Integrated Case Study and Revision

12.1 Case study: a training-centre portal

A training centre wants a portal to maintain learner details, course enrolments and marks. Staff need to add new learners, update records and generate course summaries. The portal will use PHP to process requests and MySQL to store data. The workplace also needs a safe workstation, inclusive user experience and a responsible plan for retiring old computers.

12.2 Plan the database

Identify entities such as learners, courses and enrolments. Assign primary keys and use foreign keys to represent relationships. Avoid storing repeated course details in every learner row. Decide which fields are required, which values need constraints and which reports are necessary.

12.3 Plan the application

Separate presentation, application logic and database access. Validate input on the server, use prepared statements, encode output and verify permissions for every sensitive action. Do not rely on hidden buttons or client-side checks to protect data. Keep credentials outside public source files and use a dedicated database account.

12.4 Plan quality and workplace responsibility

Write test cases for successful and invalid submissions, access control, database errors and responsive display. Review accessibility with keyboard navigation and readable labels. Organise the work area safely, report hazards, use energy-saving settings and send retired electronic equipment through an approved process.

12.5 Revision questions

1. Explain the role of a DBMS in a web application.
2. Compare primary keys and foreign keys.
3. Write SQL to retrieve students with marks above 70, ordered from highest to lowest.
4. Explain DDL, DML, DCL and transaction-control commands.
5. What is the purpose of SQL aggregate functions?
6. Describe how a PHP request can retrieve data from MySQL.
7. Explain how prepared statements help prevent SQL injection.
8. Compare indexed and associative PHP arrays.
9. What makes a function reusable and easy to test?
10. Define hazard and risk, and explain the hierarchy of controls.
11. Describe a responsible way to manage electronic waste.
12. Give examples of equality, equity and digital accessibility.

12.6 Answer-planning guide

For definition questions, begin with a precise meaning and add one example. For comparison questions, explain the main difference and then compare practical uses. For code questions, state the purpose, show the syntax and describe any important safety condition. For workplace questions, identify the risk or barrier, propose a practical action and mention the relevant procedure or policy. These steps help produce clear, complete answers.

HANDBOOK

Artificial Intelligence Assistant

Grade IX – Vocational Education

AI Concepts • Scratch • Python • Data • Machine Learning

A textbook-style learning resource with worked examples, diagrams and review questions

Based on the supplied Grade IX study material

Contents

1 Introduction to Artificial Intelligence **1**
1.1 What is artificial intelligence? . . . 1
1.2 Core abilities associated with AI . . . 1
1.3 AI in everyday life . . . 1
1.4 Why learn AI at school? . . . 2
1.5 Types of Artificial Intelligence . . . 2
1.5.1 Types based on capability . . . 2
1.5.2 Types based on how a system works . . . 2
1.6 Responsible use of AI . . . 3
1.7 Chapter questions . . . 3

2 Programming Basics and Algorithms **4**
2.1 What is programming? . . . 4
2.2 Algorithms and logical steps . . . 4
2.3 Sequence, selection and repetition . . . 4
2.4 Variables, input and output . . . 4
2.5 Testing and debugging . . . 5
2.6 Chapter questions . . . 5

3 Coding with Scratch **6**
3.1 Introducing block-based programming . . . 6
3.2 The Scratch workspace . . . 6
3.3 Events and motion . . . 6
3.4 Control and sensing . . . 6
3.5 Planning a simple project . . . 6
3.6 Common errors . . . 7
3.7 Chapter questions . . . 7

4 Introduction to Python **8**
4.1 What is Python? . . . 8
4.2 A first program . . . 8
4.3 Comments and indentation . . . 8
4.4 Reading input and displaying output . . . 8
4.5 Errors in Python . . . 8
4.6 Chapter questions . . . 8

5 Data Types and Operators **10**
5.1 Why data types matter . . . 10
5.2 Arithmetic operators . . . 10
5.3 Comparison and logical operators . . . 10
5.4 Small worked example . . . 10
5.5 Chapter questions . . . 10

6 Basics of Data Literacy **11**
6.1 Understanding data . . . 11
6.2 Sources and types of data . . . 11
6.3 A basic data workflow . . . 11
6.4 Reading a dataset . . . 11
6.5 Privacy and fairness . . . 11
6.6 Chapter questions . . . 11

7 Data Pre-processing **12**
7.1 Why data need preparation . . . 12
7.2 Common data-quality issues . . . 12
7.3 A careful cleaning process . . . 12
7.4 Preparing data for machine learning . . . 12
7.5 Chapter questions . . . 12

8 Mathematics for AI: Data Analysis **13**
8.1 Why mathematics is useful . . . 13
8.2 Counts, totals and averages . . . 13
8.3 Percentages and ratios . . . 13
8.4 Correlation and caution . . . 13
8.5 Chapter questions . . . 13

9 Data Visualisation **14**
9.1 Choosing a chart . . . 14
9.2 Make a chart understandable . . . 14
9.3 Interpreting visual information . . . 14
9.4 Chapter questions . . . 14

10 Introduction to Machine Learning **15**
10.1 What is machine learning? . . . 15
10.2 A typical machine-learning workflow . . . 15
10.3 Features, labels and models . . . 15
10.4 Overfitting and evaluation . . . 15
10.5 Limits of machine learning . . . 15
10.6 Chapter questions . . . 15

11 Types of Machine Learning **16**
11.1 Supervised learning . . . 16
11.2 Unsupervised learning . . . 16
11.3 Reinforcement learning . . . 16
11.4 Deep learning . . . 16
11.5 Selecting an approach . . . 16
11.6 Chapter questions . . . 17

Chapter 1

Introduction to Artificial Intelligence

1.1 What is artificial intelligence?

Artificial Intelligence (AI) is a field of computing concerned with building systems that perform tasks commonly associated with human intelligence. These tasks may include recognising patterns, understanding language, interpreting images, making predictions and selecting actions. An AI system does not have to think exactly like a person; it is designed to perform a particular task using data, rules or learned patterns.

The word **artificial** means made by people, while **intelligence** includes abilities such as learning, reasoning, understanding information and solving problems. A spam filter, a voice assistant, a translation service and a system that recognises objects in photographs are everyday examples of AI-supported applications.

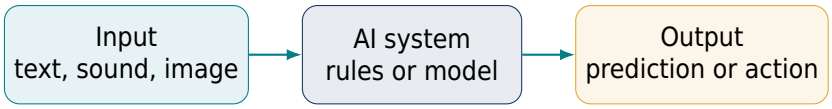


Figure 1.1. A simplified view of how an AI application receives input and produces an output.

1.2 Core abilities associated with AI

- **Learning:** using examples or experience to improve performance on a task.
- **Reasoning:** applying rules or relationships to reach a conclusion.
- **Problem solving:** considering possible steps and selecting a suitable solution.
- **Perception:** interpreting images, sound or other signals from the environment.
- **Language processing:** working with written or spoken human language.

Not every AI application uses all these abilities. A simple rule-based system may follow conditions written by a developer, while a machine-learning system may learn useful patterns from examples.

1.3 AI in everyday life

AI-supported features can appear in email spam detection, maps and route suggestions, speech recognition, translation, accessibility tools, shopping recommendations, fraud alerts and image search. In each case, the system uses input data and a designed method to produce a result. The result may be helpful, but it can still be incomplete or incorrect.

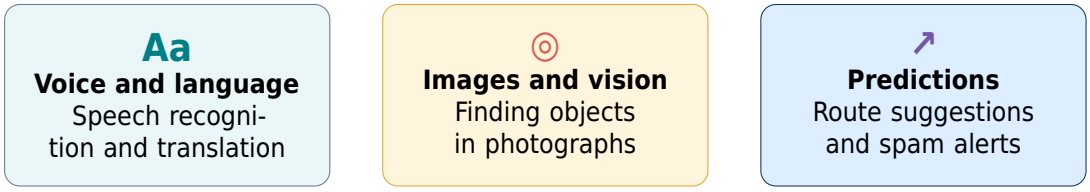


Figure 1.4. Examples of everyday tasks where AI-supported features may be used.

1.4 Why learn AI at school?

Learning AI helps students understand digital services they already use and develop the vocabulary needed to discuss them. It also builds habits of logical thinking, careful observation, data interpretation and responsible technology use. Students can begin with visual programming and gradually move towards text-based programming and data projects.

1.5 Types of Artificial Intelligence

AI can be grouped in more than one way. One common classroom explanation groups it by the range of tasks it could perform. Another describes how a system behaves or uses information. These are useful learning frameworks, not proof that every category already exists as a real product.

1.5.1 Types based on capability

- **Narrow AI (ANI):** designed for a specific task or a limited set of related tasks. Examples include speech recognition, spam filtering and route suggestions. Most AI applications used today fall into this category.
- **Artificial General Intelligence (AGI):** a proposed system that could learn and perform a broad range of intellectual tasks at a flexible, human-like level. It has not been established as a generally available system.
- **Artificial Superintelligence (ASI):** a hypothetical concept describing AI that would exceed human abilities across many important areas. It is an idea discussed in research and public debate, not an established current technology.

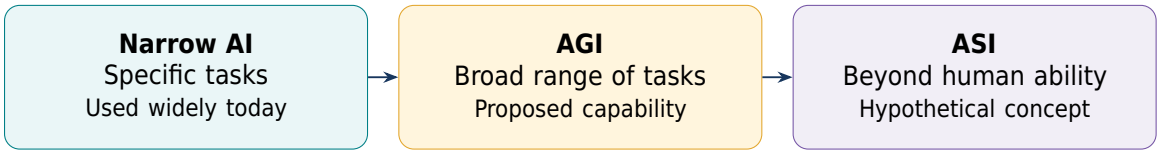


Figure 1.2. Capability categories, from task-specific AI to broader hypothetical capabilities. The arrows show a conceptual progression, not a guaranteed development path.

1.5.2 Types based on how a system works

- **Reactive machines:** respond to the current input using a defined method, without storing experiences to improve future decisions. They are a simple conceptual category.
- **Limited-memory AI:** uses recent data or information from past observations to support a task. Many modern AI applications use some form of stored context or historical data, although the details differ.
- **Theory-of-mind AI:** a proposed type that would interpret other people's beliefs, intentions or emotions in a deeper, context-aware way. Current systems may recognise cues but can misunderstand what people actually think or feel.
- **Self-aware AI:** a hypothetical system with genuine self-awareness. There is no established evidence that today's AI systems are self-aware.

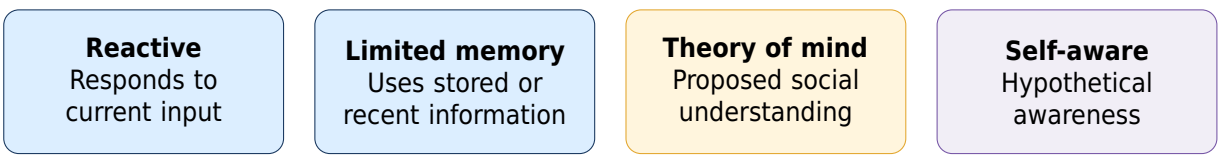


Figure 1.3. A simplified comparison of functional categories often used to introduce AI. The last two categories remain theoretical.

1.6 Responsible use of AI

AI outputs should be checked, particularly when a decision affects a person's safety, education, finances or opportunities. Avoid entering private information into tools unless their data practices are understood and authorised. Consider fairness, privacy, transparency, accessibility and the possibility that a model may make mistakes. Human judgement remains important.

1.7 Chapter questions

1. Define artificial intelligence in your own words.
2. Name four abilities associated with AI and give one example for each.
3. Explain two uses of AI in everyday life.
4. Name the three capability-based types of AI and give one-line descriptions.
5. Which functional categories of AI are considered hypothetical, and why?
6. What is the difference between narrow AI and the idea of AGI?
7. Why should AI-generated results be checked before they are used?

Chapter 2

Programming Basics and Algorithms

2.1 What is programming?

Programming is the process of writing instructions that a computer can execute. A program translates a solution into steps that are precise enough for a computer to follow. Programming is used to create websites, games, mobile applications, data tools and AI systems.

2.2 Algorithms and logical steps

An **algorithm** is a finite, ordered set of steps for solving a problem. Before writing code, it is useful to state the problem, identify the required input, decide the processing steps and describe the expected output. A clear algorithm is easier to test and improve than a collection of unplanned instructions.

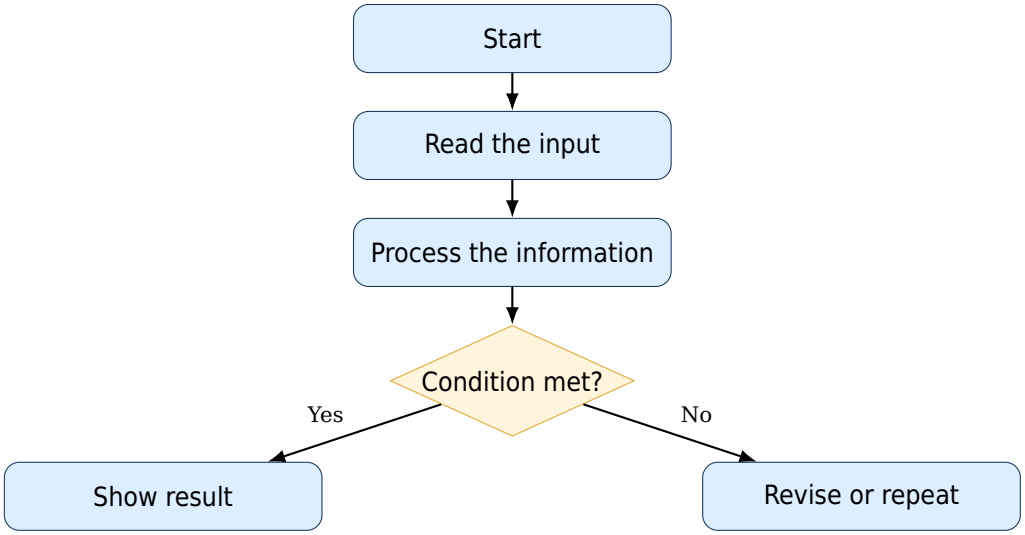


Figure 2.1. A flowchart shows the order of steps and decisions in an algorithm.

2.3 Sequence, selection and repetition

Sequence means instructions run in order. **Selection** chooses between alternatives using a condition, such as whether a value is greater than zero. **Iteration** repeats a set of instructions until a condition is met or a fixed number of repetitions has completed. These three structures are common in many programming languages.

2.4 Variables, input and output

A variable is a named place for a value used by a program. Input is information supplied to the program; output is the result presented to the user or another system. For example, a program may read two numbers, add them and display the sum. Meaningful variable names and consistent indentation make code easier to read.

2.5 Testing and debugging

Testing checks whether a program behaves as expected. Try ordinary values, boundary values and unexpected inputs. Debugging is the process of locating and correcting the cause of a problem. A useful approach is to reproduce the error, inspect the relevant steps, change one thing at a time and test again.

2.6 Chapter questions

1. What is an algorithm? Why should it be planned before coding?
2. Explain sequence, selection and iteration with examples.
3. What is the purpose of a variable?
4. Distinguish testing from debugging.
5. Draw a flowchart for deciding whether a number is positive or not positive.

Chapter 3

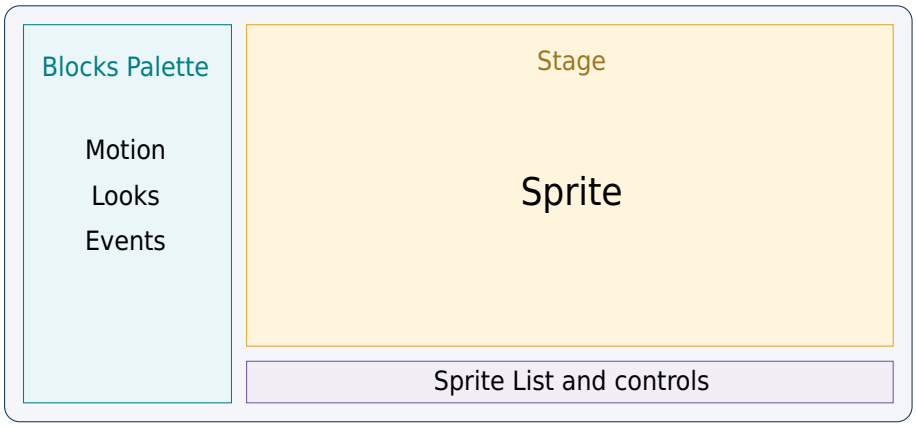
Coding with Scratch

3.1 Introducing block-based programming

Scratch is a visual programming environment in which programs are assembled from connected blocks. It helps learners explore computational thinking without needing to memorise punctuation and syntax at the beginning. A project can include sprites, backdrops, sounds, events, motion, looks, sensing and control blocks.

3.2 The Scratch workspace

The Stage displays the project while sprites are the characters or objects that perform actions. The Blocks Palette contains programming blocks, the Code Area is where blocks are connected, and the Sprite List helps select and manage project objects. A backdrop changes the Stage background.



Illustrative layout of a block-based programming workspace

3.3 Events and motion

An event block starts a script when something happens, such as clicking the green flag or pressing a key. Motion blocks move a sprite, turn it or change its position. Looks blocks change appearance, costumes or speech bubbles. Combining event and motion blocks can create a simple interactive animation.

3.4 Control and sensing

Control blocks repeat actions, wait for a period or run instructions only when a condition is true. Sensing blocks detect events or information, such as whether a sprite touches another object. Operators perform comparisons and calculations. Variables can store a score, a timer or the number of attempts.

3.5 Planning a simple project

For a simple catching game, define the goal first: the player moves a sprite to catch a falling object. Decide which keys control movement, how a catch is detected, how the score changes and when the game ends. Build one feature at a time and test it before adding sound or extra levels. Keep the rules clear so another learner can understand the game.

Figure 3.1. A simplified Scratch script: an event starts the action, then motion instructions run. Real Scratch blocks have their own shapes and colours.

3.6 Common errors

Disconnected blocks do not run as part of the intended script. A script may begin with the wrong event, repeat forever or check a condition before the sprite reaches the target. Test with slow movement first, then adjust speed and timing. Give sprites and variables meaningful names.

3.7 Chapter questions

1. Name four main areas of the Scratch workspace.
2. What is the role of an event block?
3. How are control blocks different from motion blocks?
4. Describe the steps needed to plan a simple catching game.
5. Give two ways to find errors in a Scratch project.

Chapter 4

Introduction to Python

4.1 What is Python?

Python is a high-level programming language known for readable syntax. It is used for automation, web development, data analysis, scientific work and many AI projects. Python programs are written as text and executed by a Python interpreter. The exact interface depends on the installed environment.

4.2 A first program

The statement below displays a message:

```
print("Hello, AI!")
```

The function `print()` sends information to the output. Text placed inside quotation marks is a string. Parentheses group the value passed to the function.

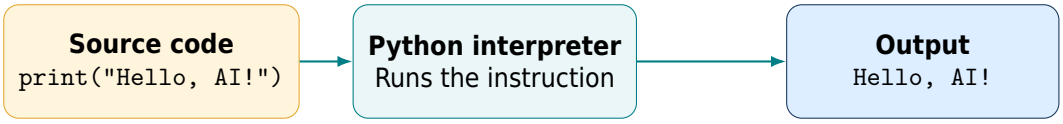


Figure 4.1. A short Python statement is interpreted and produces visible output.

4.3 Comments and indentation

A comment begins with the # symbol and is ignored when the program runs. Comments explain why a step is needed. Indentation marks the body of many Python structures, such as conditional statements and loops. Inconsistent indentation can cause an error, so use a consistent style.

4.4 Reading input and displaying output

```
name = input("Enter your name: ")
print("Welcome", name)
```

The `input()` function reads text from the user. A value read with `input()` is a string by default. If a program needs a number, it must convert the text carefully, for example with `int()` for a whole number.

4.5 Errors in Python

A syntax error means the code does not follow the language rules. A runtime error occurs while the program is executing. A logic error means the program runs but produces an incorrect result. Read the error message, check the line it identifies and test a small example to isolate the problem.

4.6 Chapter questions

1. What is Python used for?

2. Write a Python statement that prints your school name.
3. What is a comment, and how is it written in Python?
4. Why is indentation important?
5. Distinguish syntax, runtime and logic errors.

Chapter 5

Data Types and Operators

5.1 Why data types matter

A data type describes the kind of value a variable holds. Choosing a suitable type helps a program perform the intended operation and prevents avoidable errors. Common introductory Python types include integers, floating-point numbers, strings and Boolean values.

Type	Meaning	Example
`int`	Whole number	`students = 32`
`float`	Number with a decimal part	`temperature = 28.5`
`str`	Text	`city = "Bhopal"`
`bool`	True or false value	`is_ready = True`

5.2 Arithmetic operators

Python supports addition (+), subtraction (−), multiplication (*), division (/), floor division (//), remainder (%) and exponentiation (**). Use parentheses to make the intended order of operations clear. Division by zero is invalid and should be handled when user input is involved.

5.3 Comparison and logical operators

Comparison operators such as ==, !=, >, <, >= and <= produce Boolean results. Logical operators `and`, `or` and `not` combine or reverse conditions. They are often used in decisions and loops.

5.4 Small worked example

```
marks = 76
if marks >= 35:
print("Pass")
else:
print("Try again")
```

The condition is evaluated as true or false. The program prints one message depending on the result. In a complete application, the value should be read and validated rather than fixed in the source code.

5.5 Chapter questions

1. List four common Python data types.
2. Explain the difference between = and ==.
3. What result does the remainder operator produce?
4. Give an example of a logical condition using `and`.
5. Write a short program that checks whether a number is greater than zero.

Chapter 6

Basics of Data Literacy

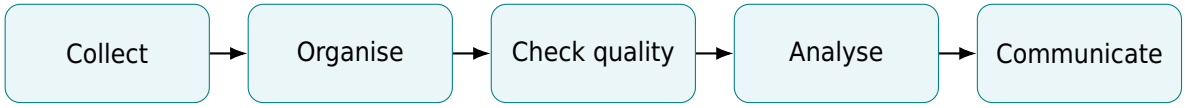
6.1 Understanding data

Data are recorded observations, values or descriptions. They may be numerical, textual, categorical, visual or audio-based. Information is produced when data are organised and interpreted in a context. Data literacy is the ability to read, question, analyse and communicate information from data responsibly.

6.2 Sources and types of data

Data may come from surveys, forms, sensors, observations, transactions or existing datasets. Primary data are collected directly for a purpose; secondary data were collected earlier by another person or organisation. Quantitative data describe counts or measurements. Qualitative data describe qualities or categories. Before using a dataset, identify its source, purpose, units, time period and any limitations.

6.3 A basic data workflow



6.4 Reading a dataset

A dataset arranged in rows and columns often uses each row for one record and each column for one variable. A column heading should clearly describe the values below it. Check whether units are consistent, whether values are missing and whether the same record appears more than once. A surprising value is a reason to investigate, not automatically a reason to delete it.

6.5 Privacy and fairness

Collect only data needed for a legitimate purpose. Do not publish personal details without appropriate permission. A dataset may under-represent a group or reflect past unfair decisions. Ask who is included, who is missing and how conclusions may affect different people. Good data practice includes secure storage, careful access and honest communication of uncertainty.

6.6 Chapter questions

1. Define data literacy.
2. Distinguish primary and secondary data.
3. Explain quantitative and qualitative data with examples.
4. Name four checks that can improve data quality.
5. Why should a dataset’s source and limitations be documented?

Chapter 7

Data Pre-processing

7.1 Why data need preparation

Collected data are often inconsistent or incomplete. Data pre-processing is the set of steps used to inspect and prepare data before analysis or machine learning. Poor-quality input can lead to misleading summaries and unreliable model predictions. Preparation should be appropriate to the question being studied.

7.2 Common data-quality issues

Missing values occur when a field has not been recorded. Duplicates represent repeated records. Inconsistent formats may use different spellings, date patterns or units. Invalid values fall outside the rules for a field. Outliers are unusually high or low values; they may be genuine observations or errors and require investigation.

7.3 A careful cleaning process

1. Keep an unchanged copy of the original dataset.
2. Inspect column names, data types, units and sample records.
3. Identify missing values, duplicates and inconsistent formats.
4. Decide how each issue should be handled and record the reason.
5. Validate the cleaned data and compare it with the original.

Do not automatically replace every missing value with zero: zero may have a real meaning. Depending on the task, a missing value may be left unknown, filled using a justified method or excluded with a clear explanation.

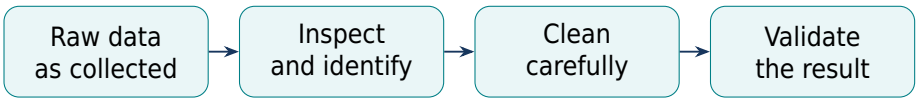


Figure 7.1. A simple data-cleaning workflow. Keep the original data and record important changes.

7.4 Preparing data for machine learning

For supervised learning, input features and the target label must be identified. Categorical data may need encoding and numerical features may need scaling for some algorithms. Split data into training and test sets before using test results to make decisions. Any transformation that learns from data should be fitted using the training data only, to reduce information leakage.

7.5 Chapter questions

1. What is data pre-processing?
2. Give three examples of data-quality problems.
3. Why should the original dataset be preserved?
4. Why is replacing every missing value with zero unsafe?
5. Explain the purpose of a training and test split.

Chapter 8

Mathematics for AI: Data Analysis

8.1 Why mathematics is useful

Mathematics helps describe data, compare values and reason about patterns. Basic arithmetic, fractions, percentages, averages, ratios and simple graphs are useful when interpreting data. Mathematical summaries do not replace context: the same number can mean different things depending on how data were collected.

8.2 Counts, totals and averages

The total is the sum of values. The mean is the total divided by the number of values. The median is the middle value after sorting, and the mode is the most frequent value. The mean can be affected by extreme values, while the median is often more resistant to them.

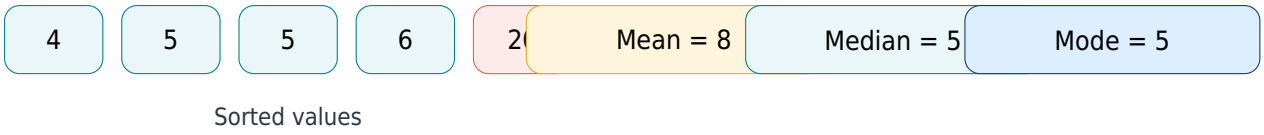


Figure 8.2. A compact visual summary of one dataset. The unusually large value 20 pulls the mean above the median and mode.

8.3 Percentages and ratios

A percentage expresses a part out of one hundred. The percentage of a group meeting a condition is calculated as the number meeting the condition divided by the total number, multiplied by 100. A ratio compares quantities. Always state the denominator and the population being compared so that a percentage is not misinterpreted.

8.4 Correlation and caution

Two variables may change together, but this does not prove that one causes the other. A third factor may influence both, or the pattern may be coincidental. When interpreting data, distinguish observation from explanation and avoid claims that go beyond the evidence.

8.5 Chapter questions

1. Calculate the mean of 6, 8, 10 and 12.
2. Find the median of 3, 7, 9, 12 and 18.
3. How is a percentage calculated?
4. Why can the median be more useful than the mean for data with extreme values?
5. Explain why correlation does not by itself prove causation.

Chapter 9

Data Visualisation

9.1 Choosing a chart

Data visualisation presents information in a visual form so that patterns and comparisons can be noticed. A bar chart compares categories; a line chart shows change over an ordered sequence such as time; a pie chart may show parts of a whole when there are only a few categories; and a scatter plot shows the relationship between two numerical variables.

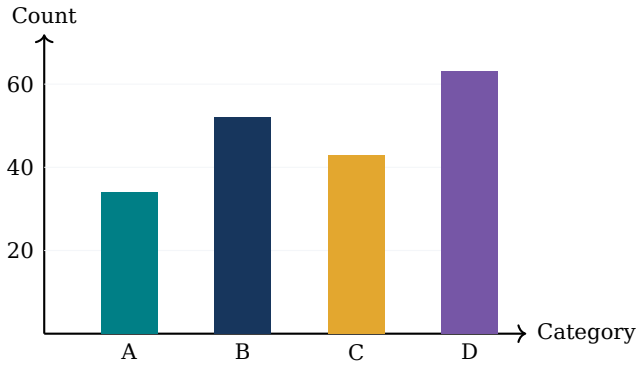


Figure 8.1. An illustrative bar chart. Values are examples, not measurements from the source material.

9.2 Make a chart understandable

Every chart should have a descriptive title, labelled axes where needed, clear units and a scale that does not mislead. Use a legend only when multiple series need identification. Avoid decorative effects that make comparisons difficult. If values are estimates or incomplete, explain that limitation.

9.3 Interpreting visual information

Read the title and labels first. Check the scale before comparing bar heights or slopes. Look for the largest and smallest values, changes over time and unusual points. Do not infer a cause from a chart alone. A chart should support a conclusion that can also be explained in words.

9.4 Chapter questions

1. Which chart is suitable for comparing categories?
2. When is a line chart useful?
3. Name four features of a clear chart.
4. How can a graph mislead a reader?
5. Why should the source and units of plotted data be stated?

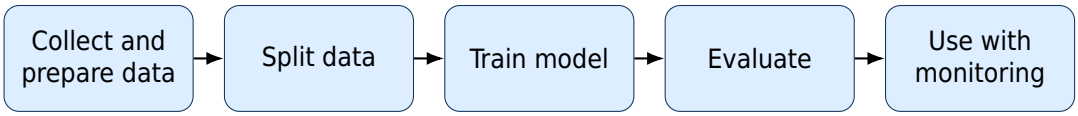
Chapter 10

Introduction to Machine Learning

10.1 What is machine learning?

Machine Learning (ML) is a part of AI in which a system uses data to learn patterns that help it make predictions or decisions. Instead of writing a separate rule for every possible case, a developer chooses a method, supplies suitable examples and evaluates the resulting model. ML performance depends on the data, the task and the way the model is tested.

10.2 A typical machine-learning workflow



10.3 Features, labels and models

A **feature** is an input used by a model, such as the size or colour of an object. A **label** is the answer associated with an example in a labelled dataset. A **model** is the learned pattern or relationship used to produce an output. Training adjusts the model based on examples; evaluation checks how well it works on data not used for training.

10.4 Overfitting and evaluation

A model may memorise details of training examples instead of learning patterns that generalise. This is called overfitting. Testing with separate data gives a more realistic indication of performance on new cases. Accuracy can be useful, but it may hide important errors when classes are imbalanced. Consider the consequences of false positives and false negatives for the task.

10.5 Limits of machine learning

ML systems can reflect gaps and bias in their training data. A model may perform well in one setting and poorly in another. Predictions are not guarantees. Systems used in real settings need monitoring, privacy safeguards, appropriate human oversight and a way to report problems.

10.6 Chapter questions

1. Define machine learning and explain its relationship to AI.
2. What are features, labels and models?
3. Why should evaluation data be kept separate from training data?
4. What is overfitting?
5. Give two limitations or risks of ML systems.

Chapter 11

Types of Machine Learning

11.1 Supervised learning

Supervised learning uses examples that include inputs and known target labels. A classification model predicts a category, such as whether an email is likely to be spam. A regression model predicts a numerical value, such as an estimated measurement. The quality of the labels and the relevance of the examples affect the result.

11.2 Unsupervised learning

Unsupervised learning works with data that do not have target labels supplied for each example. A method may group similar records or identify structure in the data. Clustering can help explore patterns, but the groups need interpretation and are not automatically meaningful categories.

11.3 Reinforcement learning

In reinforcement learning, an agent takes actions in an environment and receives feedback in the form of rewards or penalties. The agent learns a strategy that aims to improve long-term reward. This approach is used in some control, robotics and game-playing tasks. A reward design that does not match the real goal can lead to undesirable behaviour.

Learning type	Training information	Example task
Supervised	Inputs with known labels or target values	Classify messages as spam or not spam
Unsupervised	Inputs without target labels	Explore groups of similar records
Reinforcement	Actions, environment feedback and rewards	Learn a strategy in a simulated game

11.4 Deep learning

Deep learning uses neural networks with multiple processing layers to learn representations from data. It can be effective for images, audio, language and other complex inputs, but it often needs substantial data and computing resources. Deep learning is one family of methods within machine learning, not a synonym for all AI.

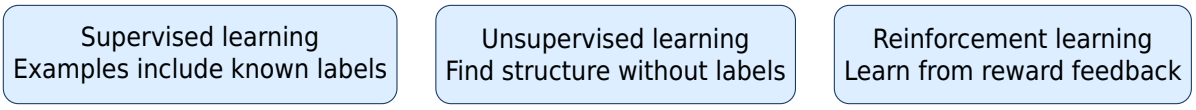


Figure 10.1. Three common learning approaches differ in the kind of feedback or information available.

11.5 Selecting an approach

Start with the problem and the available data. If examples have correct labels, supervised learning may be suitable. If the aim is to explore structure without labels,

an unsupervised method may help. If actions influence an environment and feedback is available, reinforcement learning may be relevant. The choice should be tested rather than assumed.

11.6 Chapter questions

1. Explain supervised learning with an example.
2. How does unsupervised learning differ from supervised learning?
3. What role does a reward play in reinforcement learning?
4. What is deep learning?
5. Match a suitable learning type to a spam filter, customer grouping and a game-playing agent.

HANDBOOK

Artificial Intelligence Assistant

Grade X – Vocational Education

Python • NumPy • Pandas • Neural Networks • AI Project

A textbook-style learning resource with examples, diagrams, code and review questions

Prepared with reference to the supplied Grade X study material

Contents

1 Python Programming **1**
1.1 Control structures in Python . . . 1
1.1.1 Selection with `if`, `if...else` and `if...elif...else` . . . 1
1.1.2 Repetition using loops . . . 1
1.1.3 Loop control and common errors . . . 1
1.2 Functions in Python . . . 2
1.2.1 Parameters, return values and scope . . . 3
1.2.2 Designing a useful function . . . 3
1.2.3 Tracing a program step by step . . . 3
1.2.4 Writing readable and maintainable code . . . 3
1.2.5 Testing with normal, boundary and invalid values . . . 4

2 Data Science with NumPy **5**
2.1 Introduction to NumPy . . . 5
2.1.1 Array properties . . . 5
2.2 Array manipulation . . . 5
2.2.1 Indexing and slicing . . . 6
2.2.2 Broadcasting and practical cautions . . . 6
2.2.3 Joining, splitting and copying . . . 6
2.3 Array computation . . . 6
2.3.1 Why vectorised operations are useful . . . 6
2.3.2 Choosing an array structure . . . 7
2.3.3 Missing values and unusual measurements . . . 7
2.3.4 Efficiency and memory . . . 7
2.3.5 Practical exercise: summarising measurements . . . 7

3 Data Analysis **9**
3.1 Introduction to Pandas . . . 9
3.2 Working with Series and DataFrames . . . 9
3.2.1 Selecting, sorting and grouping records . . . 9
3.2.2 Cleaning and summarising data . . . 10
3.3 Data visualisation using Matplotlib . . . 10
3.3.1 Choosing a chart for the question . . . 11
3.3.2 Understanding records and data types . . . 11
3.3.3 Filtering records carefully . . . 11
3.3.4 Grouping and summarising . . . 12
3.3.5 Reproducible data cleaning . . . 12

4 Neural Networks and Machine Learning Tools **13**
4.1 Artificial Neural Networks . . . 13
4.1.1 Training and prediction . . . 13
4.1.2 Weights, activation functions and loss . . . 13
4.1.3 Overfitting and generalisation . . . 13

4.2 Applications of neural networks 14
4.2.1 Responsible use 14
4.2.2 Privacy, fairness and human oversight 14
4.3 Machine learning tools 14
4.3.1 How a neural network makes a prediction 14
4.3.2 Training, validation and testing 15
4.3.3 Classification and regression 15
4.3.4 Limitations of neural networks 15
4.3.5 Practical comparison exercise 15

5 AI Project **16**
5.1 Project guidelines 16
5.1.1 A practical project cycle 16
5.2 Project formats and documentation 16
5.2.1 Choosing evaluation measures 16
5.3 Project review 16
5.4 Sample project: student marks analysis 17
5.4.1 Planning the work and presenting results 17
5.4.2 Defining a measurable project objective 17
5.4.3 Data permissions and documentation 18
5.4.4 A practical project schedule 18
5.4.5 Interpreting results honestly 18
5.4.6 Project report structure 18
5.4.7 Reflection after completion 18

Chapter 1

Python Programming

1.1 Control structures in Python

A Python program normally executes statements in the order in which they appear. Control structures let a program choose between alternatives or repeat a set of instructions. These structures are important when a solution must respond to different inputs rather than follow one fixed path.

1.1.1 Selection with `if`, `if...else` and `if...elif...else`

An `if` statement tests a condition. If the condition is true, its indented block runs. An `if...else` statement provides an alternative when the condition is false. An `if...elif...else` ladder checks several conditions in sequence; the first true branch is executed.

```
marks = int(input("Enter marks: "))
if marks >= 80:
    print("Grade A")
elif marks >= 60:
    print("Grade B")
elif marks >= 40:
    print("Grade C")
else:
    print("Needs improvement")
```

Indentation is part of Python syntax. Statements belonging to a branch must be indented consistently. Conditions can use comparison operators such as ==, !=, >, <, >= and <=, along with logical operators `and`, `or` and `not`.

1.1.2 Repetition using loops

A `for` loop iterates over items in a sequence or over values produced by `range()`. A `while` loop repeats while its condition remains true. Every loop should make progress toward termination; otherwise, a `while` loop can continue indefinitely.

```
total = 0
for number in range(1, 6):
    total += number
print("Total:", total)
```

The program adds the integers from 1 through 5. The `range(1, 6)` expression stops before 6, so the values are 1, 2, 3, 4 and 5. A nested loop is a loop placed inside another loop and is useful for tables, patterns and two-dimensional data.

1.1.3 Loop control and common errors

The `break` statement stops the nearest loop, while `continue` skips the remaining statements in the current iteration and moves to the next one. These statements should be used only when they make the program easier to follow. A common error is an off-by-one mistake, in which a loop repeats one time too many or too few. Carefully check the starting value, stopping condition and update step.

```
for number in range(1, 8):
    if number == 5:
```

```
        continue
    print(number)
```

This example prints 1, 2, 3, 4, 6 and 7. The value 5 is skipped, but the loop continues. Testing a loop with a small set of known values is a useful way to confirm that it behaves as intended.

When a program receives data from a user, it should also consider invalid input. For example, converting a word to an integer with `int()` raises an error. A more robust program can validate input or handle a conversion error instead of terminating unexpectedly.

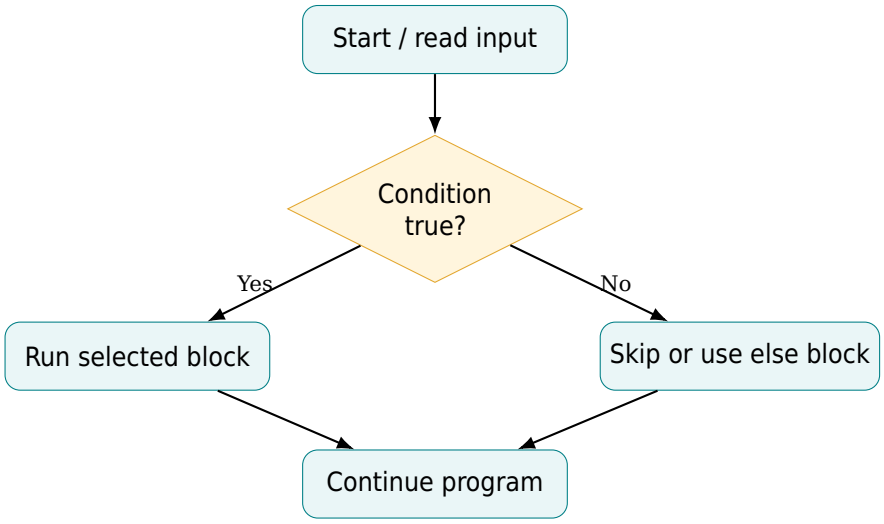


Figure 1.1. A simplified decision flow for a conditional statement.

1.2 Functions in Python

A function is a named, reusable block of code that performs a task. Functions help divide a larger problem into smaller parts, reduce repetition and make a program easier to test. Python includes built-in functions such as `print()`, `len()` and `sum()`; programmers can also define their own functions with `def`.

```
def mean(values):
    if len(values) == 0:
        return None
    return sum(values) / len(values)

marks = [72, 81, 67, 90]
print(mean(marks))
```

The function accepts a list as an argument and returns its arithmetic mean. The `return` statement sends a result back to the calling code. Parameters are names in a function definition; arguments are the values supplied when it is called. A local variable created inside a function is normally available only within that function.

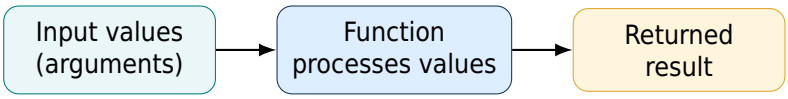


Figure 1.2. A function receives arguments, performs a task and may return a result.

1.2.1 Parameters, return values and scope

Functions may accept zero or more parameters. A function that returns a value can be used in an expression, while a function that only performs an action may return `None`. Variables defined inside a function have local scope. Clear names, short functions and appropriate validation improve readability and reliability.

1.2.2 Designing a useful function

A well-designed function usually has one clear responsibility. Its name should describe the task, its parameters should represent the information needed, and its return value should be predictable. Repeating the same calculation in several places makes later changes harder; placing that calculation in one function helps keep results consistent.

```
def percentage(score, maximum):
    if maximum <= 0:
        raise ValueError("Maximum must be positive")
    return (score / maximum) * 100

print(percentage(45, 50))
```

The function checks that the maximum possible score is positive before performing the calculation. This is an example of input validation. In larger programs, functions can be tested with normal values, boundary values and invalid values. A function should also be documented when its purpose or expected inputs are not obvious.

1.2.3 Tracing a program step by step

Tracing means following a program’s statements in the order in which they execute and recording how important variables change. It is useful when the output differs from what was expected. Start with the initial values, follow each condition, and update variables only when the relevant statement executes. A small trace table written in a notebook can help beginners reason about loops without relying on guesswork.

Consider a counter that begins at zero and increases once for each item processed. If the update statement is accidentally placed outside the loop, the counter may not represent the number of items. Similarly, changing the order of statements can change the result. Reading the code line by line and testing a small example makes such mistakes easier to identify.

1.2.4 Writing readable and maintainable code

Readable code uses meaningful names, consistent indentation and short blocks that have a clear purpose. A name such as `total_marks` communicates more than a single-letter name such as `x`. Comments should explain the reason for a decision or a non-obvious step rather than repeat every line in English. Blank lines can separate logical parts of a program, but excessive spacing makes a short program harder to follow.

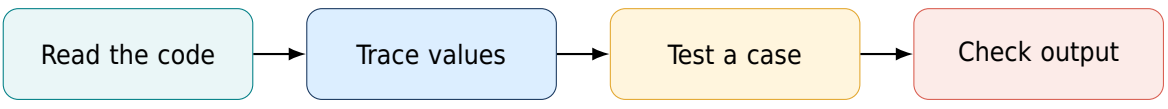


Figure 1.3. A simple method for tracing and checking a program.

1.2.5 Testing with normal, boundary and invalid values

A normal test checks an expected everyday input. A boundary test checks a value near a limit, such as marks equal to the minimum passing score. An invalid-input test checks what happens when the program receives a negative number where only positive values are permitted, or text where a number is expected. Testing all three kinds of input can reveal problems that one successful example would not show.

For example, a percentage function should be tested with a score of zero, a score equal to the maximum, and a maximum value of zero. The last case should be rejected because division by zero is undefined. Testing does not prove that a program has no defects, but it provides evidence that the program behaves as expected for the cases considered.

Review Questions

1. Distinguish between `if...else` and `if...elif...else`.
2. What is the difference between a `for` loop and a `while` loop?
3. Write a function that returns the larger of two numbers.
4. Explain why indentation matters in Python.

Chapter 2

Data Science with NumPy

2.1 Introduction to NumPy

NumPy is a Python library for numerical computing. Its central data structure is the multidimensional array, commonly called an `ndarray`. Compared with ordinary Python lists, NumPy arrays support convenient element-wise operations and efficient numerical processing. NumPy is widely used as a foundation for scientific computing and data analysis.

```
import numpy as np
marks = np.array([72, 81, 67, 90])
print(marks.mean())
print(marks.max())
print(marks.min())
```

The import statement makes NumPy available under the short name `np`, a common convention. The array stores four numerical values. Its methods calculate the mean, maximum and minimum without requiring a manually written loop.

2.1.1 Array properties

The `ndim` attribute reports the number of dimensions, `shape` reports the size along each dimension, `size` gives the total number of elements, and `dtype` indicates the element data type. A one-dimensional array can represent a sequence; a two-dimensional array can represent rows and columns.

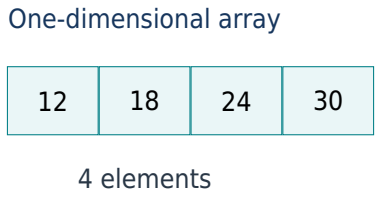


Figure 2.2. A one-dimensional NumPy array stores an ordered sequence of values.

```
table = np.array([[10, 20, 30],
                  [40, 50, 60]])
print(table.ndim)   # 2
print(table.shape)  # (2, 3)
print(table.size)   # 6
```

2.2 Array manipulation

Arrays can be reshaped, joined, split, indexed and sliced. A reshape operation changes the arrangement of elements without changing their total number. Indexing selects particular elements; slicing selects a range. The requested shape must be compatible with the number of values in the array.

```
values = np.array([1, 2, 3, 4, 5, 6])
matrix = values.reshape(2, 3)
print(matrix)
print(matrix[:, 1])
```

The expression `matrix[:, 1]` selects the second column. In a slice, a colon represents a range along an axis. Understanding array shape is essential when combining data from multiple sources.

2.2.1 Indexing and slicing

NumPy uses zero-based indexing, which means the first element has index 0. Negative indices count from the end of an array. Slicing selects a range using the form `start:stop:step`; the stop position is excluded. For a two-dimensional array, separate index expressions identify the row and column.

```
scores = np.array([55, 68, 72, 89, 94])
print(scores[0])        # first value
print(scores[-1])       # last value
print(scores[1:4])      # values at positions 1, 2, 3
print(scores[scores >= 70])
```

The last line uses a Boolean condition to select values that are at least 70. Such filtering is useful when exploring measurements, marks or other numerical records. The original data should be preserved when an analysis requires a record of the unmodified values.

2.2.2 Broadcasting and practical cautions

Broadcasting allows NumPy to combine arrays of compatible shapes. For example, adding a single number to an array adds that number to every element. More complex broadcasting follows shape rules, so dimensions should be checked when an operation produces an unexpected result. Arrays should also use appropriate numeric types: converting a decimal measurement to an integer can discard its fractional part.

2.2.3 Joining, splitting and copying

Functions such as `np.concatenate()` join arrays along a compatible axis. Splitting functions divide arrays into smaller arrays. A view can share underlying data with another array, whereas `copy()` creates independent data. Changing a shared view may therefore affect the original array.

2.3 Array computation

NumPy supports arithmetic, comparison, logical, statistical and mathematical operations. Many operations are performed element by element. Broadcasting allows arrays with compatible shapes to participate in an operation without manually copying values.

```
a = np.array([1, 2, 3])
print(a * 2)        # [2 4 6]
print(a + 10)       # [11 12 13]
print(np.sqrt(a))   # square roots
```

Common statistical functions include `sum()`, `mean()`, `median()`, `min()` and `max()`. Mathematical utilities include trigonometric functions and conversions between degrees and radians. The result type depends on the operation and input types.

2.3.1 Why vectorised operations are useful

A vectorised operation expresses a calculation for an entire array instead of describing each element with a Python loop. This often makes numerical code shorter and can improve performance. The expression should still be chosen for clarity: a com-

pact calculation is not helpful if its meaning is difficult to understand.

For example, if a set of temperatures is stored in an array in degrees Celsius, the conversion to Fahrenheit can be applied to every value using `temperatures * 9 / 5 + 32`. The formula is applied element by element. Before interpreting a result, check the units, the range of values and whether any missing or unusual measurements need attention.

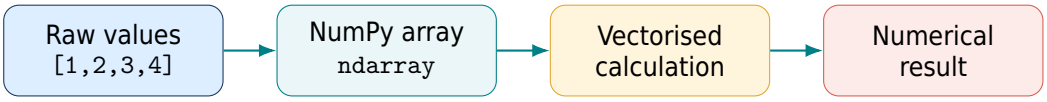


Figure 2.1. A typical numerical data-processing path.

2.3.2 Choosing an array structure

A one-dimensional array is suitable for a single sequence of measurements, such as daily temperatures. A two-dimensional array can represent a grid or a table of numerical values. Higher-dimensional arrays can represent batches of images or measurements collected across several dimensions. The structure should reflect the meaning of the data, not merely the shape that is convenient to type.

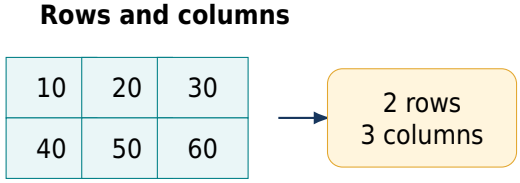


Figure 2.3. A two-dimensional array can represent numerical data arranged in rows and columns.

2.3.3 Missing values and unusual measurements

A numerical array can contain values that are mathematically valid but incorrect for the real problem. For instance, a temperature of 900 degrees may be a valid number in a computer but an implausible reading for a classroom thermometer. A negative mark may be outside the permitted range for an examination. Range checks and knowledge of the measurement context help identify such cases.

Missing values require a deliberate decision. If a sensor did not record a reading, replacing it with zero may falsely imply that the measured quantity was zero. Depending on the purpose, the missing value may be excluded from a calculation, investigated, or replaced using a justified method. Always record the choice and consider how it could affect the conclusion.

2.3.4 Efficiency and memory

NumPy is designed for numerical work with arrays and often performs large element-wise calculations more efficiently than manually looping through ordinary Python lists. The actual performance depends on the size of the data, the operation and the computer. For a tiny list, readability matters more than trying to optimise every operation. For large datasets, choosing an appropriate data type and avoiding unnecessary copies can reduce memory use.

2.3.5 Practical exercise: summarising measurements

Create an array containing five fictional daily temperatures. Print the minimum, maximum and mean. Then select values above the mean and explain what the filtered result represents. Change one value to an implausible number and discuss

how a range check could identify it. The exercise should include the code, its output and a short interpretation in words.

Review Questions

1. What do `shape`, `ndim` and `dtype` describe?
2. Explain indexing and slicing.
3. What is the difference between a view and a copy?
4. State two advantages of NumPy arrays.

Chapter 3

Data Analysis

3.1 Introduction to Pandas

Pandas is an open-source Python library used to organise, clean, analyse and summarise data. It is built around two important data structures: `Series`, a labelled one-dimensional structure, and `DataFrame`, a two-dimensional structure with rows and columns. Pandas is useful when data is arranged as a table, such as attendance records, marks, product lists or survey responses.

```
import pandas as pd
students = pd.DataFrame({
    "Name": ["Asha", "Ravi", "Meena"],
    "Marks": [78, 85, 91],
    "Class": ["X", "X", "X"]
})
print(students)
print(students["Marks"].mean())
```

A DataFrame can contain columns of different data types. Column labels help identify fields, while the row index identifies records. Data should be checked for missing values, duplicate records and inconsistent formats before analysis.

Name	Marks	Class
Asha	78	X
Ravi	85	X

Figure 3.2. A simplified DataFrame with named columns and individual records.

3.2 Working with Series and DataFrames

A Series stores values with associated labels. A DataFrame groups multiple columns into a table. Pandas can create a Series from a list or dictionary, select columns, filter rows, sort values and calculate summaries.

```
marks = pd.Series([72, 88, 64, 95])
print(marks.mean())
print(marks[marks >= 80])
```

Boolean filtering produces the values that satisfy a condition. In a DataFrame, a similar condition can be used to select records. Labels and positions should not be confused: label-based selection and integer-position selection have different purposes.

3.2.1 Selecting, sorting and grouping records

Pandas makes it possible to select a subset of rows using a condition, sort records by a column and calculate summaries for groups. These operations help turn a large table into information that answers a particular question. A filter should be linked to a clear question, such as identifying records above a chosen threshold.

```
high_scores = students[students["Marks"] >= 80]
sorted_students = students.sort_values("Marks", ascending=False)
print(high_scores)
print(sorted_students)
```

The first expression selects students whose marks are at least 80; the second orders the records from highest marks to lowest. In real projects, the meaning of each column and the unit of measurement should be recorded so that the output is interpreted correctly.

3.2.2 Cleaning and summarising data

Data cleaning may include handling missing values, correcting data types, removing duplicates and standardising text. The correct treatment depends on what a missing or unusual value means in the real context. Deleting records without review can distort the analysis.

A missing value can mean that information was not collected, was not applicable or was accidentally omitted. These situations do not always have the same meaning. Depending on the task, a missing value may be left as missing, investigated with the data owner, or handled using a documented method. Replacing all missing values with zero is not always appropriate because zero may be a valid measurement.

Duplicate records also need careful review. Some duplicates are accidental copies, while others represent legitimate repeated events. Before removing them, check which fields define a unique record. Cleaning decisions should be documented so that another person can understand how the original data was transformed.

```
print(students.head())
print(students.info())
print(students.isna().sum())
print(students.describe())
```

The `head()` method previews records, `info()` summarises columns and data types, `isna().sum()` counts missing values by column, and `describe()` gives descriptive statistics for suitable columns.

3.3 Data visualisation using Matplotlib

Data visualisation presents information graphically so that patterns, comparisons and changes can be understood. Matplotlib is a Python library for creating charts. The chart type should match the question: bar charts compare categories, line charts show change over an ordered sequence, and scatter plots explore relationships between numerical variables.

```
import matplotlib.pyplot as plt
subjects = ["English", "Maths", "Science"]
marks = [76, 88, 81]
plt.bar(subjects, marks)
plt.title("Marks by Subject")
plt.xlabel("Subject")
plt.ylabel("Marks")
plt.tight_layout()
plt.show()
```

A clear chart needs a meaningful title, labelled axes and a scale that does not mislead. Colours and decorative effects should support interpretation rather than distract from the data. Charts should be checked against the source values before

conclusions are drawn.

3.3.1 Choosing a chart for the question

A bar chart is suitable for comparing a small number of categories. A line chart is often appropriate for a measurement recorded over time, such as monthly rainfall. A scatter plot can help reveal whether two numerical variables appear to be related. A histogram shows the distribution of a numerical variable by grouping values into intervals. No chart proves a cause-and-effect relationship by itself.

A chart should include a title that states what is being shown, labels that explain the axes, and units where relevant. The scale should not exaggerate small differences. When comparing groups, use consistent scales and explain any filtering or exclusions. A well-labelled chart helps readers understand both the result and its limits.

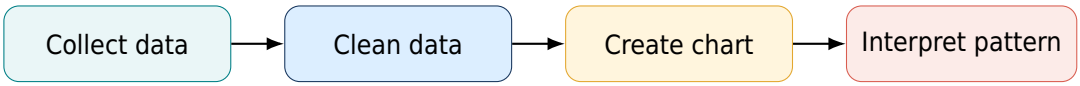


Figure 3.3. A simple sequence for producing a trustworthy data visualisation.

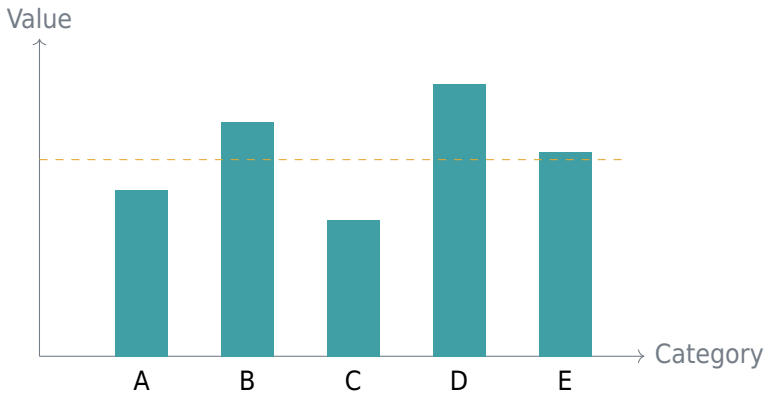


Figure 3.1. A schematic bar chart showing how categories can be compared.

3.3.2 Understanding records and data types

A data table represents observations as rows and properties as columns. In a student marks dataset, one row may represent one learner and columns may represent a name, subject, marks and examination date. A column should have a consistent meaning. If marks are stored partly as numbers and partly as text, calculations and comparisons may behave unexpectedly.

A Pandas Series is a one-dimensional labelled structure. A DataFrame combines rows and columns and can contain columns with different data types. The index labels rows, while column names identify fields. An index is not necessarily a unique identity for a real-world person or item; if a reliable identifier is required, it should be designed and checked separately.

3.3.3 Filtering records carefully

Filtering selects records that satisfy a condition. Multiple conditions can be combined, but each condition should be enclosed in parentheses when using Pandas Boolean expressions. The symbols `&` and `|` represent element-wise AND and OR; Python's `and` and `or` are not substitutes for these operators in a Series expression.

```python
import pandas as pd

students = pd.DataFrame({
    "Name": ["Asha", "Ravi", "Meena", "Imran"],
    "Marks": [78, 91, 84, 65],
    "Present": [True, True, False, True]
})
high = students[students["Marks"] >= 80]
print(high[["Name", "Marks"]])
```

The expression creates a smaller DataFrame containing only records with marks of at least 80, and then displays selected columns. In a real report, the filter condition should be stated clearly so readers understand which records were included.

3.3.4 Grouping and summarising

Grouping is useful when the question asks for a summary by category, such as average marks by subject or the number of records in each region. A group summary can hide differences between individual observations, so it should be interpreted at the correct level. When group sizes differ, a total and an average answer different questions; choose the statistic that fits the purpose.

3.3.5 Reproducible data cleaning

A reproducible cleaning process records what was changed and why. Keep an untouched copy of the original data where appropriate, and perform cleaning steps in a clear order. Check the number of rows before and after removing duplicates, confirm that conversions succeeded, and inspect a sample of the cleaned result. These checks reduce the risk of silently removing valid records or introducing errors.

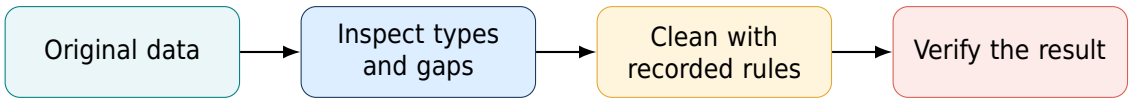


Figure 3.4. A repeatable workflow for preparing tabular data.

Review Questions

1. Differentiate between a Series and a DataFrame.
2. Why is data cleaning necessary?
3. Name four useful Pandas methods.
4. Choose an appropriate chart to compare marks in several subjects and explain why.

Chapter 4

Neural Networks and Machine Learning Tools

4.1 Artificial Neural Networks

An Artificial Neural Network (ANN) is a computational model inspired in a broad way by networks of connected neurons. In a typical neural network, units receive numerical inputs, apply weights and a bias, pass the result through an activation function, and send outputs to the next layer. Networks can learn patterns by adjusting parameters during training.

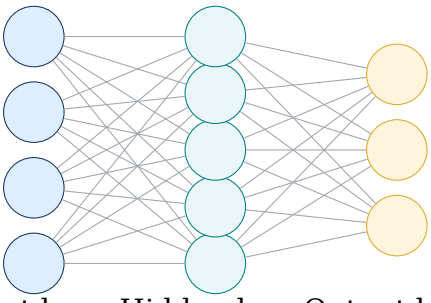


Figure 4.1. A simplified fully connected neural network. The drawing is conceptual; real networks can contain many layers and different connection patterns.

4.1.1 Training and prediction

During training, a network processes examples and compares its output with the expected target. A loss function measures the difference, and an optimisation method adjusts model parameters to reduce that loss. After training, the model can make predictions on new inputs. Performance on training examples alone is not enough; evaluation on separate data helps estimate how well the model generalises.

4.1.2 Weights, activation functions and loss

A weight controls how strongly an input contributes to a unit’s calculation. A bias shifts the calculation, and an activation function transforms the result. Common activation functions include ReLU for many hidden layers and sigmoid for some binary-output tasks. The choice depends on the model design and the problem being solved.

A loss function measures how far a model’s predictions are from the expected targets during training. The training process adjusts weights and biases to reduce the loss. A lower training loss does not automatically mean that a model will perform well on unfamiliar examples, so evaluation must use data that was not used to fit the model.

4.1.3 Overfitting and generalisation

Overfitting occurs when a model learns details or noise specific to its training examples and performs poorly on new data. A model that is too simple may also fail to capture useful patterns; this is often described as underfitting. Careful data splitting, suitable model complexity and evaluation on representative examples help assess generalisation. The test set should not be repeatedly used to tune every decision, because that can make the final estimate overly optimistic.

4.2 Applications of neural networks

Neural networks are used in image and speech recognition, handwriting recognition, language processing, recommendation systems, forecasting and selected healthcare applications. Face recognition can support device authentication; speech-to-text systems convert spoken language into text; recommendation systems rank items that may interest a user. Such applications can make errors, so human review and suitable safeguards remain important, especially in high-impact settings.

4.2.1 Responsible use

Data quality, privacy, fairness and transparency affect the reliability of an AI application. A model trained on unrepresentative examples may perform poorly for some groups. Sensitive personal data should be handled carefully, and a prediction should not automatically be treated as a fact. The system's purpose, limitations and evaluation results should be documented.

4.2.2 Privacy, fairness and human oversight

Responsible AI development begins with asking whether the data is needed and whether it may be used for the intended purpose. Personal details should not be collected simply because they are available. Access should be limited, files should be protected, and information should not be shared without appropriate permission.

Fairness requires checking whether the examples represent the people and situations in which the system will be used. An error can affect groups differently, especially when the training data is incomplete or unbalanced. A responsible project explains known limitations and provides a way for a person to review important decisions. AI outputs should be treated as information to assess, not as unquestionable truth.

4.3 Machine learning tools

Python's data-science ecosystem includes tools for different stages of an AI workflow. NumPy supports numerical arrays; Pandas supports tabular data handling; Matplotlib supports visualisation; Scikit-learn provides common machine-learning algorithms and evaluation utilities; TensorFlow and Keras support the development of neural-network models; XGBoost provides optimised gradient-boosting methods.

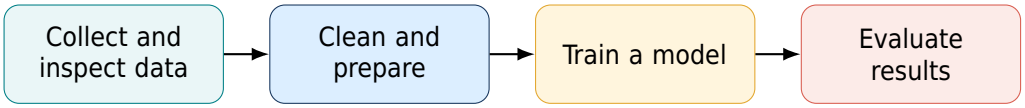


Figure 4.2. A simplified machine-learning workflow.

4.3.1 How a neural network makes a prediction

A neural network transforms input values through a series of connected calculations. In a simple unit, inputs are multiplied by weights, combined with a bias and passed through an activation function. The output of one layer becomes the input to the next layer. During training, the model adjusts its parameters using examples and a learning procedure.

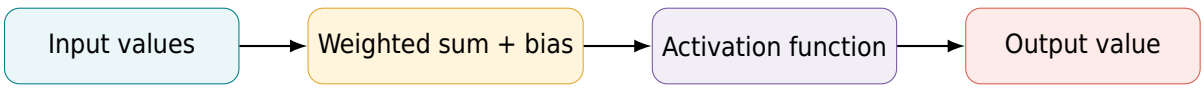


Figure 4.3. A simplified view of the calculation performed by a neural-network unit.

4.3.2 Training, validation and testing

A dataset is often divided into separate parts for training, validation and testing. Training examples are used to fit model parameters. Validation examples help compare model settings during development. Test examples are held aside for a final evaluation after important choices have been made. The exact split depends on the amount and structure of the available data, and time-based data may require a time-aware split.

If examples from the same person or event appear in both training and test sets, the test result may be overly optimistic. This is a form of data leakage. Data preparation steps such as scaling should also be fitted using training data only and then applied consistently to validation and test data. A fair evaluation depends on keeping information from the test set out of the training process.

4.3.3 Classification and regression

Classification predicts a category, such as whether an image contains a cat or a dog. Regression predicts a numerical value, such as an estimated temperature. Accuracy can be useful for some classification tasks, but it may be misleading when one category is much more common than another. Precision and recall can provide additional information when false positives and false negatives have different consequences.

For regression, measures such as mean absolute error describe the typical size of prediction errors in the units of the target. The chosen measure should be explained in plain language. A model should also be compared with a simple baseline, such as always predicting the average, to check whether the more complex method adds value.

4.3.4 Limitations of neural networks

Neural networks may require substantial data and computing resources, and their behaviour can be difficult to explain. They can reproduce patterns and biases in training data. A high score on one dataset does not guarantee the same performance in a different school, region, language or time period. Monitoring, human review and clear limits on use are important where mistakes could affect people.

4.3.5 Practical comparison exercise

Imagine a system that predicts whether a learner may need additional help based on practice quiz results. Discuss what the inputs might be, which information should not be collected, and how the system could be checked for errors. Explain why a teacher should review the context before acting on a prediction. The goal is to evaluate responsible design, not to label any real learner.

Review Questions

1. Name the main layers in a simple ANN.
2. What is the purpose of a loss function?
3. Give three applications of neural networks.
4. Match NumPy, Pandas, Matplotlib and Scikit-learn with their common uses.
5. Why should a model be evaluated on data not used for training?

Chapter 5

AI Project

5.1 Project guidelines

An AI project begins with a clear problem rather than with a tool. The problem statement should explain who experiences the problem, why it matters and what a useful result would look like. A manageable project has a defined scope, accessible data, a realistic schedule and criteria for evaluating the result.

5.1.1 A practical project cycle

A typical project cycle includes identifying a problem, defining the objective, collecting or selecting data, understanding and preparing the data, choosing an approach, building a prototype, testing it, evaluating outcomes and presenting findings. The stages may be revisited when testing reveals an issue.

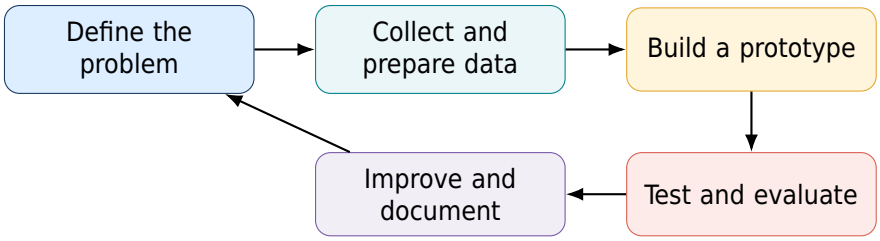


Figure 5.1. An iterative project cycle. Evaluation may lead to improvements or a revised problem definition.

5.2 Project formats and documentation

A project report should communicate the problem, objective, data source, method, implementation, evaluation, limitations and conclusion. Include diagrams or tables when they make the method easier to understand. Code should be organised, named clearly and accompanied by instructions for running it. Data sources and external materials should be acknowledged.

5.2.1 Choosing evaluation measures

The measure depends on the task. A classification project may report accuracy and, where appropriate, precision, recall and a confusion matrix. A numerical prediction task may use an error measure such as mean absolute error. A visualisation project should be assessed for correctness, readability and whether it answers the original question. A single score rarely tells the whole story.

5.3 Project review

A useful review considers whether the project solves the stated problem, whether the data and method are appropriate, whether the result can be reproduced, and whether limitations are acknowledged. Reviewers should consider privacy, possible bias, safety and the consequences of incorrect predictions. Clear documentation makes the project easier to understand and maintain.

Review area	Questions to consider
Problem and scope	Is the objective clear and achievable?
Data	Are the source, quality, permissions and limitations recorded?
Method	Is the selected approach suitable for the task?
Evaluation	Are the measures relevant and the results honestly reported?
Responsible use	Have privacy, bias and potential harm been considered?
Documentation	Can another person understand and reproduce the work?

5.4 Sample project: student marks analysis

A small marks-analysis project is a useful example of a data-focused AI learning task. A learner can create a table of fictional student marks, inspect the values with Pandas, calculate summaries with NumPy and present subject comparisons using Matplotlib. This project demonstrates data handling and communication; it should not be described as an AI prediction model unless a learning method is actually implemented and evaluated.

```
import pandas as pd
import matplotlib.pyplot as plt

data = pd.DataFrame({
    "Subject": ["English", "Maths", "Science"],
    "Marks": [76, 88, 81]
})
print("Average:", data["Marks"].mean())
plt.bar(data["Subject"], data["Marks"])
plt.title("Sample Marks by Subject")
plt.ylabel("Marks")
plt.tight_layout()
plt.show()
```

The values are illustrative, not real student records. A report should describe the data, explain each step, include the chart, note limitations and avoid identifying real learners without a valid reason and appropriate permission.

5.4.1 Planning the work and presenting results

Before writing code, prepare a short plan with the project objective, required inputs, expected outputs and a list of tasks. Divide the work into manageable steps, such as preparing a data file, checking its quality, performing calculations, creating a chart and writing a conclusion. A small prototype should be tested early so that major problems are discovered before the final report is prepared.

The final presentation should explain the original question, the approach used and the evidence that supports the conclusion. Screenshots or diagrams can help readers follow the process, but they should be relevant and clearly labelled. If the result is uncertain or the dataset is small, state that limitation rather than making a broader claim than the evidence supports. Keep a copy of the code and note the software libraries required to run it.

5.4.2 Defining a measurable project objective

A good objective describes the problem, the intended user and the result that would count as useful. “Use AI to improve education” is too broad to guide a small project. “Create a chart that compares fictional marks across three subjects and explain the

highest and lowest values" is specific and can be completed with a small dataset. If the project claims to predict an outcome, the report must explain how the prediction is trained and evaluated.

Before selecting a tool, ask whether AI is necessary. A clear set of rules or a simple chart may solve the problem more transparently than a machine-learning model. The best project is not necessarily the one with the most complex technology; it is the one that addresses the stated need safely and convincingly.

5.4.3 Data permissions and documentation

Use data that is permitted for the project. For classroom practice, fictional or appropriately anonymised examples are often sufficient. Do not upload personal records to online services unless there is a valid reason and appropriate permission. Keep track of the data source, collection date, fields, units, cleaning decisions and known limitations. If external code or images are used, acknowledge the source and respect its licence.

5.4.4 A practical project schedule

Divide a project into small tasks: define the question, prepare data, implement the method, test the result, create a visual explanation and write the report. Set a completion check for each task. Keep the scope realistic so that enough time remains for testing and documenting the work. If a step takes longer than expected, simplify optional features rather than skipping evaluation.

5.4.5 Interpreting results honestly

A result should be described in proportion to the evidence. A small sample may illustrate a method but cannot represent every learner or community. A correlation between two measurements does not prove that one causes the other. If the output changes substantially when one data point is removed, mention that sensitivity. Include failures and limitations as part of the report, not as details to hide.

5.4.6 Project report structure

A useful report can contain a title, problem statement, objective, data description, method, tools, implementation steps, results, evaluation, limitations, responsible-use considerations and conclusion. Each section should help the reader understand a different part of the work. Diagrams should be labelled, code should be formatted clearly, and every figure should be discussed in the text. A conclusion should answer the original question and avoid claims that were not tested.

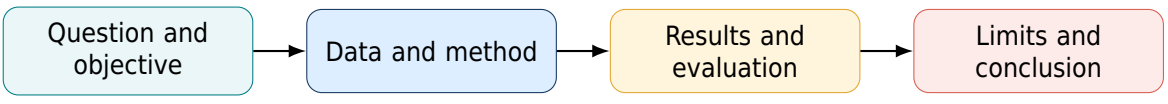


Figure 5.2. A logical structure for explaining an AI project.

5.4.7 Reflection after completion

After completing the project, review whether the original objective was met, what evidence supports the result and which steps were difficult. Identify one improvement that would make the result more reliable or easier to use. A useful reflection is specific: it describes what happened and what would be changed next time, rather than simply saying that the project was good or bad.

Review Questions

1. List the major stages of an AI project.
2. Why is the problem statement important?
3. What should a project report contain?
4. Explain why evaluation measures depend on the task.
5. Identify two privacy or fairness questions to consider before using data.

HANDBOOK

Junior Software Developer

Grade XI – IT-ITeS Vocational Education

Computer Systems • Networks • Python • Data Structures

A structured textbook for computing foundations, logical problem solving and introductory programming.

Prepared as a learning-focused companion based on the supplied Junior Software Developer Grade XI draft study material.

Contents

1 Computer System Architecture **1**
1.1 The information-processing cycle . . . 1
1.2 CPU and its components . . . 1
1.3 Memory and storage . . . 1
1.4 Input, output and peripheral devices . . . 2
1.5 Choosing hardware for development . . . 2

2 Data Representation in Computers **3**
2.1 Bits, bytes and number systems . . . 3
2.2 Converting number systems . . . 3
2.3 Text, images and sound . . . 3
2.4 Data quality and safe handling . . . 3

3 Basic Mathematics and Statistics for Computing **4**
3.1 Sets and operations . . . 4
3.2 Logic and truth values . . . 4
3.3 Relations, functions and graphs . . . 4
3.4 Basic statistics . . . 4

4 Problem-Solving Methods **5**
4.1 Define and analyse the problem . . . 5
4.2 Algorithms and pseudocode . . . 5
4.3 Flowcharts . . . 5
4.4 Testing and debugging . . . 5
4.5 Efficiency and documentation . . . 6

5 Programming Language Concepts **7**
5.1 Machine, assembly and high-level languages . . . 7
5.2 Compiler, interpreter and runtime . . . 7
5.3 Syntax, semantics and paradigms . . . 7
5.4 Development tools . . . 7

6 Operating System Fundamentals **8**
6.1 Core responsibilities . . . 8
6.2 Processes and memory . . . 8
6.3 Files, folders and permissions . . . 8
6.4 User interfaces and maintenance . . . 8

7 Computer Networks **10**
7.1 Network types and topologies . . . 10
7.2 Network devices . . . 10
7.3 Addresses and name resolution . . . 10
7.4 Client-server and peer-to-peer . . . 10

8 Transmission Media and Network Protocols **12**
8.1 Guided and unguided media . . . 12
8.2 Protocol layers . . . 12
8.3 Packets and ports . . . 12
8.4 Troubleshooting connectivity . . . 12

9 Network Security **13**
9.1 Threats and vulnerabilities . . . 13
9.2 Security controls . . . 13
9.3 Safe network behaviour . . . 13
9.4 Incident response . . . 13

10 Python Basics **15**
10.1 Installing and running Python . . . 15
10.2 Variables, names and data types . . . 15
10.3 Operators and expressions . . . 15
10.4 Comments, indentation and errors . . . 16

11 Control Structures **17**
11.1 Selection with if . . . 17
11.2 Iteration with for and while . . . 17
11.3 Loop control and tracing . . . 17

12 Functions **18**
12.1 Defining and calling functions . . . 18
12.2 Parameters, arguments and scope . . . 18
12.3 Built-in and user-defined functions . . . 18
12.4 Testing functions . . . 18

13 Strings **19**
13.1 Creating and accessing strings . . . 19
13.2 String methods . . . 19
13.3 Formatting and validation . . . 19

14 Lists **20**
14.1 Creating and accessing lists . . . 20
14.2 Traversing and processing . . . 20
14.3 Nested lists and common mistakes . . . 20

15 Tuples and Dictionaries **21**
15.1 Tuples . . . 21
15.2 Dictionaries . . . 21
15.3 Choosing a collection . . . 21

16 Introduction to Data Structures **22**
16.1 Abstract data types and operations . . . 22
16.2 Linear and non-linear structures . . . 22
16.3 Selecting a structure . . . 22

17 Linear Data Structures **23**
17.1 Arrays . . . 23
17.2 Linked lists . . . 23
17.3 Stacks . . . 23
17.4 Queues . . . 23

18 Non-Linear Data Structures **24**
18.1 Trees and terminology 24
18.2 Tree traversal 24
18.3 Graphs 24
18.4 Applications 24

19 Integrated Practice and Revision **25**
19.1 Scenario: a learner-results application 25
19.2 Suggested design 25
19.3 Testing plan 25
19.4 Final revision questions 25
19.5 Study and problem-solving approach 26

Chapter 1

Computer System Architecture

A computer is an electronic system that accepts data, processes it according to instructions, stores information and produces results. The same basic idea appears in desktop computers, laptops, smartphones, servers and embedded devices. Understanding the system underneath software helps a developer make sensible decisions about performance, storage, compatibility and troubleshooting.

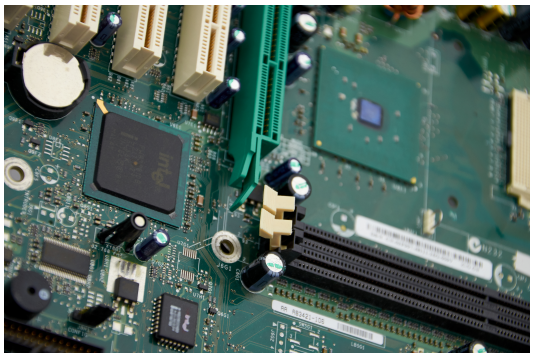


Figure 1.1. A motherboard connects the processor, memory and other essential components; photograph: Lenharth Systems, Wikimedia Commons, CC0 1.0.

1.1 The information-processing cycle

The common stages are input, processing, output, storage and communication. A keyboard, camera or sensor supplies input. The CPU executes instructions and processes data. A display, printer or speaker presents output. Storage keeps information for later use, while communication transfers data between devices.

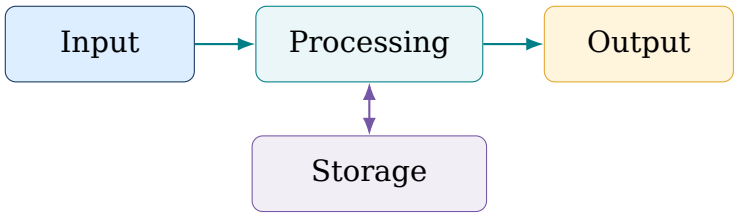


Figure 1.2. A simplified view of how computer components exchange information.

1.2 CPU and its components

The central processing unit (CPU) executes program instructions. The control unit coordinates operations, the arithmetic logic unit (ALU) performs arithmetic and logical comparisons, and registers hold small amounts of data immediately needed by the processor. Modern CPUs may contain multiple cores, allowing them to work on more than one stream of instructions.

1.3 Memory and storage

Primary memory includes RAM and ROM. RAM is volatile: its contents are normally lost when power is removed. ROM and other non-volatile memory retain information

without continuous power. Cache is small, fast memory used to reduce the time needed to access frequently used data. Secondary storage, such as solid-state drives, hard disk drives, USB flash drives and memory cards, retains files and programs for longer periods.

Component	Main purpose	Important property
RAM	Holds active programs and working data	Fast, volatile memory
ROM / firmware memory	Stores startup or device instructions	Non-volatile
Cache	Keeps frequently needed data close to CPU	Very fast, limited capacity
SSD / HDD	Stores files, applications and operating system	Non-volatile, larger capacity

1.4 Input, output and peripheral devices

Input devices include keyboards, mice, touchscreens, scanners, microphones, webcams, barcode readers, OCR and OMR equipment. Output devices include monitors, printers, speakers and projectors. A touchscreen can act as both input and output. Peripheral devices connect to a computer through interfaces such as USB, Bluetooth, HDMI or network connections.

1.5 Choosing hardware for development

Software development workloads vary. A basic editor and small Python scripts need modest resources, while virtual machines, large projects and data processing may require more memory, storage and CPU capacity. A developer should check operating-system support, available ports, upgrade options, reliability and backup requirements instead of choosing hardware by one specification alone.

Review Questions

1. Explain the input-processing-output cycle with one everyday example.
2. Distinguish the control unit and the arithmetic logic unit.
3. Compare RAM, ROM and cache memory.
4. Give four examples each of input and output devices.
5. What factors should a learner consider when selecting a computer for programming?

Chapter 2

Data Representation in Computers

Computers represent data using binary digits, 0 and 1. Electronic circuits can use two distinguishable states to encode these digits. Text, numbers, pictures, sound and video are all represented through patterns of bits, although the encoding rules differ.

2.1 Bits, bytes and number systems

A bit is a binary digit. A group of eight bits is called a byte. Larger quantities are commonly expressed using kilobytes, megabytes, gigabytes and terabytes; exact conventions may vary between decimal storage units and binary units such as KiB and MiB. Binary uses base 2, decimal uses base 10, octal uses base 8 and hexadecimal uses base 16.

2.2 Converting number systems

To convert a positive decimal integer to binary, repeatedly divide by two and record the remainders. Read the remainders from bottom to top. For example, decimal 13 becomes binary 1101 because $13 = 8 + 4 + 1$. To convert binary 1011 to decimal, calculate $1(2^3) + 0(2^2) + 1(2^1) + 1(2^0) = 11$.

Hexadecimal uses the symbols 0–9 and A–F, where A represents ten and F represents fifteen. One hexadecimal digit corresponds to four binary bits, making hexadecimal a compact way to read long binary values.

2.3 Text, images and sound

Character encodings assign numeric codes to characters. ASCII represents a basic set of characters; Unicode supports characters from many writing systems. UTF-8 is a widely used Unicode encoding for text on the web. Digital images use pixels and colour values; audio can be represented by samples taken at intervals. Image resolution, colour depth, sample rate and compression affect file size and quality.

2.4 Data quality and safe handling

Encoding does not guarantee that data is accurate. Developers must also validate inputs, use suitable data types, handle missing values and protect personal information. Files should have meaningful names and be backed up according to their importance.

Review Questions

1. Define bit and byte.
2. Convert decimal 13 to binary and show the method.
3. Convert binary 1011 to decimal.
4. Why is hexadecimal useful when working with binary values?
5. Distinguish ASCII and Unicode.

Chapter 3

Basic Mathematics and Statistics for Computing

Mathematics helps programmers describe relationships, compare values and reason about the behaviour of algorithms. Discrete mathematics focuses on distinct objects and structures, such as sets, logical statements, relations, functions, graphs and counting problems.

3.1 Sets and operations

A set is a collection of distinct elements. If $A = \{1, 2, 3\}$ and $B = \{3, 4, 5\}$, the union contains elements in either set, $A \cup B = \{1, 2, 3, 4, 5\}$, while the intersection contains elements shared by both, $A \cap B = \{3\}$. The difference $A - B$ contains elements in A that are not in B. These ideas help model categories, permissions and relationships between data groups.

3.2 Logic and truth values

A proposition is a statement that can be true or false. Logical operators include AND, OR and NOT. In programming, conditions combine comparisons to decide which instructions should run. A truth table lists the possible inputs and resulting truth values and is useful when a condition becomes complicated.

3.3 Relations, functions and graphs

A relation describes a connection between elements. A function maps each valid input to exactly one output. Graphs consist of vertices and edges and can represent routes, social connections, network links or dependencies between tasks. Directed edges indicate direction; undirected edges represent a two-way relationship unless otherwise specified.

3.4 Basic statistics

Mean is the arithmetic average, median is the middle value after sorting, and mode is the most frequent value. Range is the difference between the maximum and minimum. Statistics help summarise test results, response times and software usage, but conclusions should consider sample size, data quality and unusual values.

Review Questions

1. Find the union and intersection of $A = \{1, 2, 3\}$ and $B = \{3, 4, 5\}$.
2. Explain AND, OR and NOT with simple examples.
3. Distinguish a relation from a function.
4. What are vertices and edges in a graph?
5. Find the mean and median of 4, 6, 7, 7 and 11.

Chapter 4

Problem-Solving Methods

Programming begins with understanding a problem, not with typing code. A reliable solution identifies the expected result, the available input, the rules that transform input into output and the special cases that may cause failure.

Figure 4.1. A programming editor helps developers write and inspect instructions; screenshot: Zhanjack822, Wikimedia Commons, CC0 1.0.

4.1 Define and analyse the problem

Read the requirement carefully. Separate known facts from assumptions, identify constraints and ask questions where the requirement is unclear. Determine valid input ranges, expected output format and what should happen when data is missing or invalid. A small example often exposes misunderstandings early.

4.2 Algorithms and pseudocode

An algorithm is a finite, ordered set of steps for solving a problem. Good algorithms are clear, terminate, accept appropriate input and produce the required output. Pseudocode describes logic using plain structured language without committing to a particular programming language.

```
START
  INPUT first_number, second_number
  total = first_number + second_number
  OUTPUT total
STOP
```

4.3 Flowcharts

Flowcharts use standard symbols to show the flow of a process. An oval commonly represents start or end, a parallelogram represents input or output, a rectangle represents a process, and a diamond represents a decision. Arrows show the direction of control. A flowchart should be readable and avoid crossing lines where possible.

4.4 Testing and debugging

A test checks whether a program behaves as expected for selected input. Test normal values, boundary values, invalid input and empty input where relevant. A syntax

error breaks the language rules; a runtime error occurs while the program runs; a logic error produces an incorrect result without necessarily crashing. Debugging means locating and correcting the cause, then rerunning tests.

4.5 Efficiency and documentation

Efficiency can involve execution time and memory use. For small problems, clarity is often the most important goal; for large inputs, algorithm choice can make a substantial difference. Comments should explain intent or non-obvious decisions, while documentation describes how to install, run and use the software.

Review Questions

1. List the main stages of problem solving.
2. Define an algorithm and state two qualities of a good algorithm.
3. Name four common flowchart symbols and their meanings.
4. Distinguish syntax, runtime and logic errors.
5. Why should a program be tested with boundary and invalid values?

Chapter 5

Programming Language Concepts

A programming language provides rules for expressing instructions that a computer can execute directly or through translation. Languages differ in syntax, type systems, abstraction, libraries, runtime environments and the kinds of problems for which they are commonly used.

5.1 Machine, assembly and high-level languages

Machine language consists of instructions encoded for a processor. Assembly language uses symbolic names for low-level instructions and is translated by an assembler. High-level languages such as Python, Java and C++ provide abstractions that make many programs easier to read and maintain.

5.2 Compiler, interpreter and runtime

A compiler translates source code into another form before execution; an interpreter executes instructions through a runtime process. Real language implementations may combine compilation, interpretation and virtual machines. A runtime provides services needed while a program executes, such as memory management or standard libraries.

5.3 Syntax, semantics and paradigms

Syntax describes the structure of valid expressions and statements. Semantics concerns their meaning. Procedural programming organises work into procedures or functions; object-oriented programming models data and behaviour through objects and classes; functional programming emphasises functions and expressions. Many languages support more than one paradigm.

5.4 Development tools

A code editor helps create and edit source files. An integrated development environment (IDE) may combine an editor, debugger, build tools, project navigation and testing support. Version control records changes over time and helps teams collaborate. Good naming, small functions, consistent formatting and clear commit messages improve maintainability.

Review Questions

1. Compare machine language, assembly language and high-level language.
2. Explain one difference between a compiler and an interpreter.
3. Distinguish syntax from semantics.
4. Describe procedural and object-oriented programming.
5. What features might an IDE provide?

Chapter 6

Operating System Fundamentals

An operating system (OS) manages the hardware resources of a computing device and provides services to applications. It offers a consistent way to work with files, memory, devices, processes and users. Examples include Windows, Linux, macOS, Android and iOS.

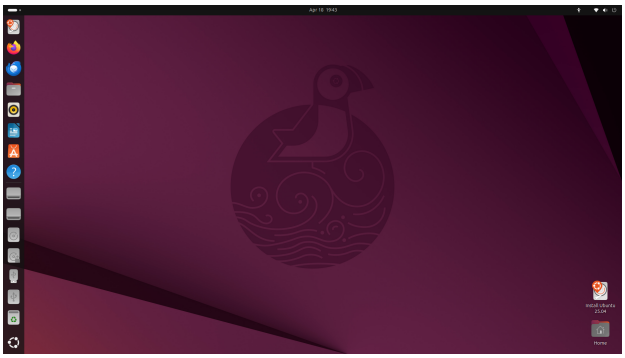


Figure 6.1. The operating system provides the desktop, application launcher and workspace; screenshot: Canonical Limited, Wikimedia Commons, GPL.

6.1 Core responsibilities

The OS schedules CPU time, manages memory, organises files, controls device access, provides a user interface and enforces security boundaries. It also detects certain errors and supports logging, networking and system updates. The kernel is the central component that manages privileged operations; shells and graphical interfaces allow users to interact with system services.

6.2 Processes and memory

A program is stored instructions; a process is a running instance of a program. The OS allocates CPU time and memory, tracks process states and coordinates input/output. Multitasking allows several tasks to make progress over time. Virtual memory helps manage address spaces and may use storage to extend the available memory model, though it is slower than RAM.

6.3 Files, folders and permissions

Files store content and folders organise files into a hierarchy. File names, extensions and metadata help users identify content. Permissions determine who may read, modify or execute files. Keep project files organised, use clear names, back up important data and avoid granting broader permissions than needed.

6.4 User interfaces and maintenance

A command-line interface accepts typed commands; a graphical user interface uses visual elements such as windows, menus and icons. Updates repair defects and address security issues. Install software from trusted sources, use standard user accounts for everyday work and keep recovery information available.

Review Questions

1. Define an operating system and give four examples.
2. Explain the roles of kernel and shell.
3. Distinguish a program and a process.
4. What is virtual memory?
5. State five safe computer-maintenance practices.

Chapter 7

Computer Networks

A computer network connects devices so they can exchange information and share services. Networks support web browsing, messaging, file transfer, printing, online learning and distributed applications. A network may be small and local or connect devices across continents.

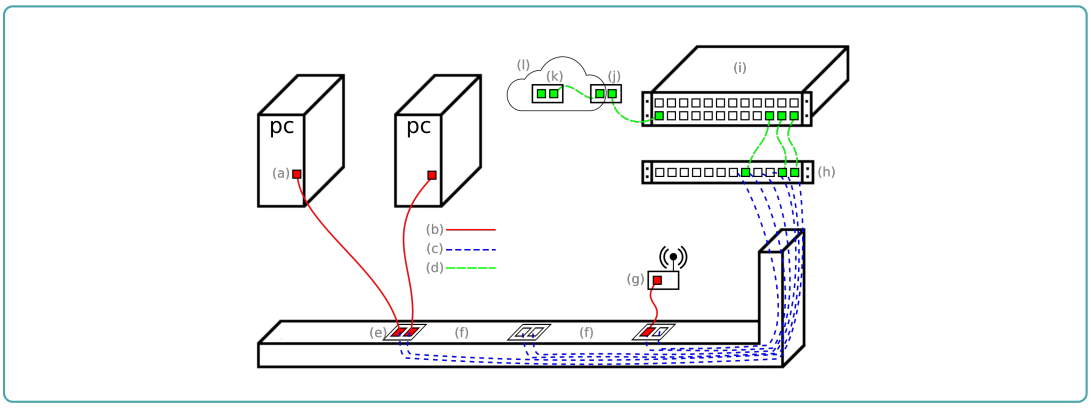


Figure 7.1. A local-area network connects end devices through outlets, switches and a router; diagram: Mattias.Campe, Wikimedia Commons, CC BY-SA 3.0.

7.1 Network types and topologies

A personal area network (PAN) connects nearby personal devices. A local area network (LAN) covers a limited site such as a home, laboratory or office. A wide area network (WAN) connects larger geographic areas. Common topologies include star, bus, ring and mesh; each has different cost, resilience and troubleshooting characteristics.

7.2 Network devices

A network interface connects a device to a network. Switches forward frames within a local network, while routers forward packets between networks. Wireless access points connect Wi-Fi devices. Modems or other access equipment connect a local network to an internet service. The exact device combination depends on the provider and network design.

7.3 Addresses and name resolution

A MAC address identifies a network interface at the link layer. An IP address supports logical addressing and routing. DNS translates domain names such as example.com into records used to locate services. DHCP can automatically assign network configuration such as IP addresses, gateway and DNS servers.

7.4 Client-server and peer-to-peer

In a client-server model, clients request services from servers. A web browser requesting a page is a common example. In peer-to-peer systems, devices can share responsibilities more directly. Both models require appropriate authentication, access control and safe handling of data.

Review Questions

1. Compare LAN and WAN.
2. Explain the purpose of a switch and a router.
3. Distinguish MAC and IP addresses.
4. What does DNS do?
5. Compare client-server and peer-to-peer models.

Chapter 8

Transmission Media and Network Protocols

Data travels through wired or wireless media. The medium affects speed, range, interference, installation cost and security. Network protocols define how devices format, address, transmit and interpret messages so that different systems can communicate.

8.1 Guided and unguided media

Twisted-pair copper cable is common in Ethernet LANs. Coaxial cable is used in some broadband and video systems. Optical fibre carries light and supports high bandwidth over long distances with low electromagnetic interference. Wireless communication uses radio waves, including Wi-Fi and cellular networks, and may be affected by walls, distance and interference.

8.2 Protocol layers

Layered networking divides a complex communication task into manageable responsibilities. Ethernet and Wi-Fi operate at the link level; IP provides addressing and routing; TCP offers reliable ordered delivery; UDP offers a simpler datagram service; HTTP and HTTPS support web communication. TLS helps protect communication against eavesdropping and tampering when configured correctly.

8.3 Packets and ports

Applications send data through protocols that may divide it into packets or segments. Port numbers help the operating system deliver traffic to the appropriate service. Firewalls can allow or block traffic according to rules. Open only the ports required for a service and keep network software updated.

8.4 Troubleshooting connectivity

Check power and physical connections first. Confirm Wi-Fi or Ethernet status, then inspect IP configuration and DNS resolution. Try to isolate whether the problem affects one device, one service or the whole network. Record error messages and change one variable at a time so the cause can be identified.

Review Questions

1. Compare twisted-pair cable and optical fibre.
2. Explain why network protocols are needed.
3. Distinguish TCP and UDP at a high level.
4. What is the role of a port number?
5. List a sensible sequence for troubleshooting a network connection.

Chapter 9

Network Security

Network security protects systems, communications and information from unauthorised access, misuse and disruption. Security depends on technology and human behaviour: even well-designed systems can be exposed by weak passwords, unpatched software, unsafe configuration or deceptive messages.

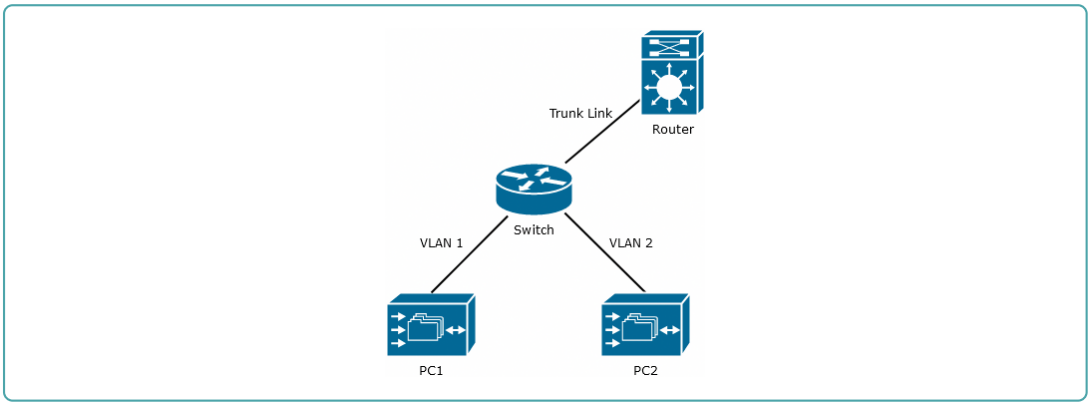


Figure 9.1. A router can route traffic between separate virtual LANs; diagram: Anatoly099, Wikimedia Commons, CC BY-SA 4.0.

9.1 Threats and vulnerabilities

Malware includes viruses, worms, trojans, ransomware and spyware. Phishing attempts to trick people into revealing information or running unsafe files. Password attacks, unpatched services, misconfiguration and denial-of-service attacks can affect confidentiality, integrity or availability. A vulnerability is a weakness that a threat may exploit.

9.2 Security controls

Use strong unique passwords and multifactor authentication where available. Apply updates, use endpoint protection, enable firewalls and restrict privileges. Back up important information and test restoration. Encrypt sensitive data in transit and at rest where appropriate. Security controls should be selected based on the risks and the value of the information being protected.

9.3 Safe network behaviour

Verify unexpected links and attachments through a trusted channel. Avoid entering credentials on unfamiliar pages. Use secure Wi-Fi configurations and be cautious on public networks. Do not share passwords or one-time codes. Report suspicious activity promptly and preserve relevant details according to organisational procedure.

9.4 Incident response

A basic response includes identifying the event, limiting further harm, informing the appropriate people, preserving evidence, recovering safely and reviewing what can be improved. Do not attempt to hide an incident or destroy records. Follow organisational policy and applicable law.

Review Questions

1. Define malware, phishing and vulnerability.
2. Explain the confidentiality, integrity and availability goals of security.
3. Give five ways to protect a computer on a network.
4. Why are backups and restoration tests important?
5. List the main stages of a basic incident response.

Chapter 10

Python Basics

Python is a high-level, general-purpose programming language used for automation, data analysis, web development, scientific computing and many other tasks. Its readable syntax makes it suitable for learning programming concepts while still supporting large applications.

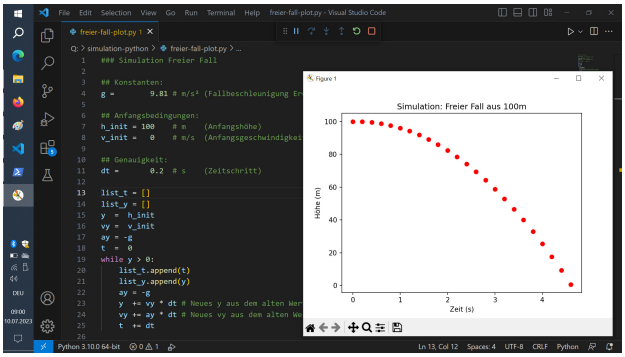


Figure 10.1. A Python program can combine source code and a visual output window; screenshot: MikeRun, Wikimedia Commons, CC BY-SA 4.0.

10.1 Installing and running Python

Use the official Python distribution or an approved environment. The interpreter executes Python statements. IDLE is a simple environment that includes an editor and interactive shell; other editors and IDEs are also available. Check the installation by running a short command and confirming the version reported by the system.

```
print("Hello, learner!")
name = input("Enter your name: ")
print("Welcome,", name)
```

The first line displays a message. The second reads text from the user and stores it in a variable. The final line combines output with the entered value.

10.2 Variables, names and data types

A variable is a name that refers to a value. Identifiers should be meaningful and follow Python’s naming rules. Common types include int for integers, float for floating-point numbers, str for text, bool for true/false values, and collections such as list, tuple, set and dict. Python is case-sensitive, so `score` and `Score` are different names.

10.3 Operators and expressions

Arithmetic operators include +, -, *, /, //, % and **. Comparison operators include ==, !=, <, >, <= and >=. Logical operators and, or and not combine conditions. Parentheses make complicated expressions easier to read. Use explicit conversion such as `int(input(...))` when numeric input is required, and handle invalid input where necessary.

10.4 Comments, indentation and errors

Comments begin with the hash symbol. Indentation defines blocks in Python and must be consistent. Syntax errors prevent code from being parsed; exceptions occur during execution; logic errors produce the wrong answer. Read tracebacks from the final error message and inspect the relevant line before changing code.

Review Questions

1. State four common uses of Python.
2. Write a program that displays a message and reads a user’s name.
3. Compare int, float, str and bool.
4. Explain the difference between = and ==.
5. Why is indentation important in Python?

Chapter 11

Control Structures

Control structures determine the order in which instructions run. Sequence executes statements in order, selection chooses between alternatives, and iteration repeats a block. These structures allow a program to respond to input and process collections of data.

11.1 Selection with if

An `if` statement runs a block when a condition is true. `elif` checks another condition, and `else` provides a fallback. Conditions may use comparison and logical operators.

```
marks = int(input("Enter marks: "))
if marks >= 80:
    print("Grade A")
elif marks >= 60:
    print("Grade B")
elif marks >= 40:
    print("Grade C")
else:
    print("Needs improvement")
```

The order matters: the most restrictive threshold is tested first. Validate input ranges if marks must lie between 0 and 100.

11.2 Iteration with for and while

A `for` loop is useful for iterating over a sequence or range. A `while` loop repeats while a condition remains true. Each loop should make progress toward termination unless it is intentionally designed to run continuously.

```
for number in range(1, 6):
    print(number)

count = 3
while count > 0:
    print(count)
    count -= 1
```

11.3 Loop control and tracing

`break` exits a loop and `continue` skips to the next iteration. A trace table records variable values at each step and is useful for checking a loop's logic. Avoid changing many variables in a single step when debugging a beginner program.

Review Questions

1. Compare sequence, selection and iteration.
2. Write an if-else program to determine whether a number is even or odd.
3. What is the difference between for and while loops?
4. Explain break and continue.
5. Trace a loop that prints numbers from 1 to 5.

Chapter 12

Functions

A function is a named block of code that performs a defined task. Functions help divide a problem into smaller parts, avoid repeated code and make programs easier to test. A function can receive inputs called parameters and return a result.

12.1 Defining and calling functions

Use the `def` keyword to define a function. The indented body contains the instructions. Calling a function runs that body; a return statement sends a value back to the caller.

```
def rectangle_area(length, width):
    return length * width

area = rectangle_area(8, 3)
print("Area:", area)
```

12.2 Parameters, arguments and scope

Parameters appear in the function definition; arguments are values supplied when it is called. Local variables are created within a function and normally cannot be accessed directly outside it. Default parameters can make an argument optional. Choose names that communicate the purpose of each value.

12.3 Built-in and user-defined functions

Python provides built-in functions such as `print`, `input`, `len`, `type`, `sum`, `min` and `max`. User-defined functions capture a task specific to a program. A useful function has a clear responsibility, predictable inputs and outputs, and a name that explains its role.

12.4 Testing functions

Test normal cases, boundary values and invalid inputs. A function that calculates a percentage should define what happens when the total is zero. Small tests make errors easier to isolate and allow later changes without manually checking every part of the application.

Review Questions

1. Define a function and state two benefits.
2. Write a function that returns the square of a number.
3. Distinguish parameters and arguments.
4. What is the purpose of return?
5. Why should functions be tested with boundary values?

Chapter 13

Strings

A string is an immutable sequence of characters. Strings represent names, messages, addresses, codes and other text. Because strings are immutable, an operation that appears to change a string normally returns a new string rather than altering the original object.

13.1 Creating and accessing strings

Strings can be written with single or double quotes. Triple quotes can represent multiline text. Indexing accesses a character; positive indexes start at zero, while negative indexes count from the end. Slicing selects a range and uses an exclusive stop index.

```
word = "Python"
print(word[0])      # P
print(word[-1])     # n
print(word[1:4])    # yth
print(len(word))    # 6
```

13.2 String methods

Useful methods include `lower`, `upper`, `strip`, `replace`, `split`, `join`, `startswith` and `endswith`. The `in` operator checks whether a substring is present. Methods should be selected based on the intended transformation and the format of the input.

13.3 Formatting and validation

F-strings make it convenient to insert values into text, for example `f"Hello, {name}"`. User input should be validated for expected length, allowed characters and required format. Validation is not the same as security by itself; applications should also apply context-appropriate encoding and access controls.

Review Questions

1. What does it mean that strings are immutable?
2. Explain indexing and slicing.
3. Name five useful string methods.
4. Write code to convert a name to uppercase and remove surrounding spaces.
5. Why should user-provided text be validated?

Chapter 14

Lists

A list is an ordered, mutable collection that can hold multiple values. Lists are useful when a program needs to store, examine, sort or update a group of items. List elements can have different types, although collections with a consistent type are often easier to work with.

14.1 Creating and accessing lists

A list uses square brackets. Indexing retrieves an element and slicing retrieves a sublist. Lists support updates, append, insert, remove, pop, sort and reverse. These operations change the list in different ways, so consult the method's purpose before using it.

```
scores = [72, 85, 64, 91]
scores.append(78)
scores[0] = 75
print(scores)
print("Count:", len(scores))
print("Highest:", max(scores))
```

14.2 Traversing and processing

A for loop can visit each element. A running total can calculate a sum, and a condition can count values that meet a requirement. Python also provides built-in functions and comprehensions for common transformations. Begin with a clear loop when learning, then use concise expressions when their meaning remains easy to understand.

14.3 Nested lists and common mistakes

A list can contain other lists, which can represent rows and columns. Indexing outside the valid range raises an error. Removing an item while iterating over the same list can lead to skipped values; when filtering, creating a new list is often safer.

Review Questions

1. Define a list and explain why it is mutable.
2. Write a program to find the sum of numbers in a list.
3. Compare append, insert and remove.
4. What is a nested list?
5. Describe two common list errors and how to avoid them.

Chapter 15

Tuples and Dictionaries

Tuples and dictionaries organise related data in different ways. A tuple is ordered and immutable. A dictionary maps unique keys to values and is useful when values need to be retrieved by a meaningful label.

15.1 Tuples

Tuples are written using commas, often inside parentheses. They are useful for fixed groups of values, such as coordinates or a returned pair of results. A one-element tuple requires a trailing comma. Since tuples are immutable, their element assignments cannot be changed after creation.

15.2 Dictionaries

A dictionary uses key-value pairs. Keys must be hashable and unique within the dictionary. Values can be updated, added or removed. Methods such as `keys`, `values`, `items`, `get`, `update` and `pop` support common operations.

```
learner = {
    "name": "Ananya",
    "grade": 11,
    "active": True
}
print(learner["name"])
learner["grade"] = 12
print(learner.get("course", "Not assigned"))
```

15.3 Choosing a collection

Use a list when order and changeable elements are important. Use a tuple for a fixed sequence. Use a dictionary when a key should identify a value. Choose the structure that makes the data and intended operations easiest to understand.

Review Questions

1. Compare lists and tuples.
2. Why must dictionary keys be unique?
3. Write a dictionary describing a book with three fields.
4. Explain the difference between dictionary indexing and get.
5. Give one suitable use for each of list, tuple and dictionary.

Chapter 16

Introduction to Data Structures

A data structure is a way of organising data so that operations can be performed effectively. Different structures support different patterns of access, insertion, deletion and searching. Choosing a suitable structure can improve clarity and performance.

16.1 Abstract data types and operations

An abstract data type describes the behaviour expected from a collection without requiring one specific implementation. Common operations include insert, delete, search, traverse and update. A stack, for example, supports adding and removing items at one end; a queue processes items in arrival order.

16.2 Linear and non-linear structures

Linear structures arrange elements in a sequence, as in arrays, linked lists, stacks and queues. Non-linear structures express branching or network relationships, as in trees and graphs. A structure's usefulness depends on the problem rather than on whether it is more advanced.

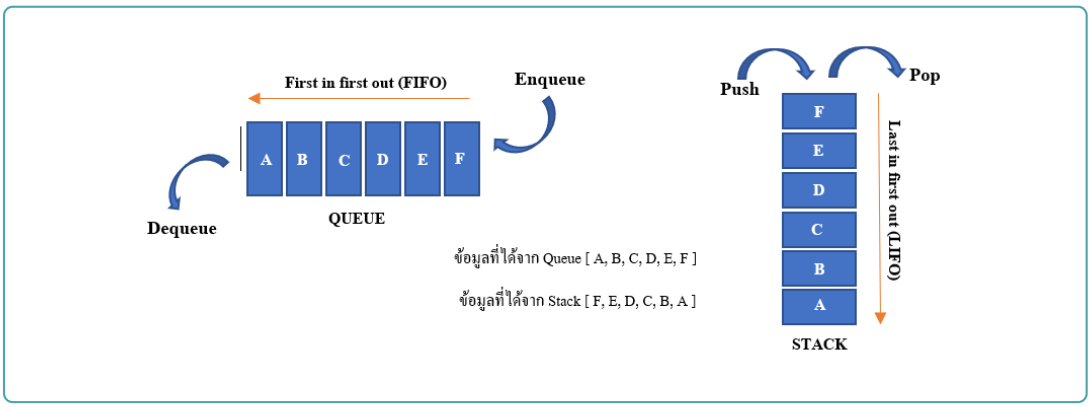


Figure 14.1. A stack follows last-in, first-out order, while a queue follows first-in, first-out order; diagram: Choteseeworld, Wikimedia Commons, CC BY-SA 4.0.

16.3 Selecting a structure

Consider the data's relationships, the operations used most often, memory constraints and implementation complexity. For frequent lookup by key, a dictionary or hash table may be appropriate. For hierarchical information, a tree may fit naturally. For ordered processing, a queue may be the simplest model.

Review Questions

1. Define a data structure.
2. List five common data-structure operations.
3. Compare linear and non-linear structures.
4. Give one real-world example of a stack and a queue.
5. What factors influence the choice of a data structure?

Chapter 17

Linear Data Structures

Linear data structures arrange elements in a sequence. Arrays, linked lists, stacks and queues are common examples. Each offers a different balance between access speed, insertion, deletion and memory organisation.

17.1 Arrays

An array stores elements in indexed positions. Index-based access is efficient when the position is known. Inserting or deleting elements in the middle may require moving other values. Python lists behave like dynamic arrays for many common uses.

17.2 Linked lists

A linked list contains nodes. Each node stores data and one or more links to other nodes. A singly linked list points to the next node; a doubly linked list points both forward and backward. Linked lists can insert or remove nodes efficiently when the relevant position is known, but finding a distant element usually requires traversal from the start.

17.3 Stacks

A stack follows last-in, first-out (LIFO). The main operations are push, pop and peek. A stack models undo operations, function calls and expression evaluation. Removing from an empty stack should be handled as an error or an explicitly defined empty result.

17.4 Queues

A queue follows first-in, first-out (FIFO). Enqueue adds an item and dequeue removes the oldest item. Queues model print jobs, service requests and tasks waiting for processing. Circular queues can reuse array space, while priority queues process items according to priority rather than arrival order.

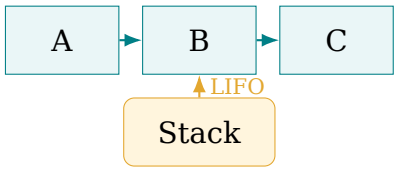


Figure 15.1. Sequence-based structures support different ways of adding and removing items.

Review Questions

1. Compare arrays and linked lists.
2. Explain the LIFO rule with an example.
3. Explain the FIFO rule with an example.
4. What do push, pop, enqueue and dequeue mean?
5. Which structure would you consider for a printer queue, and why?

Chapter 18

Non-Linear Data Structures

Non-linear data structures represent branching or many-to-many relationships. Trees are hierarchical; graphs model general connections. These structures appear in file systems, routing, dependency analysis, maps and social networks.

18.1 Trees and terminology

A tree consists of nodes connected by edges and has a root node. A parent can have child nodes; a node without children is a leaf. A subtree is a node together with its descendants. A binary tree allows each node to have at most two children. A binary search tree arranges values so that values in the left subtree are smaller and values in the right subtree are larger, according to its ordering rule.

18.2 Tree traversal

Pre-order visits root, left subtree, right subtree. In-order visits left subtree, root, right subtree. Post-order visits left subtree, right subtree, root. For a binary search tree, in-order traversal visits keys in sorted order if the tree follows the required ordering rules.

18.3 Graphs

A graph consists of vertices and edges. It can be directed or undirected, weighted or unweighted. A path is a sequence of connected vertices; a cycle returns to a previously visited vertex. Graphs can be represented by an adjacency list or adjacency matrix. An adjacency list stores neighbours for each vertex, while a matrix records whether pairs of vertices are connected.

18.4 Applications

Trees can represent folders, organisational structures and syntax. Graphs can represent road networks, computer networks and links between web pages. The representation should match the expected operations and the size and density of the structure.

Review Questions

1. Define root, parent, child and leaf in a tree.
2. Explain pre-order, in-order and post-order traversal.
3. What is a binary search tree?
4. Compare directed and undirected graphs.
5. Compare adjacency lists and adjacency matrices.

Chapter 19

Integrated Practice and Revision

19.1 Scenario: a learner-results application

A school wants a small program that stores learner names and marks, calculates class summaries and produces a report. Begin by clarifying the expected input, valid mark range and output format. Select suitable data structures, write an algorithm, implement the solution in Python and test it with normal, boundary and invalid values.

19.2 Suggested design

Use a list to hold marks and a dictionary when each learner needs a labelled record. Create small functions to validate marks, calculate summaries and format output. Handle an empty list before calculating an average. Keep the program's responsibilities separate so that changes to input or output do not require rewriting the calculation logic.

```
def average(values):
    if not values:
        return None
    return sum(values) / len(values)

marks = [72, 85, 64, 91]
result = average(marks)
if result is None:
    print("No marks available")
else:
    print(f"Average: {result:.2f}")
```

19.3 Testing plan

Test an ordinary list of marks, a list containing one value, an empty list, values at the allowed boundaries and invalid values. Check whether the program returns a sensible result, communicates errors clearly and avoids dividing by zero. Record defects and rerun the tests after making changes.

19.4 Final revision questions

1. Describe the main functional units of a computer system.
2. Explain how binary represents data and convert a simple decimal integer to binary.
3. Explain sets, logical operators and functions with examples.
4. Describe the stages of problem analysis, algorithm design, implementation and testing.
5. Compare high-level languages, compilers and interpreters.
6. Explain the responsibilities of an operating system.
7. Compare LAN and WAN and describe the roles of switches and routers.
8. Explain the purpose of IP, TCP, DNS and HTTPS.
9. List practical measures for protecting a computer and network.

10. Write a Python program using selection to classify a number as positive, negative or zero.
11. Write a loop that calculates the total of a list of numbers.
12. Define a function that returns the largest of three values.
13. Explain string indexing and slicing.
14. Compare list, tuple and dictionary with examples.
15. Compare stack and queue and identify their processing order.
16. Explain the difference between a tree and a graph.

19.5 Study and problem-solving approach

For a definition question, provide a clear meaning and one example. For a comparison, state the distinguishing feature and explain where each option is useful. For a programming question, identify the input, outline the logic, write readable code and describe the expected output. Test assumptions rather than relying on a single example. When code fails, read the error message, isolate the smallest failing case and change one thing at a time.

HANDBOOK

Junior Software Developer

Grade XII – IT-ITeS Vocational Education

RDBMS • Advanced Python • Software Engineering • Emerging Technologies

A structured textbook for database practice, software development and responsible technology use.

Prepared as a learning-focused companion based on the supplied Junior Software Developer Grade XII study material.

Contents

1 Relational Database Foundations **1**
1.1 Data and information 1
1.2 File systems and databases 1
1.3 Tables, keys and constraints 1
1.4 Database models 2

2 SQL: Defining and Manipulating Data **3**
2.1 SQL command families 3
2.2 Create, insert and retrieve 3
2.3 Joins and transactions 4

3 SQL Functions and Relational Design **5**

4 Python Data Structures: Stack and Queue **6**

5 Exception Handling in Python **7**

6 File Handling in Python **8**

7 NumPy Arrays **9**

8 Pandas and Series **10**

9 Visualisation with Matplotlib **11**

10 Connecting Python with MySQL **12**

11 Software Engineering Concepts **13**

12 Software Development Life Cycle **14**

13 Software Testing and Maintenance **15**

14 Emerging Technologies **16**

15 Social Impact, Ethics and Cyber Safety **18**

16 Integrated Practical Project **19**

Chapter 1

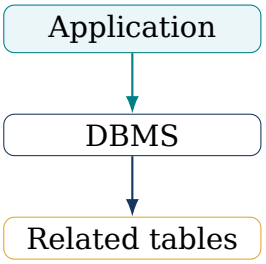
Relational Database Foundations

1.1 Data and information

Data are recorded facts such as student numbers, prices, dates or quantities. Information is data arranged to answer a question. A database is an organised collection of related data that can be searched and updated consistently. A database management system (DBMS) defines, stores, retrieves and protects data. A relational DBMS represents data through tables of rows and columns.

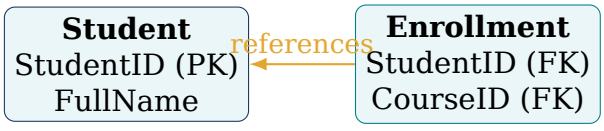
1.2 File systems and databases

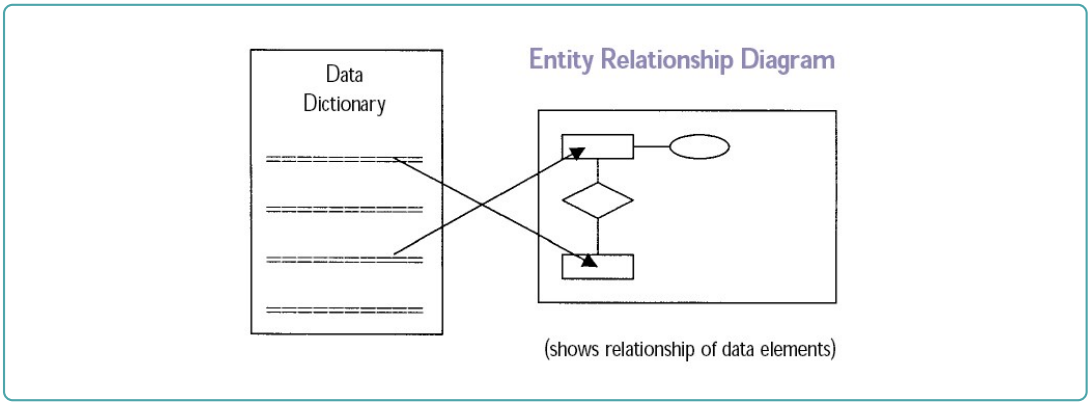
Independent files may duplicate values and become inconsistent when only some copies are updated. A DBMS supports controlled access, validation, shared data, backup and query-based retrieval. These benefits depend on sound design and configuration.



1.3 Tables, keys and constraints

A relation is a table; a tuple is a row; an attribute is a column; a domain is the set of valid values; a schema is the planned structure; an instance is the data present at a particular time. A candidate key uniquely identifies a row. One candidate becomes the primary key. A composite key combines attributes. A foreign key references a key in another table. Constraints such as NOT NULL, UNIQUE and CHECK express rules close to the data.





An ER diagram maps entities and the relationships that become tables and key links in a relational database. Source and licence are listed in IMAGE-CREDITS.txt.

1.4 Database models

Relational databases use related tables. Document stores hold document-like records; key-value stores map keys to values; graph databases emphasise relationships. Select a model based on data shape, consistency needs and query patterns.

Review Questions

1. Explain data versus information.
2. Give four limitations of independent file-based records.
3. Define relation, tuple, attribute, schema and instance.
4. Distinguish primary and foreign keys.
5. Give one relational database use case.

Chapter 2

SQL: Defining and Manipulating Data

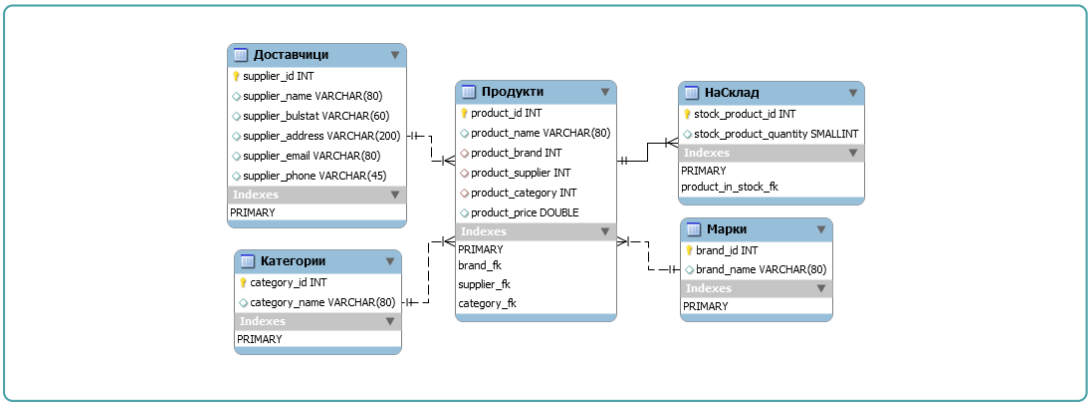
2.1 SQL command families

SQL defines database structures, changes records, retrieves information and controls access. DDL includes CREATE, ALTER and DROP; DML includes INSERT, UPDATE and DELETE; SELECT retrieves records; DCL manages permissions; TCL includes COMMIT and ROLLBACK.

2.2 Create, insert and retrieve

```
CREATE DATABASE training;
USE training;
CREATE TABLE Student (
  StudentID INT PRIMARY KEY,
  FullName VARCHAR(80) NOT NULL,
  ClassLevel TINYINT NOT NULL,
  Email VARCHAR(120) UNIQUE
);
INSERT INTO Student (StudentID, FullName, ClassLevel)
VALUES (1204, 'Anu Rao', 12);
SELECT StudentID, FullName
FROM Student
WHERE ClassLevel = 12
ORDER BY FullName;
```

Choose types deliberately. Use date types for dates, suitable text types for names and exact decimal types for currency. WHERE filters rows; ORDER BY sorts; GROUP BY forms groups; HAVING filters groups after aggregation.



The diagram illustrates how a database design organises related records and supports consistent retrieval. Source and licence are listed in IMAGE-CREDITS.txt.

```
SELECT ClassLevel, COUNT(*) AS StudentCount
FROM Student
GROUP BY ClassLevel
HAVING COUNT(*) >= 5
ORDER BY StudentCount DESC;
```

2.3 Joins and transactions

An inner join returns matching rows; a left join retains all rows from the left table. A transaction groups changes into one logical unit. ACID describes atomicity, consistency, isolation and durability. Check the condition of UPDATE or DELETE with SELECT first. Use parameterised queries in applications instead of concatenating untrusted input.

A code editor is a practical place to inspect SQL and application logic. Image source and licence are listed in IMAGE-CREDITS.txt.

Review Questions

1. Explain DDL and DML with examples.
2. Write a query to retrieve Grade XII students alphabetically.
3. Distinguish WHERE and HAVING.
4. What is a join?
5. Why are parameterised queries important?

Chapter 3

SQL Functions and Relational Design

Single-row functions transform values per row. String functions change case or measure text; numeric functions round values; date functions work with calendar values. Aggregate functions such as COUNT, SUM, AVG, MIN and MAX summarise groups. COUNT(column) generally excludes null values, while COUNT(*) counts rows.

```
SELECT UPPER(FullName) AS DisplayName,
       CHAR_LENGTH(FullName) AS NameLength
FROM Student;
```

Normalisation reduces duplication and update anomalies. First normal form expects atomic values and no repeating groups. Second normal form removes partial dependencies on part of a composite key. Third normal form removes inappropriate transitive dependencies among non-key attributes.

If an order table repeats a supplier address on every row, corrections require many updates. A separate Supplier table referenced by a key reduces duplication.

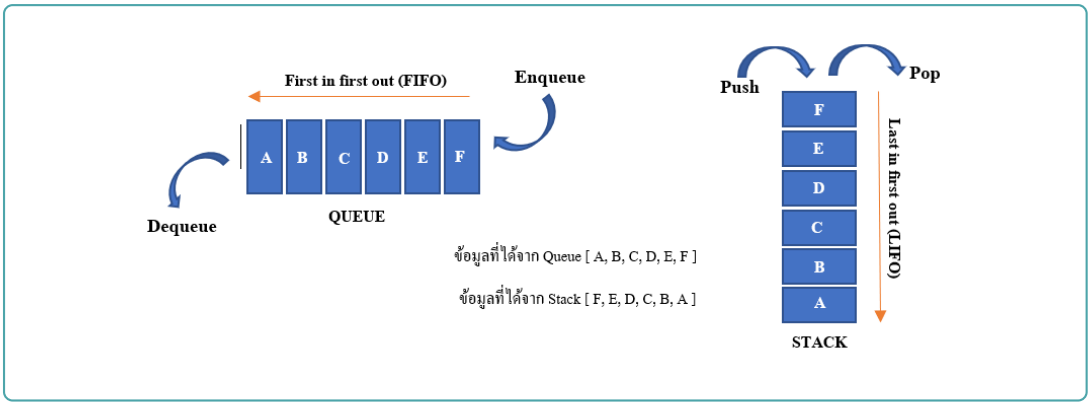
Review Questions

1. Distinguish single-row and aggregate functions.
2. Why can COUNT(*) differ from COUNT(Email)?
3. What is an update anomaly?
4. Explain normalisation.
5. List the main database design steps.

Chapter 4

Python Data Structures: Stack and Queue

A stack follows last-in, first-out (LIFO); the last item added is removed first. A queue follows first-in, first-out (FIFO); the earliest item is removed first. Stacks suit undo histories and function calls; queues suit print jobs and service requests.



The structures differ in the order items leave the collection. Source and licence are listed in IMAGE-CREDITS.txt.

```
from collections import deque
stack = []
stack.append("first")
stack.append("second")
print(stack.pop())        # second
queue = deque(["first", "second"])
queue.append("third")
print(queue.popleft())    # first
```

Use deque for efficient queue operations. Test empty structures and decide how to handle attempts to remove an item when none exists.

Review Questions

1. Define LIFO and FIFO.
2. Give two uses of each structure.
3. Why is deque useful for queues?
4. How should an empty-stack pop be handled?

Chapter 5

Exception Handling in Python

A syntax error prevents parsing; a runtime exception occurs during execution; a logic error produces an incorrect result even though the program runs. Use try for risky operations, except for expected exception types, else for success-only actions and finally for cleanup.

```
raw = input("Enter a whole number: ")
try:
    value = int(raw)
except ValueError:
    print("Please enter a valid whole number.")
else:
    print("Double:", value * 2)
finally:
    print("Input attempt finished.")
```

Catch the narrowest useful exception. Avoid broad handlers that hide defects. Raise an exception when an input violates a function's documented contract.

Review Questions

1. Distinguish syntax, runtime and logic errors.
2. Explain else and finally.
3. Why avoid bare except clauses?
4. Write an example handling ValueError.

Chapter 6

File Handling in Python

Text files store characters; binary files store bytes. Modes include r for reading, w for replacing content, a for appending and x for exclusive creation. The with statement closes files reliably after a block.

```
with open("notes.txt", "w", encoding="utf-8") as file:
    file.write("Python file handling\n")
with open("notes.txt", "r", encoding="utf-8") as file:
    print(file.read())
```

CSV stores rows and columns; Python's csv module handles quoted values. JSON represents objects, arrays and primitive values and is commonly used by APIs. Validate user-provided files, avoid overwriting valuable data without backups and never commit secrets to source control.

```
import csv
with open("marks.csv", newline="", encoding="utf-8") as file:
    for row in csv.DictReader(file):
        print(row.get("StudentName"), row.get("Marks"))
```

Review Questions

1. Distinguish w and a modes.
2. Why is with open preferred?
3. Why use the csv module?
4. Give two file-safety precautions.

Chapter 7

NumPy Arrays

NumPy supports multidimensional arrays and numerical operations. Arrays usually contain values of a common type and support vectorised calculations.

```
import numpy as np
marks = np.array([72, 81, 69, 94])
print(marks.mean())
print(marks.max())
print(marks[1:3])
print(marks.shape, marks.ndim, marks.dtype)
```

Shape describes dimensions, ndim gives the number of dimensions and dtype reports the element type. Vectorisation applies operations to whole arrays. Broadcasting allows compatible shapes to work together; check shapes to avoid unintended calculations.

Review Questions

1. Compare lists and NumPy arrays.
2. Explain shape, ndim and dtype.
3. What is vectorisation?
4. Why check array shapes?

Chapter 8

Pandas and Series

Pandas provides labelled structures for cleaning and analysing tabular data. A Series is a one-dimensional labelled sequence; a DataFrame is a table-like structure.

```
import pandas as pd
data = pd.DataFrame({
    "student": ["Asha", "Ravi", "Meena"],
    "marks": [82, 76, 91]
})
print(data.head())
print(data["marks"].mean())
print(data.describe())
```

Use column names to select values and Boolean conditions to filter rows. Inspect missing values with isna(); remove or fill them only when justified. Grouping summarises categories. Before merging, check key uniqueness because many-to-many joins may multiply rows.

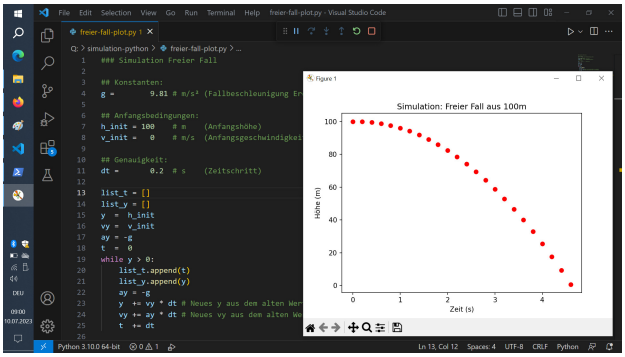
Review Questions

1. Distinguish Series and DataFrame.
2. How can missing values be inspected?
3. Why might a merge increase row count?
4. Describe a simple marks analysis.

Chapter 9

Visualisation with Matplotlib

Bar charts compare categories; line charts show change over an ordered scale; scatter plots explore numeric relationships; histograms show distributions.



A programming environment can connect calculations with visual interpretation; see IMAGE-CREDITS.txt.

```python
import matplotlib.pyplot as plt
subjects = ["SQL", "Python", "Networks"]
scores = [78, 86, 74]
plt.bar(subjects, scores)
plt.title("Practice assessment scores")
plt.xlabel("Subject")
plt.ylabel("Score")
plt.ylim(0, 100)
plt.tight_layout()
plt.show()
```

Label axes and units, use readable scales and explain the data source. Avoid decoration that distorts comparisons or implies unsupported precision.

Review Questions

1. Match chart types to questions.
2. Why label axes?
3. Name two ways charts can mislead.
4. Outline steps to create a bar chart.

Chapter 10

Connecting Python with MySQL

A database connector lets Python send SQL statements and receive results. A typical workflow loads configuration securely, opens a connection, creates a cursor, executes parameterised statements, commits successful changes, rolls back failures and closes resources.

```
# Illustrative pattern; configure a MySQL connector first.
sql = "SELECT StudentID, FullName FROM Student WHERE ClassLevel = %s"
cursor.execute(sql, (12,))
rows = cursor.fetchall()
for row in rows:
    print(row)
```

Placeholder syntax depends on the connector. Keep credentials outside source code, grant least privilege, handle network failures and avoid exposing sensitive records in logs.

Review Questions

1. Describe the Python-to-MySQL workflow.
2. Why use parameterised statements?
3. Explain commit and rollback.
4. Give three ways to protect credentials.

Chapter 11

Software Engineering Concepts

Software engineering applies systematic methods to build, test, release and maintain software. Functional requirements describe actions the system must perform. Non-functional requirements describe qualities such as response time, security, availability and accessibility. Requirements should be testable and unambiguous. Modularity divides a system into components with clear responsibilities. High cohesion means a component's tasks belong together; low coupling limits unnecessary dependencies. Quality attributes include reliability, maintainability, portability, scalability, usability and security.

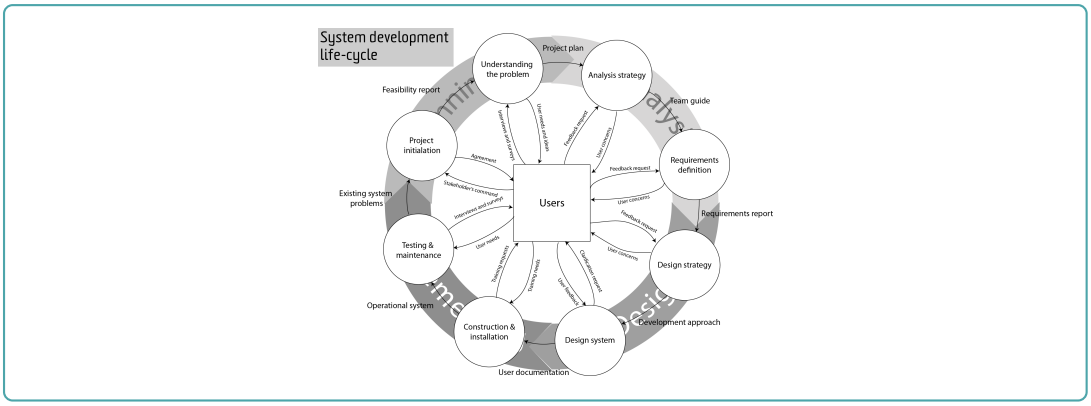
Review Questions

1. Distinguish functional and non-functional requirements.
2. Define cohesion and coupling.
3. List five quality attributes.
4. Rewrite a vague requirement measurably.

Chapter 12

Software Development Life Cycle

Typical activities include planning, requirements analysis, design, implementation, testing, deployment and maintenance. Sequential and iterative approaches organise these activities differently; feedback may send work back to earlier stages.



A published SDLC diagram illustrates the stages from planning and requirements through design, implementation, testing, deployment and maintenance. Image by John Yeung, Wikimedia Commons, CC BY-SA 3.0.

Plan-driven methods suit some stable or constrained projects. Iterative methods deliver increments and incorporate feedback. Version control records changes; meaningful commits and code review support collaboration. Risk planning records likelihood, impact, owners and mitigation actions.

Review Questions

1. List typical SDLC activities.
2. Compare plan-driven and iterative approaches.
3. Why is code review useful?
4. Describe a project risk and mitigation.

Chapter 13

Software Testing and Maintenance

Unit tests check small components; integration tests check interactions; system tests assess the full product; acceptance tests assess user needs. Regression tests check that a change has not broken existing behaviour. A test case defines inputs, preconditions, actions and expected results. Include typical values, boundaries, invalid inputs and failure conditions. Debug systematically: reproduce the issue, gather evidence, narrow the cause, make a fix and rerun relevant tests. Deployment planning includes configuration, database migration, monitoring and rollback.

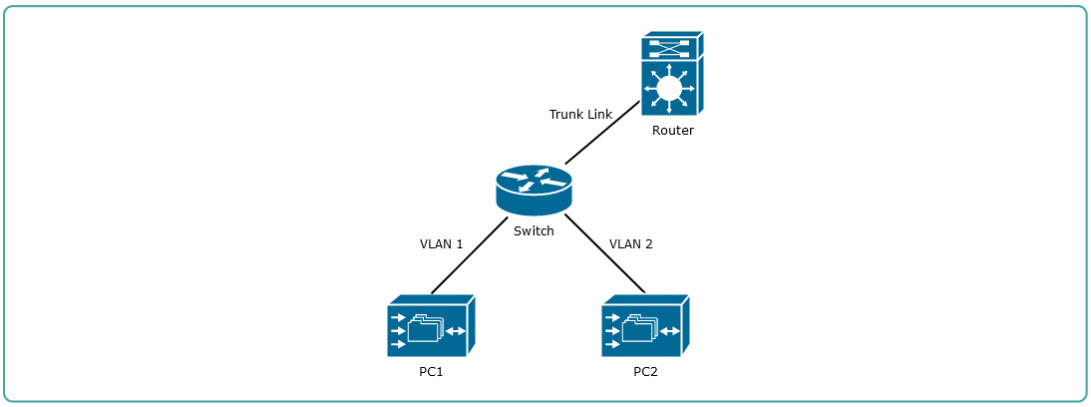
Review Questions

1. Explain unit, integration, system and acceptance testing.
2. What is regression testing?
3. Why test boundary values?
4. Outline a safe debugging process.
5. What belongs in a rollback plan?

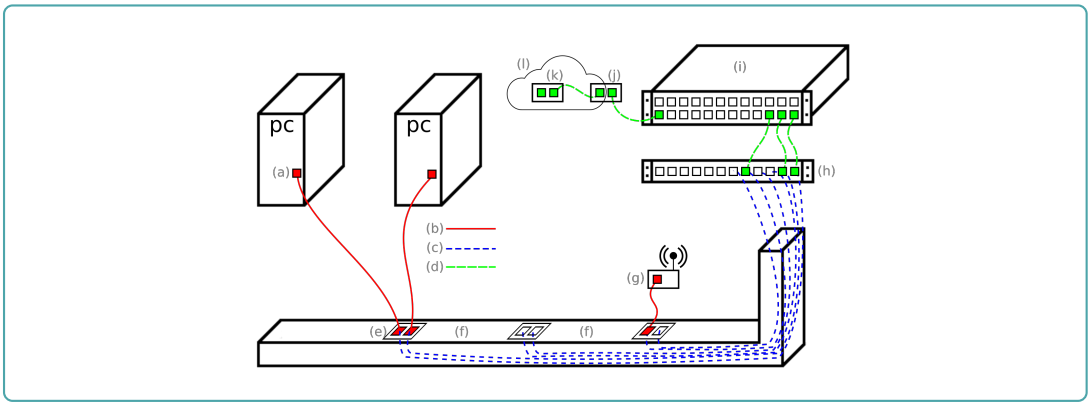
Chapter 14

Emerging Technologies

AI includes systems for perception, language, prediction or planning. Machine learning uses data to fit patterns. Generative AI produces text, images, code or audio. Outputs can be inaccurate or biased; evaluate them and keep human oversight for consequential decisions. Cloud computing offers on-demand computing, storage, databases and software. IaaS provides infrastructure, PaaS a managed platform and SaaS a ready-to-use application. Networks and routing provide the connectivity between clients, services and data centres.



Network segmentation can separate workloads and reduce unnecessary traffic; source and licence are listed in IMAGE-CREDITS.txt. Internet of Things devices combine sensors, software and connectivity. Edge computing processes data near its source, potentially reducing latency.



A network diagram helps explain how devices and infrastructure are connected; source and licence are listed in IMAGE-CREDITS.txt. Cloud and desktop environments should be configured with access controls, updates and recovery plans; an attractive interface alone does not establish security or reliability. Blockchain is a shared ledger maintained using a consensus mechanism; it is not automatically private or suitable for every database problem. Augmented reality overlays digital information on the physical world; virtual reality immerses users in a simulated environment. Robotic process automation performs repetitive rule-based tasks but needs controls for exceptions and auditability.

Review Questions

1. Distinguish AI, ML and generative AI.
2. Compare IaaS, PaaS and SaaS.
3. When might edge computing help?
4. Give one limitation of blockchain.
5. Name a responsible AI safeguard.

Chapter 15

Social Impact, Ethics and Cyber Safety

Technology can expand access but may exclude people due to cost, connectivity, language, disability or limited digital skills. Accessible design considers keyboard use, contrast, captions and assistive technology. Collect only necessary personal data, restrict access and protect it in storage and transfer. Cyber safety includes unique passwords, multifactor authentication, timely updates, backups and caution with unexpected links. Phishing attempts to trick people into disclosing information or running unsafe actions. Respect copyright, software licences, privacy and attribution. Consider energy use, repairability, device lifespan and responsible e-waste disposal.

Review Questions

1. Define digital inclusion.
2. List five principles for responsible data handling.
3. Give common phishing warning signs.
4. Why do software licences and image credits matter?
5. Suggest three sustainable computing practices.

Chapter 16

Integrated Practical Project

Design a small student learning-record system using fictional data only. Write functional requirements for adding a student, recording a score and producing a class summary. Add non-functional requirements for access control, clarity and recovery. Design tables with primary and foreign keys, create test data and query averages and counts. Use Python to process results and create a labelled chart. Separate database access, validation and presentation into functions. Prepare normal, boundary and invalid-input tests, record expected and actual results, document setup and limitations, and identify privacy risks.

Review Questions

1. Draw a basic entity relationship model.
2. Write two functional and two non-functional requirements.
3. Propose five test cases.
4. Explain how Python and SQL work together.
5. Identify two privacy risks and mitigations.